Structure Meshing for CFD

Technical Report · March 2023

DOI: 10.13140/RG.2.2.34018.48323/7

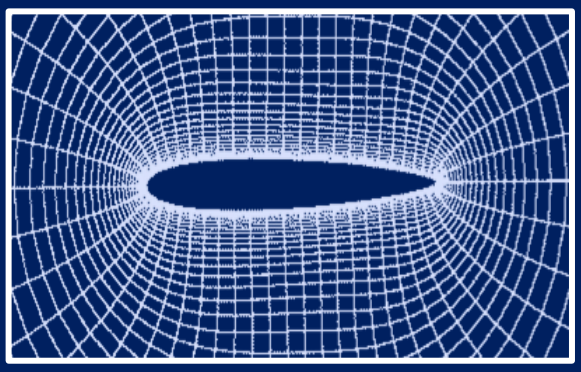
1 author:

Ideen Sadrehaghighi
CFD Open Series
91 PUBLICATIONS **305** CITATIONS

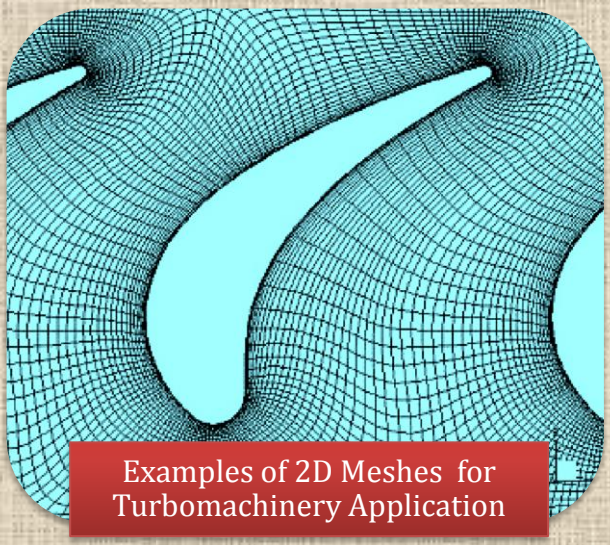
SEE PROFILE

CFD Open Series
Patch 2.40

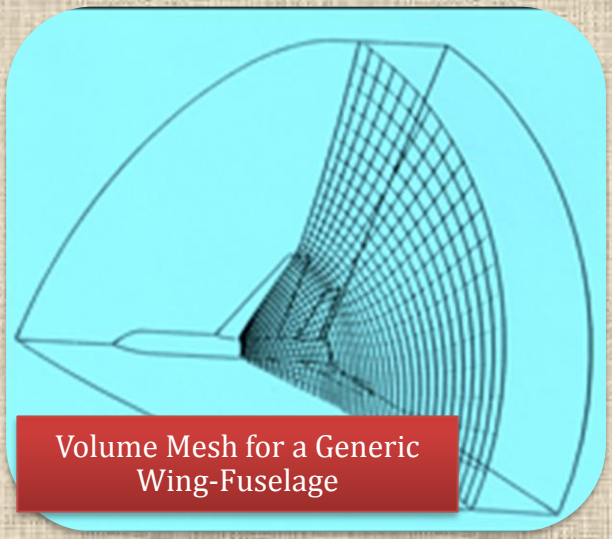
Structure Meshing for CFD



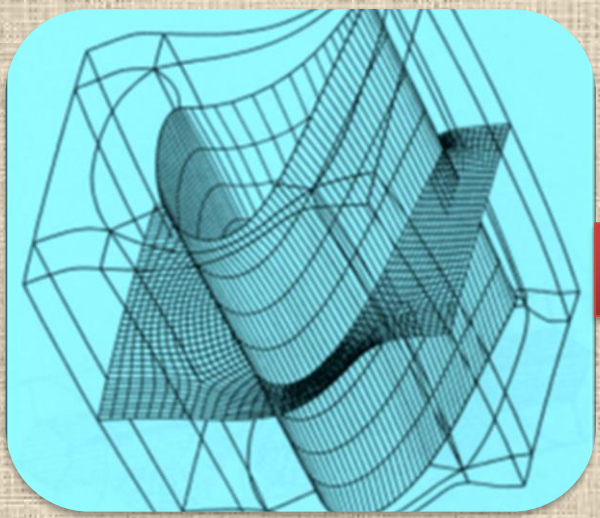
Edited & Adapted by : Ideen Sadrehaghighi



Examples of 2D Meshes for Turbomachinery Application



Volume Mesh for a Generic Wing-Fuselage



Multi-Block Meshing for a Turbine Blade (Courtsy GridPro)

ANNAPOLIS, MD

Contents

List of Tables

List of Figures

1 Introduction

1.1 Definition of Mesh vs. Grid

According to [*Doug Lipinski*] a researcher in applied math and CFD, there is no firm distinction between ***grids*** and ***meshes***. However, they are often used in slightly different ways. The following definitions are more guidelines of common usage than actual rules and you may hear people use them interchangeably in many cases***. Grids are typically a set of simulation elements that have a well-defined structure to their alignment with square or rectangular grids being the most prototypical. Meshes are often more general. They may be unstructured and use various shapes of elements, sometimes even mixing elements of different types in the same mesh***[1]***.***

The partial differential equations that govern fluid flow and heat transfer are not usually amenable to analytical solutions, except for very simple cases. Therefore, in order to analyze fluid flows, flow domains are split into smaller subdomains (made up of geometric primitives like hexahedra and tetrahedra in 3D and quadrilaterals and triangles in 2D). The governing equations are then discretized and solved inside each of these subdomains. Typically, one of three methods is used to solve the approximate version of the system of equations: finite volumes, finite elements, or finite differences. Care must be taken to ensure proper continuity of solution across the common interfaces between two subdomains, so that the approximate solutions inside various portions can be put together to give a complete picture of fluid flow in the entire domain. The subdomains are often called elements or cells, and the collection of all elements or cells is called a mesh or grid. The origin of the term mesh (or grid) goes back to early days of CFD when most analyses were 2D in nature. For 2D analyses, a domain split into elements resembles a wire mesh, hence the name. An example of a 2D analysis domain (flow over a backward facing step) and its mesh are shown in **Figure 1.1.1**. The process of obtaining an appropriate mesh (or grid) is termed mesh generation (or grid generation), and has long been considered a bottleneck in the analysis process due to the lack of a fully automatic mesh generation procedure. Specialized software programs have been developed for the purpose of mesh and grid generation, and access to a good software package and expertise in using this software are vital to the success of a modeling effort.

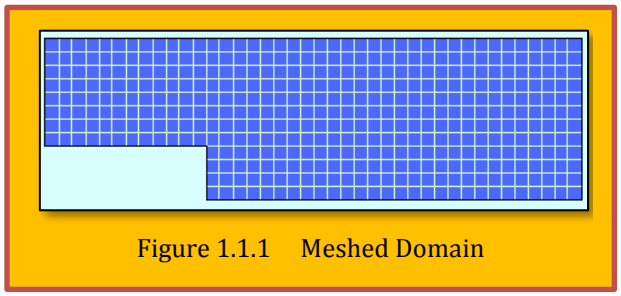


Figure 1.1.1 Meshed Domain

1.2 The Black Box Dilemma

1.2.1 Trust the Mesh Generated by the Software, or Take a Proactive Approach?

Are you the type who likes to take a peek inside the black box to see how it works? Or are you one who's willing to put your faith in the black box? Argued [Kenneth Wong][2] of ***Digital Engineering's***. The answer to that may offer clues to the type of meshing applications that appeal to you. But that's not the only factor. Your own finite element analysis (FEA) skills also play a role. Most simulation programs aimed at design engineers offer fully or almost fully automated meshing. In other words, the software makes most or all of the mesh-related decisions required. Your part may be limited to selecting the desired resolution or the level of details fine meshing (high resolution, takes more time, but more accurate) or coarse meshing (low resolution, takes less time, but more approximations involved).

There are good reasons to keep the meshing process hidden inside the black box, as it were. It takes a lot of experience and expertise (perhaps even a Ph.D.) to understand the difference between, say, a ***hexahedral mesh*** and a ***tetrahedral mesh***; or tri elements and quad elements. It takes considerable simulation runs to know what type of meshing methods work well for a particular set of solid

geometry. It requires yet another level of wisdom to know how to manually readjust the software-generated meshes to more accurately account for the problematic curvatures, corners and joints in your geometry. These are beyond the scope of what most design engineers do. Therefore, many argue presenting a design engineer with a menu of these choices is counterproductive. On the other hand, expert users with a lot of analysis experience know the correlations between mesh types and accuracy, so they may want to get more involved in the meshing process. For this reason, high-end analysis software usually offers much more knobs and dials in the meshing process. Depriving expert users of these choices would force them to accept what they know to be unacceptable approximations. To navigate between the two different approaches, you need at least some understanding of how meshing works, automated or manual.

1.2.2 Not All the Meshes Created Equal

According to [*Abdullah Karimi*], CFD analyst for ***Southland Industries***, uses fluid dynamics programs to examine airflow and heat distribution to develop the best residential heating solutions for his company's clients. Via an online blog by Southland Industries, Karimi penned an article titled "*How Not to Mesh Up: Six Mistakes to Avoid When Generating CFD Grids*". His first tip: Never use the first iteration of automatically generated mesh. "I've realized even some people with Ph.D.'s don't have a good grasp on meshing," he says. "People say, garbage in, garbage out. I say, good mesh equals good results. But the vast majority of the times I've seen the [software's] automatically generated initial mesh is too coarse. The mesh may not even work, and if it does, the result may not be accurate." If the automatically generated mesh significantly distorts the original geometry's prominent characteristics—such as rounded corners, sharp angles and smooth curves it may be a sign that the mesh needs manual intervention in those specific regions. "You should at least take a look at the mesh. You can check to see if there are sudden size transitions, aspect ratio for skewness and triangular distortions. Just by visually inspecting the mesh, you can get a good idea if this may or may not work for your problem," says *Karimi*.

In his article, *Karimi* advises, "Don't hit 'Run' without a mesh quality inspection. Depending on the robustness of the solution scheme, this could cause serious issues like straightaway divergence of the solution ... There are several quality metrics that need attention depending on mesh type and flow problem. Some of these metrics include skewness, aspect ratio, orthogonality [and] negative volume."

1.2.3 Some Commercial Meshing

With its designer friendly ***Altair Inspire*** (previously ***solidThinking Inspire***) and expert-centric Altair ***HyperMesh*** software, Altair offers different approaches to meshing. "In Inspire, meshing is mostly hidden from the user," explains Paul Eder, senior director of ***HyperWorks*** shell meshing, CAD and geometry at Altair. "The users choose to solve either in the first order [which prioritizes speed] or second order [which prioritizes accuracy]." By contrast, in ***HyperMesh***, "We expose a lot more knobs and dials, because it's for advanced users who understand the type of meshes they want to generate," he adds. A similar strategy is seen in ***ANSYS*** software offerings. "Two of our products, ***ANSYS Forte*** and ***Discovery Live***, provide a fully automated meshing experience," says Bill Kulp, lead product marketing manager for Fluids at ***ANSYS***. "***ANSYS Discovery Live*** provides instantaneous 3D simulation so there is no time to make a mesh. On the other hand, our [general-purpose CFD package] ***ANSYS Fluent*** users need to solve a wide variety of fluid flow problems that can be most accurately approached by optimizing the mesh for the task at hand."

"Push-button automated meshing is our goal because we want to take this time-consuming job away from the engineers so they can concentrate on the innovation and optimization of their products," adds [Andy Wade], lead application engineer at ***ANSYS***. "Automated meshing will enable AI and digital twins to run simulations in the future and so this area is becoming the focus." In theory, design engineers and simulation analysts could use different products, but in reality, some design engineers have sufficient expertise to make critical meshing decisions; and some analysis experts prefer the efficiency of automated or semi-automated meshing. So even with different products, satisfying both

crowds is a difficult balancing act for vendors. Though the meshing process is mostly kept in the background in ***Altair Inspire***, "If you're an advanced user and want to see the meshes, you have the option to," says Eder. "At the same time, we also offer automation in ***HyperMesh,*** because even some expert users want the same ease of use seen in Inspire." "Tools such as ***ANSYS Discovery*** Live takes the meshing away completely from the user, whereas Discovery AIM features automatic physics-aware meshing, so the user can allow the product to do the hard work but if they want to see the mesh and tweak it they can take control," says Wade. **Figure 1.2.1** shows meshes were created using ***ANSYS Mosaic-enabled Poly-Hex*** core meshing that automatically combines disparate meshes with polyhedral elements for fast, accurate flow resolution. ANSYS Fluent provides Mosaic-enabled meshing as part of a single-window, task-based workflow. Image courtesy of *Sheffield Hallam University, Centre for Sports Engineering Research; and ANSYS.*

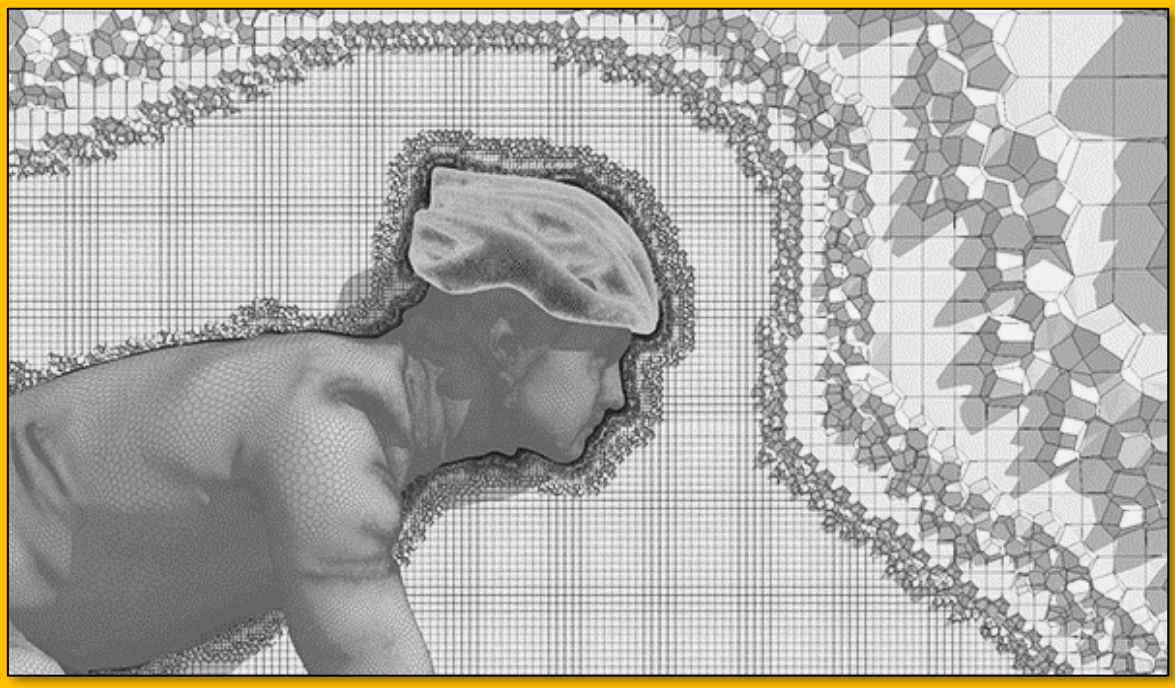


Figure 1.2.1 Meshes Created using ANSYS Mosaic-Enabled Poly-Hex Core Meshing - Courtesy of Sheffield Hallam University

1.2.4 Regional Meshing

The relatively new startup ***OnScale*** recently began offering on-demand multi-physics simulation from the browser. Some firms like Rescale offer high-performance computing (HPC) resources needed to run simulation, but not the software. By contrast, ***OnScale*** offers both the hardware and the multi-physics solver required to process jobs. "We offer automatic meshing as well as user-defined meshing. Users can define the level of fidelity desired," explains Gerald Harvey, ***OnScale's*** founder and VP of engineering. "***OnScale*** gives you the ability to refine the grid and apply finer meshes in specific regions." Not every corner, section or region in your geometry needs fine meshing. With simple geometry, a coarse mesh with fewer elements may suffice. But in certain regions where curvature, contact and joints create complex stress concentrations or flow patterns, a finer mesh (simply put, a higher number of meshes to cover the area) is warranted. Advanced simulation programs usually offer tools to specify how to treat these regions. Even in programs that target design engineers, some tools may be available to treat these regions differently." In ***Altair FEA products*** like ***SimLab***, you can perform automatic local mesh refinement," says Eder. "So you can run

an analysis, review the results, then automatically refine the mesh in areas of high strain energy error density for subsequent runs. In [expert-targeted] ***HyperMesh***, you also have many more manual mesh refinement options."

1.2.5 Simulation Cost

OnScale's Harvey suggests running a mesh study to understand the correlation between the stress effects and the mesh types and mesh density chosen. This can offer clues on how meshing affects the FEA results. "Every engineer should conduct a mesh convergence study test the meshes with some key performance indicators (KPIs) to find a happy medium," says Harvey. "Suppose you're looking at the design of a bracket. Then look at how the different meshes affect the bend angle of the bracket, for example." Calculating simulation cost is complex, in part due to the mix of licensing policies in the market. But fundamentally, two parameters are involved: the time it takes and the hardware it uses. The need to find simplified meshes (as simple as possible without infringing on the accuracy of results) largely stems from the desire to keep these two parameters as low as possible. "If you have a simple solid part and you put 3D meshes on it, it takes more times than necessary to run," notes Eder. In such a case, running simulation in a 2D cross-section of the geometry may be much more efficient. "And think of how many iterations you plan to run, because you'll be paying that penalty for every single run," he adds.

ANSYS' Wade points out that most solvers prefer hexahedral elements or quad surface mesh because "they fill the space very efficiently and using such elements when transient or explicit analysis is required can give massive gains in solve times (minimizing CPU effort for calculations). Hex elements can follow the flow direction better as well, which has some accuracy benefits. Tetrahedra, polys and other unstructured methods are very popular because they don't require the decomposition (chopping up) of the space like a hex mesh; as a result, they are excellent for automation and really minimize manual effort." Another tip from Karimi's article: "Don't fill the domain with a ridiculous number of tetrahedrons. So many times, I see meshing engineers filling up their CFD [computational fluid dynamics] domains [the target region for fluid analysis] with a large number of tetrahedrons and then struggling to get simulation results on time." Certain programs are equipped to make the mesh selection easier. "With ***OnScale***, you can conduct a study on a sweep of design, with mesh being one of the variables," Harvey points out. "In ***OnScale***, the run wouldn't cost you significantly, because it would be a one-off cost. And the payback is well worth it."

1.2.6 Physics vs. Mesh

Choosing the right kind of mesh, applying the right density to critical regions and selecting the right kind of coarseness or looseness affect the accuracy and speed of the simulation job. That is an exercise in tradeoffs, so there's no black-and-white answer. "Meshing is always an exercise in tradeoffs in the quality of the mesh versus the speed of the solution," says Karimi. "If you just want to see if a part will stand up to stresses and daily beating over time, and you're not looking at the lowest level of details but at a high level of generality, then getting your physics correct is more important than the mesh," says Eder. That means, at the general concept design level, the loads and boundary conditions—such as temperature, forces and direction of the forces may be more important than the type of meshes selected.

1.2.7 Types of Domain (Simple or Multiply Connected)

We divide the possible inputs first according to dimension, two or three (Bern & Plassmann)[1]. We distinguish four types of planar domains, as shown in **Figure 1.2.2**. For us, a simple *polygon* includes both boundary and interior. A polygon with holes is a simple polygon minus the interiors of some other simple polygons ; its boundary has more than one connected component. A *multiple domain* is more general still, allowing internal boundaries; in fact, such a domain may be any planar straight-

[1] Marshall Bern and Paul Plassmann, "*Mesh Generation*", Xerox Palo Alto Research Center, Palo Alto, and Mathematics and Computer Science Division, Argonne National Laboratory.

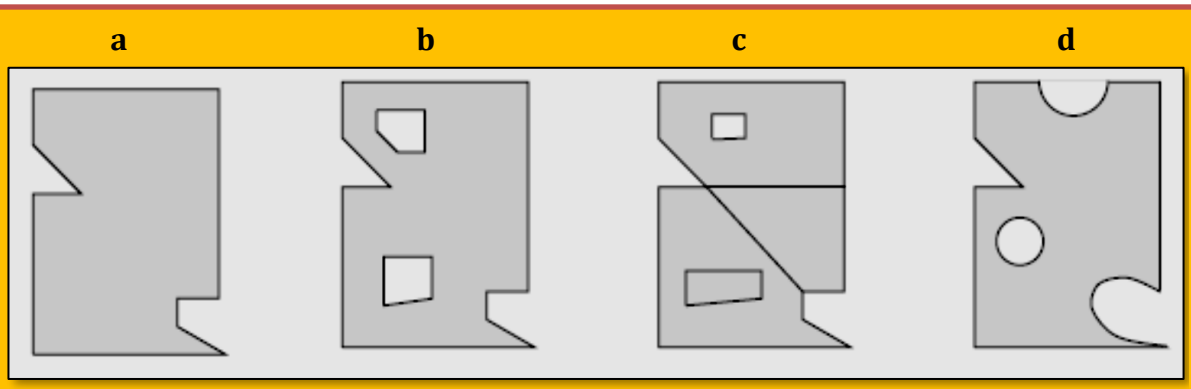


Figure 1.2.2 Types of 2D inputs : a) simple polygon , b) polygon with hole , c) multiple domain , d) curved domain - courtesy of (Bern & Plassmann)

line graph in which the infinite face is bounded by a simple cycle. *Multiple domains* model objects made from more than one material. *Curved* domains allow sides that are algebraic curves such as splines. As in the first three cases, collectively known as polygonal domains, curved domains may or may not include holes and internal boundaries.

Three-dimensional inputs have analogous types. A simple polyhedron is topologically equivalent to a ball. A general polyhedron may be multiply connected, meaning that it is topologically equivalent to a solid torus or some other higher genus solid; it may also have cavities, meaning that its boundary may have more than one connected component. We do assume ,however, that at every point on the boundary of a general polyhedron a sufficiently small sphere encloses one connected piece of the polyhedron's interior and one connected piece of its exterior.

Finally, there are multiple polyhedral domains—general polyhedral with internal boundaries—and three-dimensional curved domains, which typically have boundaries defined by *spline* patches.

1.2.8 Meshing Generalities

A pre-processing step for the computational field simulation is the discretization of the domain of interest and is called mesh generation. The process of mesh generation can be broadly classified into two categories based on the topology of the elements that fill the domain. These two basic categories are known as ***structured*** and ***unstructured*** meshes. The different types of meshes have their advantages and disadvantages in terms of both solution accuracy and the complexity of the mesh generation process. According to (Bern & Plassmann), a *structured* mesh is one in which all interior vertices are topologically alike. An *unstructured* mesh is one in which vertices may have arbitrarily varying local neighborhoods.

A ***structured*** mesh is defined as a set of hexahedral elements with an implicit connectivity of the points in the mesh. The structured mesh generation for complex geometries is a time-consuming task due to the possible need of breaking the domain manually into several blocks depending on the nature of the geometry. It also could be further distinguished in terms of cell spacing as ***uniform*** (equal spacing), or ***non-uniform*** (diverse cell spacing). (see **Figure** 1.2.3)[2].

An ***unstructured*** mesh is defined as a set of elements, commonly tetrahedrons, with an explicitly defined connectivity. The unstructured mesh generation process involves two basic steps: point creation and definition of connectivity between these points. Flexibility and automation make the unstructured mesh a favorable choice although solution accuracy may be relatively unfavorable compared to the structured mesh due to the presence of skewed elements in sensitive regions like

[2] M. Cruchaga, D. Celentano, and T. Tezduyar, "A moving Lagrangian interface technique for flow computations over fixed meshes", *Computer Methods in Applied Mechanics and Engineering*, 191 (2001) 525–543, http://dx.doi.org/10.1016/S0045-7825(01)00300-0

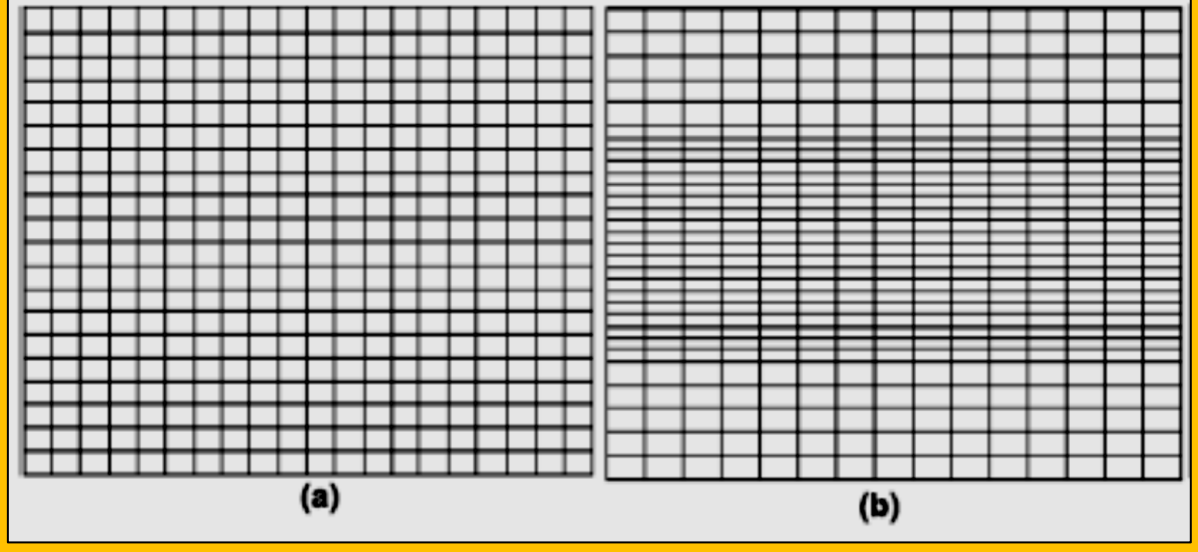


Figure 1.2.3 Fixed Meshes (a) Structured Uniform ; (b) Structured Non-uniform

boundary layers. (**Figure 1.2.4**). They also could be distinguish about cell spacing as uniform (equal spacing), non-uniform (diverse cell spacing), and unstructured.

In an attempt to combine the advantages of both structured and unstructured meshes, another approach in practice is ***hybrid*** mesh generation. A *hybrid* mesh is formed by a number of small structured meshes combined in an overall unstructured pattern (Bern & Plassmann). In a hybrid mesh, the viscous region is filled with prismatic or hexahedral cells while the rest of the domain is filled with tetrahedral cells. It has been observed that a hybrid mesh in viscous regions creates a lesser number of elements than a completely unstructured mesh with a similar resolution. This type of mesh has no restrictions on the number of edges or faces on a cell, which makes it extremely flexible for topological adaptation. It is given that unstructured mesh has an advantage over the structured mesh in handling complex geometries, mesh adaptation using local refinement and de-refinements, moving mesh capability by locally repairing the bad quality elements, and load balancing using appropriate graph partitioning algorithms. In the case of a non-matched block-to-

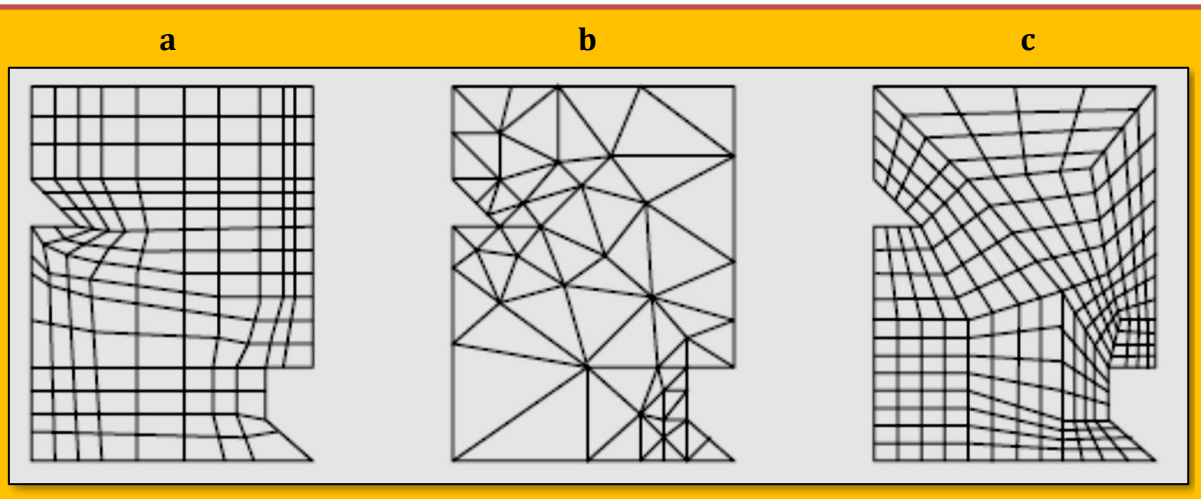


Figure 1.2.4 Type of Meshes : a) structured b) unstructured c) block structured
courtesy of (Bern & Plassmann)

block boundary, interpolation issues have to be handled properly to satisfy the conservation principles. However, the structured mesh has a better accuracy for viscous calculations due to the fact that it can handle cells with very high aspect ratio cells in the boundary layer[3]. Precipitate of most grid generation procedure can be summarized as **Figure 1.2.5** provided that everything goes according to plan.

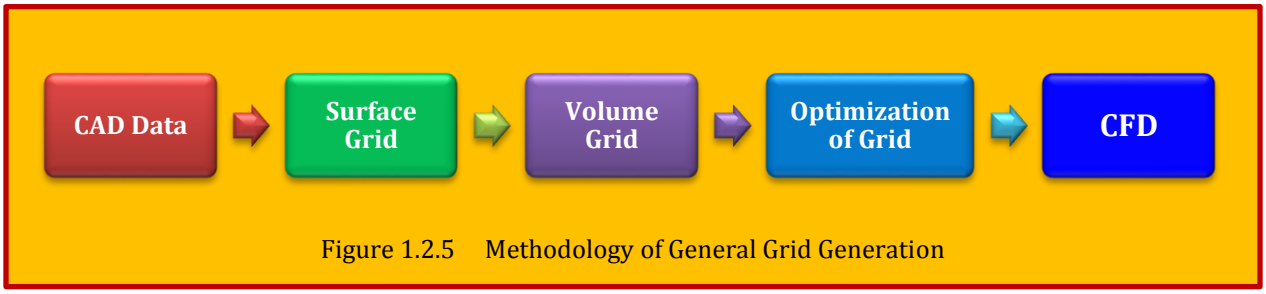


Figure 1.2.5 Methodology of General Grid Generation

1.3 Meshing Types [4]

Sometimes meshes are define in terms of :

- Cartesian
- Curvilinear (body fitted)

Meshes can be ***Cartesian*** or ***curvilinear (usually body-fitting)***. In the former, grid lines are always parallel to the coordinate axes. In the latter, coordinate surfaces are curved to fit boundaries. There is an alternative division into orthogonal and non-orthogonal grids. In orthogonal grids (for example, Cartesian or polar meshes) all grid lines cross at 90°. Some flows can be treated as axisymmetric, and in these cases, the flow equations can be expressed in terms of polar coordinates (r , θ), rather than Cartesian coordinates (x , y), with minor modifications (see **Figure 1.3.1**).

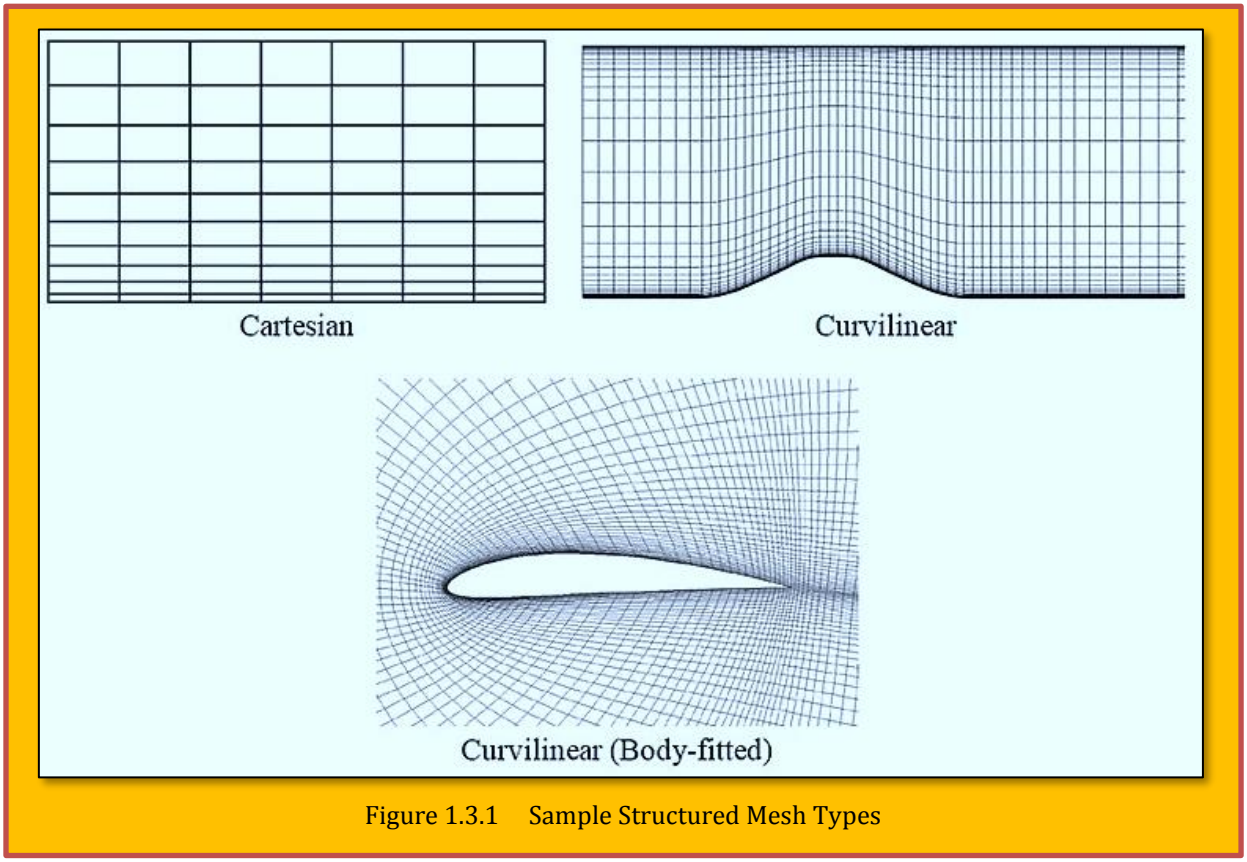


Figure 1.3.1 Sample Structured Mesh Types

1.3.1 Structured Meshes

- Matching
- Non-matching
- Chimera (composite)
- Quadrilateral (Hexahedral)

In multi-block structured grids the domain is decomposed into a small number of regions, in each of which the mesh is structured (i.e. cells can be indexed by (i , j , k)). A common arrangement is that grid lines match at the interface between two blocks, so that there are cell vertices that are common to two blocks i.e. matching cells. In some cases, the cell counts do not match at the interface i.e. non-matching cells. Non-matching cells (which could be 2 to 1, 3 to 2, ...) should be avoided as much as possible as they tend to increase the computational time. Some solvers also allow overlapping blocks

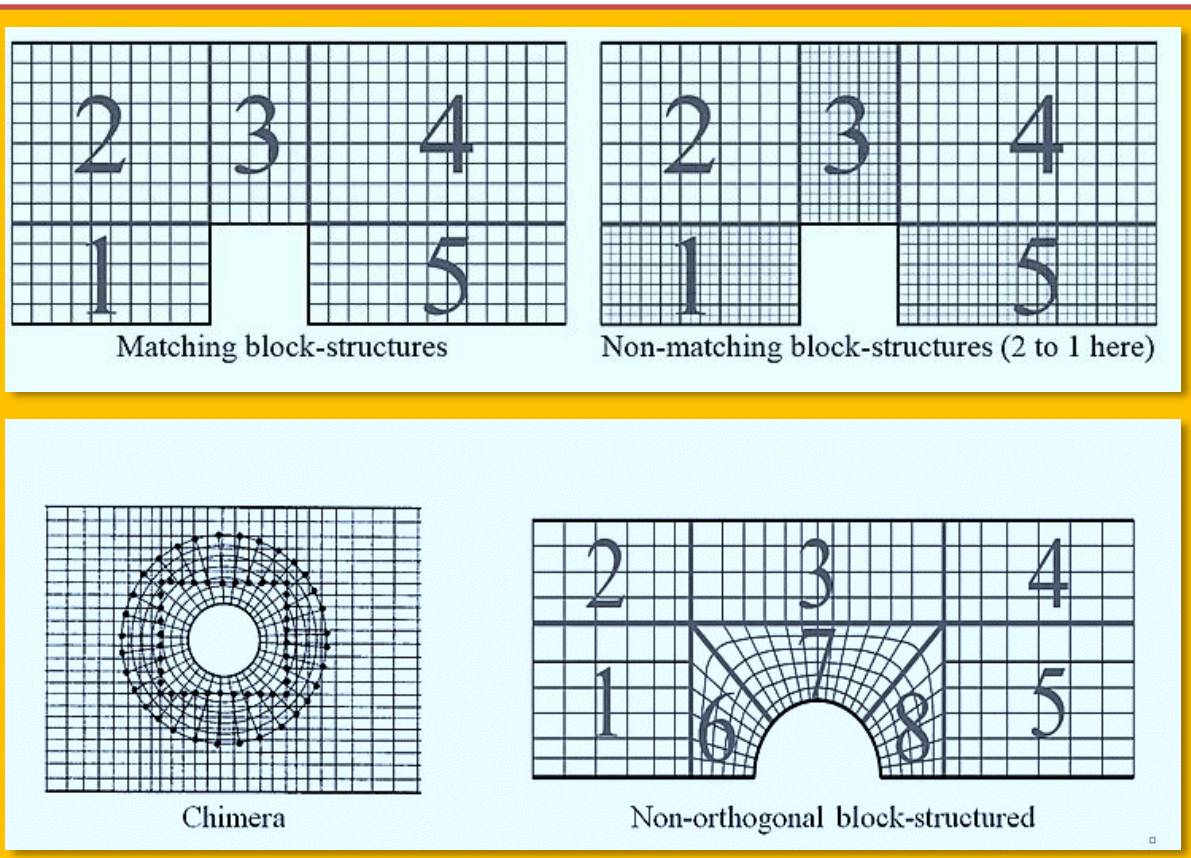


Figure 1.3.2 Multi-Blocking Terminologies

(i.e. chimera grids) where cell vertices do not align. Interpolation is then needed at the boundaries of blocks. Generally, multiple blocks are useful in maintaining a structured grid configuration around complex boundaries. There are no hard and fast rules, but it is generally desirable to avoid sharp changes in grid direction (which lead to lower accuracy) in important and rapidly changing regions of the flow, such as near solid boundaries. One should also strive to minimize the non-orthogonality

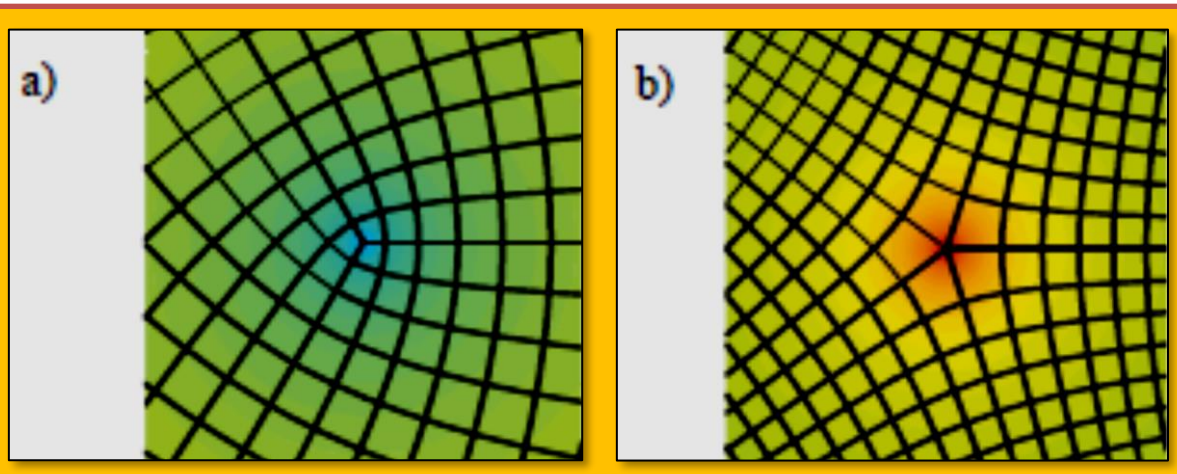


Figure 1.3.3 (a) a negative singularity; (b) a positive singularity. The scalar field shown is a harmonic function representing a continuum description of the quad mesh. It relates to the exponential of isotropic element sizes

of the grid. (**Figure 1.3.2**).

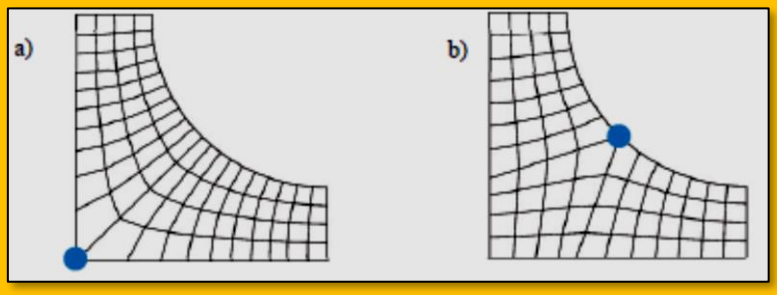


Figure 1.3.4 Singularities on the boundary of a 2D region. The number of elements incident on the boundary is different than that implied by the geometry

1.3.1.1 Singularities in Quad Meshes

Many of the key features of multi-block structured mesh generation are common to both quad and hex meshes, therefore quad meshes are described first for ease of explanation [Cecil G. Armstrong, Harold J. Fogg, Christopher M. Tierney, Trevor T. Robinson, *Common Themes in Multi-block Structured Quad/Hex Mesh Generation*, Procedia Engineering, Volume 124, 2015, Pages 70-82, ISSN 1877-7058, https://doi.org/10.1016/j.proeng.2015.10.123]. The key features of semi-structured quadrilateral meshes are the mesh '***singularities***', where the mesh grid is irregular. These are the nodes where more or less than the regular number (4 in the interior of a quad mesh) of elements meet, **Figure 1.3.3** shows isolated singularities embedded in regular grids of 2D elements. Singularities can also be placed on the boundary of the region, **Figure 1.3.4** and two or more simple

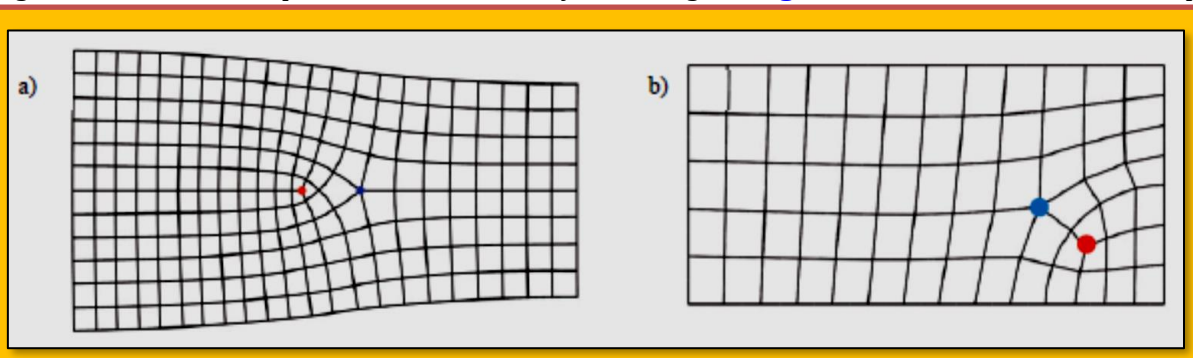


Figure 1.3.6 Controlling mesh density with an extra singularity pair

singularities can be combined into a single degenerate singularity if they are close together, **Figure 1.3.5**. The minimum number of singularities to allow a multi-block mesh of a simply connected polygonal boundary can be found by counting the number and type of the corners in the boundary of the domain, though the placement of these singularities will make a huge difference to mesh quality. If the resulting block topology does not provide adequate control of mesh density then extra control can be provided by adding a pair of negative and positive mesh singularities, **Figure 1.3.6**.

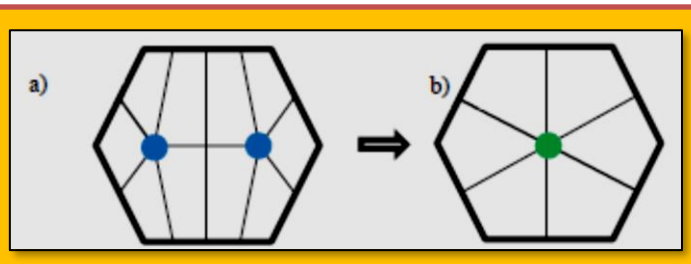


Figure 1.3.5 Two 5-valent singularities (a) combined to give a single 6-valent singularity (b) in 2D

1.3.1.1.1 Placing Mesh Singularities in 2D

The industry standard method of creating multi-block meshes is to manually create a block topology using tools such as *ICEM CFD* and then associate block edges and vertices with the appropriate entities of the real geometry. Whilst this can be effective in creating high quality meshes, and the meshing process can be used to suppress features below the scale of

interest, it is a time-consuming process requiring a great deal of knowledge and expertise from the person creating the meshes. ***The medial axis (MA)*** method was used by Tam and Armstrong [8] to partition the region of interest into small subregions of simple shape, each of which contained at most one singularity. It employed a geometric property of shape called the medial axis, which is a skeleton or dimensionally reduced version of the shape. A characteristic of this approach was that singularities were placed as far as possible from the boundary, and that a good quality mesh was produced since

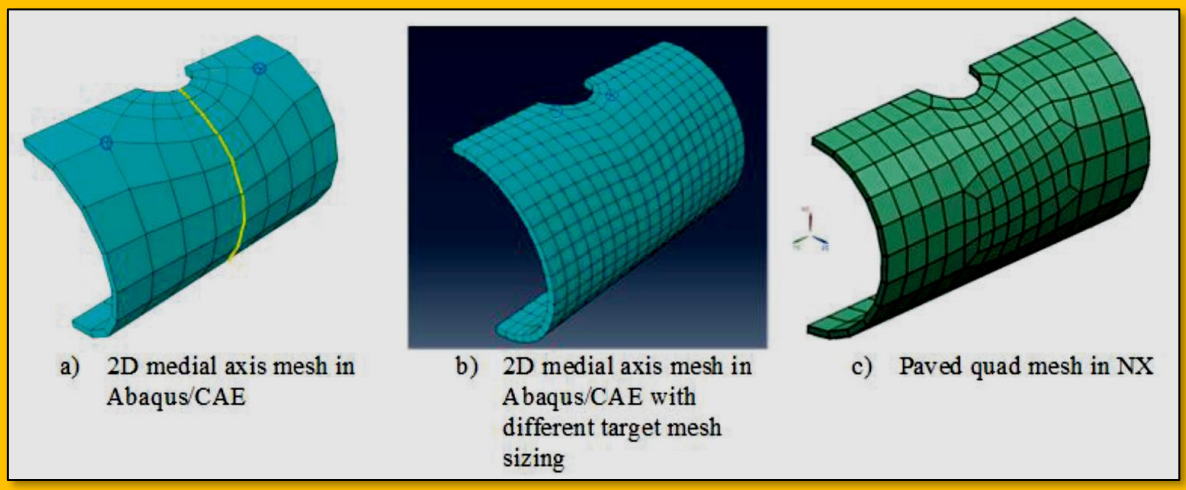


Figure 1.3.7 2D extruded meshes. Note that medial axis meshes a) and b) have only two positive singularities, which are sufficient to mesh the 6-sided extrusion

the partitions subdividing the domain were between parts of the boundary in geometric proximity. An example of the resulting decompositions is shown in **Figure 1.3.7a**, where the partition is shown in yellow. The subregions containing singularities were meshed with a technique called midpoint subdivision [5][6], which divided the region into structured blocks around the singularity: a triangle is decomposed into three blocks, a pentagon into five etc. **Figure 1.3.7a** was decomposed into two 5-sided prisms, each of which has a single positive singularity running through the wall thickness of the model. Using integer programming to control edge division numbers allows the different meshes in **Figure 1.3.7a** and **Figure 1.3.7b** to be generated from the same decomposition. Midpoint subdivision of 2D and 3D primitives has been available in *Abaqus/CAE* for some years.

The medial axis method can be viewed as a less restrictive form of the paving method [11]. In paving, quad elements are added to an advancing front from the boundary. Since paving only has a view of the local mesh geometry and topology, paving algorithms will generate many more mesh singularities than desired, with no control over where they are placed, **Figure 1.3.7c**.

The medial axis is where layers of elements advancing from the boundary collide.

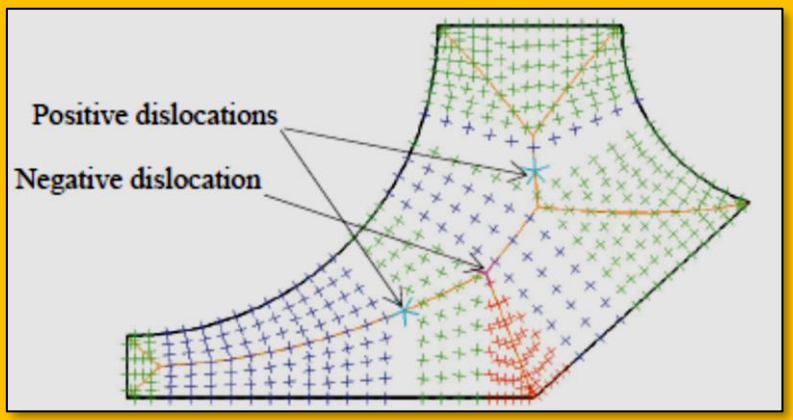


Figure 1.3.8 Cross-field, medial axis and singularity placement based on where MA changes from parallel to mesh flow to diagonally through it

Since, unlike paving, the algorithm doesn't have to worry about the detailed mesh topology where the fronts collide (the MA is just used to identify regions which are opposite or adjacent to each other) the number of mesh singularities can be minimized. Note that the medial axis runs either parallel to the mesh flow or diagonally through it, **Figure 1.3.8**.

1.3.1.1.2 Placing Mesh Singularities in 3D

The conventional approach to multi-blocking hex meshing is to manually define a block topology. Vertices, edges and faces of the block topology are then associated with real geometry. In *ICEM CFD* given element of the block topology can be associated with several geometric faces or edges, effectively accomplishing a virtual topology merging of these faces or edges. Standard block topology configurations like C- or O-type meshes around an interior hole can be used as macro elements in the block definition. An extensive set of block topology recipes is available in various packages, though it appears that these recipes usually have to be hand coded. In 3D, mesh singularities form lines of

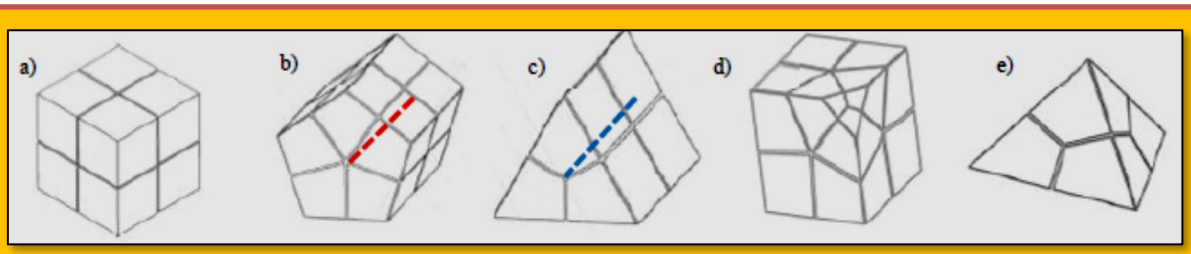


Figure 1.3.9 Hex mesh singularities in 3D: a) Regular mesh; b) +ve singularity; c) -ve singularity; d) 3 +ve and 1 -ve singularities combine; e) 4 -ve singularities combine

element edges where more or less than 4 element edges meet. Excluding degenerate cases, positive and negative mesh line singularities can interact in only a very small number of ways, **Figure 1.3.9**:

- Isolated mesh line singularities can travel through the domain, entering and exiting on the surface of the mesh. 2D extruded or swept meshes are an example of this class, where positive and negative mesh line singularities run along the sweep direction and are shown by the dashed red and blue lines in **Figure 1.3.9 b- c** respectively. The sweep direction in thin sheet regions (where the lateral dimensions are large compared to the thickness) is between the top and bottom surface of the thin sheet. In long slender regions (where the length of the region is large compared to the cross-section dimensions) the sweep direction is in the axial direction of the region. The mesh line singularities follow these sweep directions between source and target faces. In multi-sweep meshing, different combinations of line singularities enter and leave on different faces of the domain
- Isolated singularities can form a continuous loop, such as in a revolved 2D mesh
- Positive and negative singularities can combine. The rightmost two primitives in **Figure 1.3.9** show the simplest possible configurations, though additional degenerate combinations are possible. These were called the 'prime primitives' in Price et al. [19]
    - Three positive and one negative mesh line singularities can combine. This corresponds to the midpoint subdivision algorithm applied to a 'cube-with-a-corner-chipped-off', **Figure 1.3.9d**.
    - Four negative mesh line singularities can combine. This corresponds to the midpoint subdivision algorithm applied to a tetrahedral shape, **Figure 1.3.9e**. Note that this case can actually be further decomposed into two type c) negative singularities which pass each other in the interior of the tet volume, so it could be regarded as a degenerate combination.

1.3.2 Unstructured Meshes

- Triangular (Tetrahedral)
- Quadrilateral (Hexahedral)
- Polygon (Polyhedral)
- Hybrid

Unstructured meshes can accommodate completely arbitrary geometries. However, there are significant penalties to be paid for this flexibility, both in terms of the connectivity data structures and solution algorithms. Grid generators and plotting routines for such meshes are also very complex. (see **Figure 1.3.10**)[3].

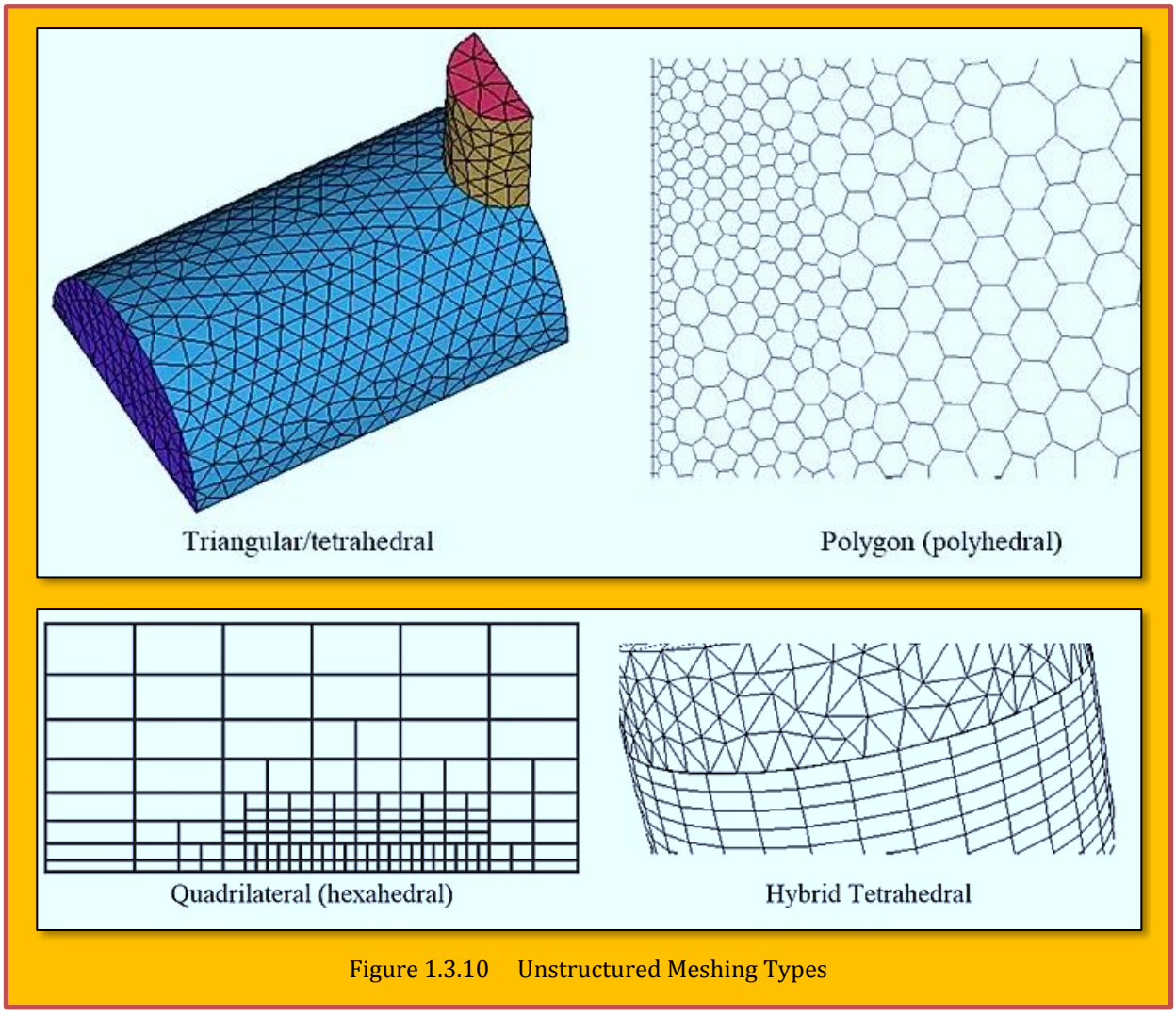


Figure 1.3.10 Unstructured Meshing Types

1.3.3 Tri/Tet vs. Quad/Hex Meshes

- For flow-aligned problems:
  - Quad/Hex meshes can provide higher quality solutions with fewer cells/nodes than a comparable Tri/Tet mesh.
  - Quad/Hex meshes show reduced numerical diffusion when the mesh is aligned with the flow. It does require more effort to generate a quad/hex mesh.
- For complex geometries:

- It would be impractical to generate a structured (flow-aligned) Hex mesh.
- You can save meshing effort by using a Tri/Tet mesh or hybrid mesh which are quicker to generate. Flow, however, is generally not aligned with the mesh.

➢ Hybrid meshes typically combine Tri/Tet elements with other elements in selected regions
- For example, use wedge/prism elements to resolve boundary layers.
- More efficient and accurate than Tri/Tet alone[4].

For a complete discussion, please refer to [Sadrehaghighi][5].

1.4 Classification of Mesh Generation Techniques

As discussed before, the mesh generation techniques can be divided to two major categories of ***structured*** and ***un-structured*** mesh. Strictly speaking, a structured mesh can be recognized by all interior nodes of the mesh having an equal number of adjacent elements. For our purposes, the mesh generated by a structured grid generator is typically all quad or hexahedral. Algorithms employed generally involve complex iterative smoothing techniques that attempt to align elements with boundaries or physical domains. Where non-trivial boundaries are required, ***block structured*** techniques can be employed which allow the user to break the domain up into topological blocks[6]. Structured grid generators are most commonly used within the CFD field, where strict alignment of elements can be required by the analysis code or necessary to capture physical phenomenon[7].
Unstructured mesh generation, on the other hand, relaxes the node valence requirement, allowing any number of elements to meet at a single node. Triangle and Tetrahedral meshes are most commonly thought of when referring to unstructured meshing, although quadrilateral and hexahedral meshes can also be unstructured. While there is certainly some overlap between structured and unstructured mesh generation technologies, the main feature which distinguish the two fields are the unique iterative smoothing algorithms employed by structured grid generation. The semi-complete picture of grid generation algorithm is updated by [*Owens*] and presented here as reference[8] (**Figure 1.4.1**). In general, on the structure side, some mapping techniques such as ***Transfinite Interpolation (TFI),*** or ***Elliptic operator*** are used extensively and proven to be sufficient for majority of applications. On unstructured side, the same could be said about ***Advancing Front*** or ***Delaunay triangulation***. The above table is too broad and extensive for our purpose. Our concentration, as red circles indicating, would be on the:

➢ ***Structured Meshes***
- Complex Variables (Restricted to 2D)
- Algebraic Techniques (TFI)
- PDE Methods (PDE)

➢ ***Unstructured Meshes***
- Delany Triangulation
- Advancing Front
- Octree Method
- Polyhedral Meshes
- Overset Meshes
- Cartesian Meshes

➢ ***Hybrid Meshes***
➢ ***Adaptive Meshes***
- Structured

- Unstructured

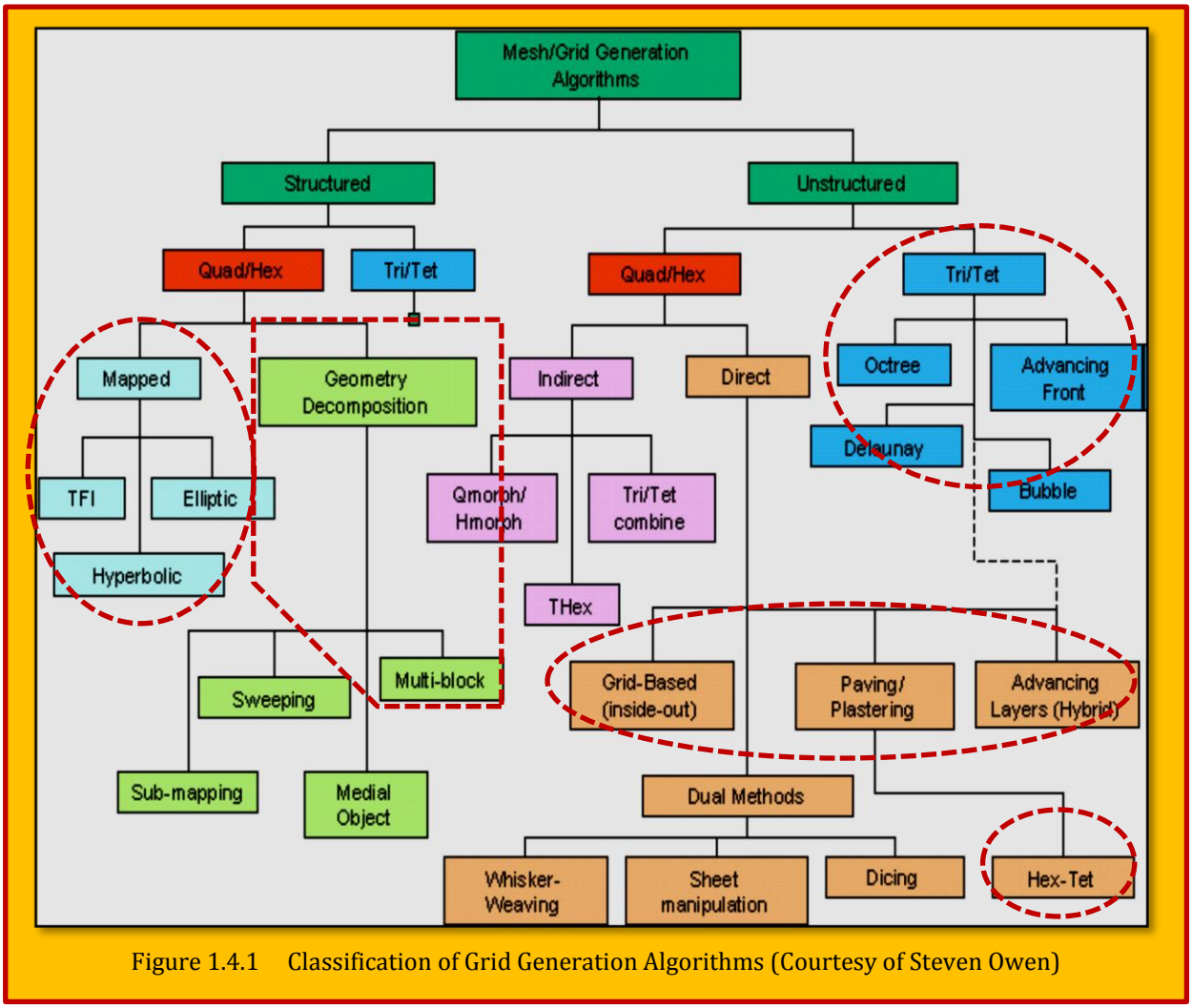


Figure 1.4.1 Classification of Grid Generation Algorithms (Courtesy of Steven Owen)

1.4.1 Field (Domain) Discretization Process (Mesh Generation)

Once a mathematical model is selected, we can start with the major process of a simulation, namely the ***domain discretization*** process. Since the computer recognizes only numbers, we have to translate our geometrical and mathematical models into numbers which of course called ***discretization.*** The first action is to discretize the space, including the geometries and solid bodies present in the flow field or enclosing the flow domain. This set of points, which replaces the continuity of the real space by a finite number of isolated points in space, is called a ***grid*** or a ***mesh***. The process of mesh generation is in general extremely complex and requires dedicated software tools to help in defining grids that follow the solid surfaces (this is called '***body-fitted***' grids, see **Figure 1.3.1** bottom) and have a minimum level of regularity. We wish already here to draw your attention to the fact that, when dealing with complex geometries, the grid generation process can be very delicate and time consuming. Mesh generation is a major step in setting up a CFD analysis, since, as we will see the outcome of a CFD simulation and its accuracy can be extremely dependent on the grid properties and quality. Please notice here that the whole object of the simulation is for the computer to provide the numerical values of all the relevant flow variables, such as velocity, pressure, temperature, etc., at the positions of the mesh points. Hence, this first step of grid generation is

essential and cannot be omitted. Without a grid, there is no possibility to start a CFD simulation. **Figure 1.4.2** shows an example of multi-block ***structured*** mesh for a turbine blade.

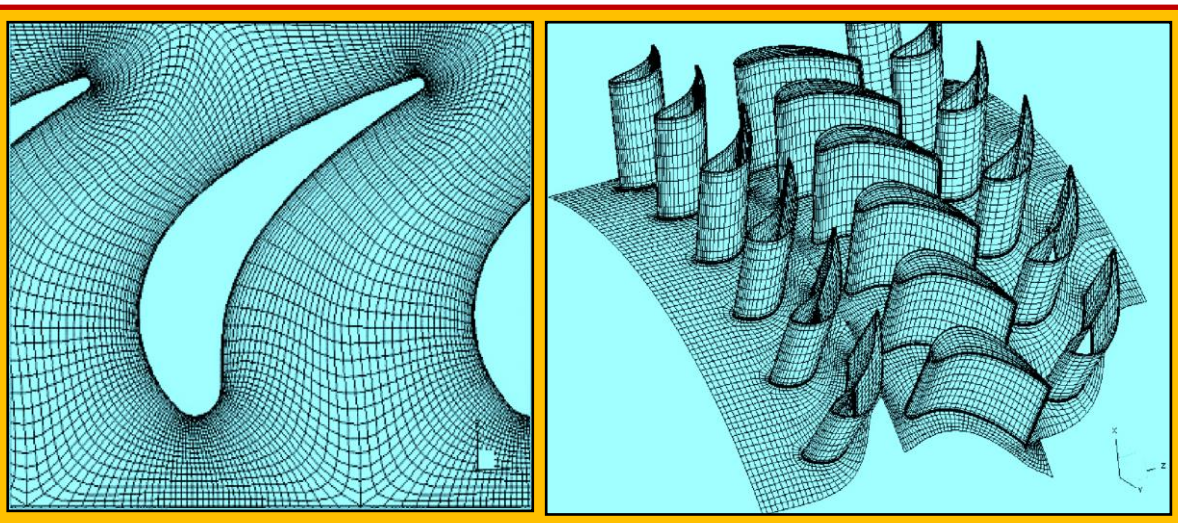


Figure 1.4.2 Examples of Structured Meshes for Turbine Blade

1.5 Mesh Generation - an Art or Science?

Ravindra Krishnamurthy detailed an interesting argument in *GridPro Blog* regarding the subject. He argued that since the birth of the engineering field of CFD, there is a raging debate on the topic, whether grid generation is "***an art***" or "***a science***"? There are strong reasons to support either one or the other, but the debate continues. CFD in general is seen as a science, a tool to mathematically obtain fluid flow information. CFD solvers making use of numerical algorithms and the grid-generators, both math based, at the fundamental level is seen as science. However the arrangement of grid points in the computational domain is seen as an "art", an element greatly influencing the outcome of the CFD simulation.

1.5.1 Why "Art" in the First Place?

Immaterial of the element shapes, whether triangles/tetrahedrons/triangular prisms, quadrilaterals/hexahedrons, or very general polyhedrons, they are glued together in some fashion to form what is called a grid (mesh) that envelops the domain of interest. The envelope forms the foundation upon which the physics solver (typically CFD solver) must operate. ***The general perception in the grid generation part of the story is, the process of generating it is more of "an art" than "a science".*** Is there an inherent rationale behind the arrangement of the basic geometrical elements or it's just a subject matter of visual feel/appeal of the meshing software ? Can a bunch of logical reasons or rational explanations be called a "science"? Is there a need for a debate on the classification of mesh-generation? Why worry on the details of grid generation?, can we not move on to the solver straight away? Why can't we trust the physics solver to generate accurate, reliable CFD numbers for the fluid flow problem at hand and move on?

If we sit back and contemplate, it's easy to see why this debate came into existence in the first place. The simple reason being ***all meshes are not created equal***. And all grids do not give you the same CFD results even for a simplistic geometry like an airfoil. Let alone the type of elements chosen, and the total number of elements, the arrangement and alignment of these elements is significant and can have a strong influence on the final outcome. ***The definition of science tells us, a process can be termed as science if it can be replicated under similar conditions by different individuals who in***

the end should converge exactly to the same results. This basic definition seems to be violated in the field of mesh-generation, and hence has taken the tag of "an art".

1.5.2 The "Science" of Mesh-Generation

Various aspects of grid generation have a deep scientific reasoning behind what we do with it , and why we go about doing it. For instance, there is a reason for the first cell height placement inside the boundary layer, a reason for the growth rate used, a reason for the choice of the far-field distance, a reason for the orthogonality of cells inside a boundary layer, a reason for the choices of grid density, and, of course, a reason to stress high grid quality and so forth. And, moreover, in addition to all of these rational choices, there is the underlying need to accurately represent the boundaries of the simulation region. These choices and associated constraints upon the mesh have their roots in the physics of fluid dynamics, the given regional geometric definitions, and the extent within which the numerical methods do function. For other physics based simulations, there are yet more reasons for additional and/or different constraints upon the generated grid.

This "***science***" based aspects of grid-generation is greatly understood and appreciated in the CFD community. In a way, this has helped to standardize the grid generation practices and has aided in automating parts of the mesh-generation process. Based only on the foundation of scientific rational reasoning, members of the CFD community have established international workshops like the (AIAA) Drag Prediction Workshop (DPW) and the AIAA High Lift Prediction Workshop (*HiLiftPW*) in order to assess the CFD methodology as validated against wind tunnel experiments. A bigger chunk of this process are the flow solver strategies and the act of grid/mesh generation. Due to its importance, the committees running these workshops have come-up with formal guidelines for grid generation that address the classification of fluid flow problems. Imbedded in these classifications from the workshops, there are significant signals of what is good and what is not.

If we were to stand back from the minute details, it is only due to a set of rational and logical reasoning in the grid generation process, we have been able to teach, replicate, and enable flow solvers to generate *meaningful* CFD data as best as they can. When we speak of "*meaningful*", what we are really saying is that we can trust the accuracy of the CFD results so that we can make good decisions from those results. Such decisions rely upon having enough CFD accuracy to answer our key questions with respect to the level of physics being modelled. All of this can make CFD a powerful and reliable design tool.

1.5.3 The "Art" of Mesh-Generation

Having seen some of the scientific aspects of the process, we are brought to the question: What is in mesh-generation which makes one perceive it as more of as an art than a science? ***The answer lies in the way we go about filling the computational domain with the geometric elements***. Though, all we do is populating the fluid region by a grid of an element type or types, it is really how we go about doing it and which of the various options we use. It is this collective whole in the end which brings us to the differences among the grids. Even, well within the standard grid generation framework that was laid out, there are various possibilities and it is up to the CFD engineer to decide which options to pick and use to create the mesh. This aspect can be clearly seen in the grids generated by the participants in CFD workshops like the AIAA DPWs and *HiLiftPW*. Even after adhering to the stringent guidelines prescribed by the committees, the grids, whether structured or unstructured are visibly different, which in turn are reflected in the simulated CFD results. It is for this reason that the organizers would like the participants to use the workshop committee grids, as this will aid in reducing the wide scatter of CFD results due to variation in grids.

What makes grid-generation seem artistic is the fuzzy value or nature of desired grid features, although many, of them, can be stated in a rather analytical way. This includes grid alignment to flows, grid point patterns, good element shapes, gentle transitions between the elements (smoothness), local enrichment for anticipated physics, and relative cell orientations. Although this list is not necessarily complete, it does indicate that various tradeoffs are required and that the

person doing the grid generation must then make a number of choices along the way. Standard grid generation guidelines have laid out stringent rules for some specifics such as grid resolution, growth rate and cell count, but not for the fuzzy items like flow alignment or topology structure which bring in the differences in sometimes less than subtle ways.

1.6 References

[1] Not a general definition. There are many researchers which use the terms (grid/mesh) interchangeably.
[2] Kenneth Wong is *Digital Engineering's* resident blogger and senior editor.
[3] Roy Koomullil, Bharat Soni, Rajkeshar Singh ,"*A comprehensive generalized mesh system for CFD applications*", Mathematics and Computers in Simulation 78 (2008) 605–617.
[4] Manchester CFD team , Web: http://www.manchestercfd.co.uk/
[5] Sadrehaghighi, I. "*Mesh Assessment & Quality Issues*", CFD Open Series, 2020.
[6] Steven J. Owen, "A Survey of Unstructured Mesh Generation Technology", Carnegie Mellon University, PA.
[7] Introduction: An Initial Guide to CFD and to this Volume; page 1, 2007.
[8] Steven Owen: Introduction to unstructured mesh generation, 2005.

2 Topology and Blocking

Mesh generation or ***Domain Discretization,*** has evolved to the point where highly complicated domains can be covered by a variety of mesh types including hexahedral, tetrahedral and overset meshes. It is an important and very tedious aspect of computational geometry and accounts for almost 70% of CFD works. The concept of a mesh as a field or domain discretization of space has been associated with computational methods since the first attempts to obtain numerical solutions of partial differential equations[3]. Establishing a suitable mesh was long considered to be a rather tedious exercise and a minor part of the computational effort involved in solving partial differential equations by either a finite difference or finite element method. But mesh generation has steadily evolved into a discipline in its own right drawing on ideas from other fields, in particular mathematics and computer science, and gradually developing a distinct identity of its own. Two series of international conferences are now devoted entirely to mesh generation and adaptation, and almost all conferences on computational methods have sessions that feature this topic. In addition, it is important to recognize the growing interest of the computer science community in mesh related problems. In addition, it is important to recognize the growing interest of the computer science community in mesh related problems[4]. Not only has this synergy brought new ideas and ways of viewing mesh related questions, it has also opened up whole new areas of application including medical imaging and segmentation, computer graphics and animation, and data interpolation and compression.[5].

2.1 Conformal Mapping of Simply-Connected Region (Sponge Analogy)

It is perhaps not surprising that conformal mapping was among the first and most effective techniques to carry out this task. The best way to the correspondence of a curvilinear grid in physical domain, with logically rectangle grid in computational domain, is through ***sponge analogy***. Consider a rectangular sponge within which an equally spaced Cartesian grid has been drowned. Now wrapped the sponge around a circular cylinder and connect the two end of sponge together. Clearly the original Cartesian grid now becomes a curvilinear grid fitted the cylinder. But the rectangle logical form of grid lattice is still preserved[6]. **Figure 2.1.1** spectacles a simply connected (as oppose to multiple connected) region which obviously results in O-type grid. Since the difference formulae were applied in mapped space it was necessary to transform the partial differential equations to the coordinate system associated with the mapping. Conformal maps lead to a new set of fairly straightforward equations without messy cross-derivative terms. In addition, the orthogonality and smoothness properties of review of conformal mapping

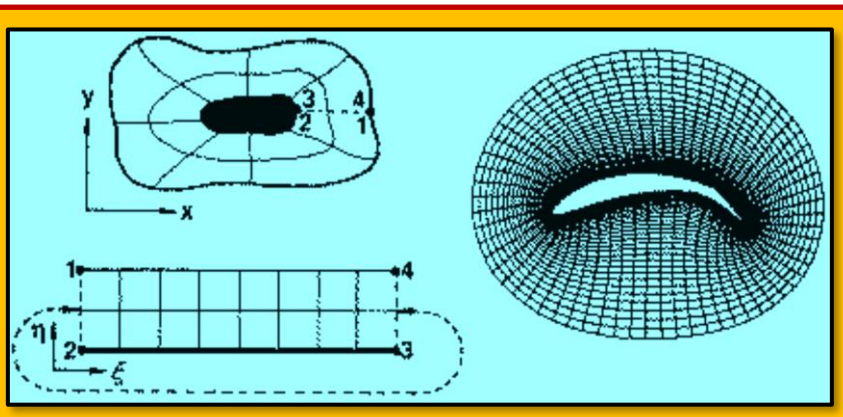


Figure 2.1.1 Sponge Analogy

[3] Richardson LF. Weather prediction by numerical process. Cambridge: Cambridge University Press; 1921.
[4] Edelsbrunner H. "Geometry and topology for mesh generation", Cambridge: Cambridge university, 2001.
[5] Baker, T., "Mesh generation: Art or science?" MAE Department, Princeton University, Princeton, NJ.
[6] Baker, T.,J., "*Mesh generation: Art or science?",* MAE Department, Princeton University, Princeton, NJ.

meshes obtained in this manner produce a high quality mesh in physical space. Perhaps the first published application of conformal mapping to (CFD) is circle plane mapping that transforms the space exterior to an airfoil onto the interior of the unit circle. This particular conformal mapping technique extends back a long way but its use for creating suitable meshes was a novel application. The same mapping was later used by [Bauer et al.][7] when they developed the first transonic flow code for solving the full potential equation. Other conformal mappings were developed to handle combinations. A comprehensive techniques for mesh generation has been given by Moretti[8]. Another useful reference is the paper by[9]. Interchanging between different grid types are easy and sometimes required. For example, *Pointwise®* provides a script capable of changing the topology of a structured block from an H topology to an O-H topology. This script can help improve grid quality when working with cylindrical configurations, as shown in **Figure 2.1.2**. https://ptwi.se/2VakhGv.

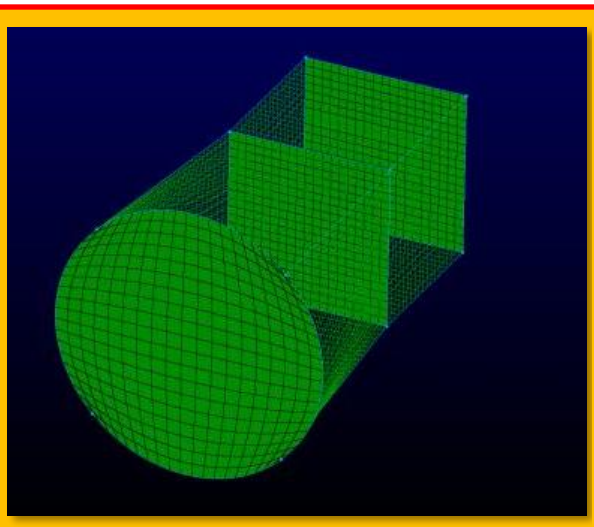


Figure 2.1.2 Converting a H Type to O-H Type Topology (Courtesy of Pointwise ®)

2.1.1 Case Study - Non-Linear Conformal Mapping Utilizing Solid Mechanics

Authors : *Michael Turner, Joaquim Peir´o, David Moxey*
Title : *A variational framework for high-order mesh generation*
Affiliation : *Department of Aeronautics, South Kensington Campus, Imperial College London, London SW7 2AZ, U.K.*
Citation: *Turner, M., Peiró, J., & Moxey, D. (2016). A variational framework for high-order mesh generation. Procedia Engineering, 163, 340-352.*

The premise of this work by [Turner, M., Peiró, J., & Moxey, D. (2016)] is to examine the generation of a boundary-conforming high-order mesh that is obtained through a deformation. The starting point is a straight-sided high-order mesh $\Omega = U^{N}_{ele=1}\ \Omega^{e}$ composed of N_{el} conformal elements, which is equipped with geometry-conforming displacements at the boundary of the domain. We view Ω as a solid body, so that the displacements at the boundary induce a deformation in the interior of the domain. The end result should be a valid high-order mesh Ω^{*}, where the interior elements have been deformed to accommodate the displacement at the boundary and improve the quality of the resulting elements. From a solid mechanics perspective, we may construct a model that, for example, represents linear or nonlinear elasticity. This often appears in the form of an elliptic partial differential equation which may be solved to calculate the displacement *u* between the original and

[7] Bauer F, Garabedian P, Korn D. Supercritical wing sections I, Lecture Notes in Economics and Mathematical Systems, vol. 66. Berlin: Springer; 1972.
[8] Moretti G."*Grid generation using classical techniques*". Proceedings of the NASA Langley workshop on numerical grid generation techniques, Langley, VA, October, 1980.
[9] CaugheyDA, "*A systematic procedure for generating useful conformal mappings*", Int J Num Meth Eng. 1978.

deformed states. As mentioned in the introduction, these approaches have both previously been examined in the context of high-order mesh generation. However the purpose of this work is to instead consider a ***variational*** approach, in which we aim to find a displacement *u* that minimizes an energy functional

$$\varepsilon(\mathbf{u}) = \int_{\Omega} W(\mathbf{u}, D\mathbf{u}) d\mathbf{X}$$

Eq. 2.1.1

where $u = x - X$ denotes the displacement of a point $X \in \Omega$ from its initial straight-sided configuration to the deformed position $x \in \Omega*$, and Du represents its first derivatives. The problem is closed by setting a boundary condition of zero displacement at the boundary once the deformation has been imposed.

In this setting, the problem is recast into one of selecting an appropriate (and generally non-linear) energy function W and a numerical method for the arising nonlinear optimization problem. In this section, we first describe the solid mechanics preliminaries that underpin this formulation and present a number of choices for W which encompass previous methods for high-order mesh generation.

To introduce the solid mechanics preliminaries that are used to define the variational problem, we follow a similar notation to that of reference [1]. The deformation of an initial, or undeformed, configuration of a solid, denoted by Ω, with boundary $\partial\Omega$ into a new, or deformed, configuration, $\Omega*$, with boundary $\partial\Omega*$ is described mathematically by a mapping $\chi : \Omega \rightarrow \Omega*$. Coordinates $\mathbf{X}$ in the initial configuration move into a point of coordinates x in the deformed configuration under the action of a mapping $\boldsymbol{\chi} : \Omega \rightarrow \Omega*$ so that $\mathbf{x} = \boldsymbol{\chi}(\mathbf{X})$. The notation used to describe the mapping representing the deformation is depicted in **Figure 2.1.3**. The deformation gradient tensor is defined as

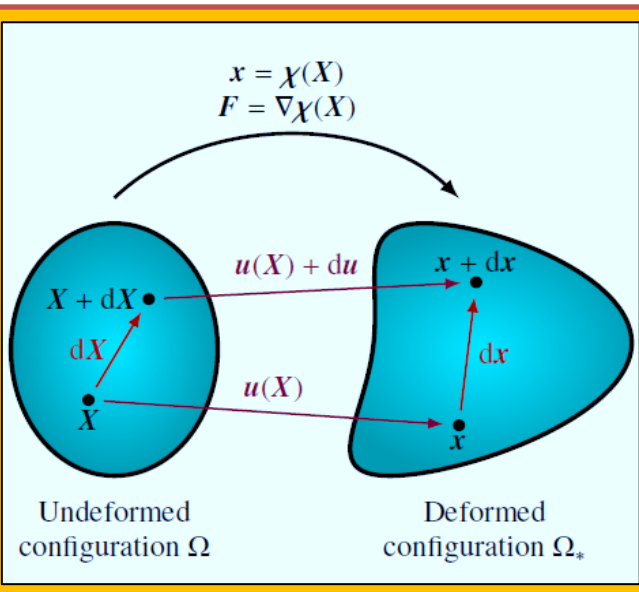


Figure 2.1.3 Notation used for the deformation mapping $\chi : \Omega \rightarrow \Omega*$

$$\mathbf{F} = \frac{\partial \boldsymbol{\chi}}{\partial \mathbf{X}} = \nabla \boldsymbol{\chi}$$

Eq. 2.1.2

which relates elemental vectors and differentials in the configurations according to $d\mathbf{x} = \mathbf{F}d\mathbf{X}$. The determinant $\mathbf{J} = \det \mathbf{F}$, is referred to as the Jacobian of the mapping, and represents volumetric deformations so that $dv = \mathbf{J}dV$, where dv and dV denote elemental volumes in the deformed and undeformed configurations respectively. General measures of deformation are represented by the strain tensors. Let us consider two elemental vectors, $d\mathbf{X}_1$ and $d\mathbf{X}_2$, in the undeformed configuration that deform into the elemental vectors dx_1 and dx_2, respectively. The right Cauchy-Green tensor, $\mathbf{C}$, characterizes their scalar product $dx_1 \cdot d\mathbf{x}_2 = d\mathbf{X}_1 \cdot C\, d\mathbf{X}_2$, which can be written in terms of the deformation gradient as $\mathbf{C} = \mathbf{F}^T\mathbf{F}$.

2.1.1.1 References

[1] J. Bonet, R. Wood., Nonlinear continuum mechanics for finite element analysis, Cambridge University Press, 1997.

2.2 Single Block Mesh Topology

Before we pay attention to the individual cell topology, we consider domain topology which are compared for the 2D case, namely H, C, and O topologies. (See **Figure 2.2.1**). Meshes with H-H and C-H topology were constructed for 3D comparison; however due to the incompatibility of the C-H structure on a sharp wing tip or trailing edge with the current solver, no C-H studies are included. Most of the studies were under lifting inviscid flow conditions. Multiple studies were conducted under turbulent conditions but only one is included. Overall, when it comes to topology, the H mesh scores first place followed by the C mesh and the O mesh comes last. For an interesting discussion in regard to mesh topology, readers are encourage to consult the work by [Marcon et al.][10].

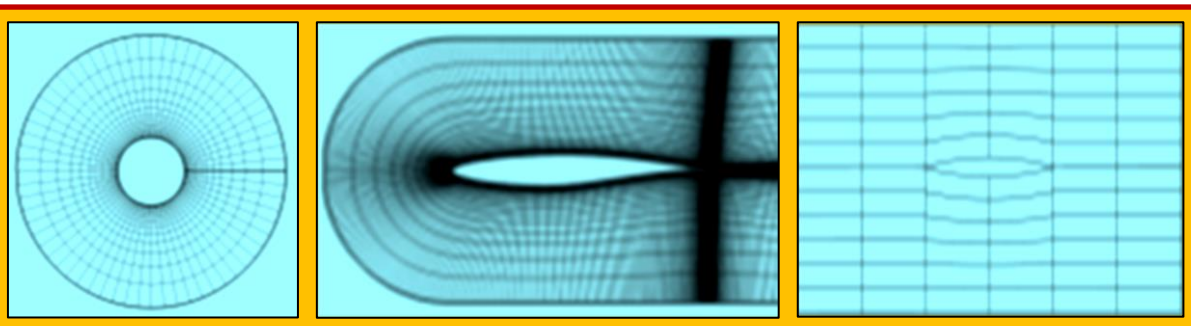


Figure 2.2.1 Domain Topology (O-Type, C-Type, and H-Type; from left to right)

2.2.1 The H Type

Figure 2.2.2 shows an H-mesh[11]. This approach is acceptable for blade profiles with sharp leading and trailing edges. With blunt blades, they end up creating singularity points. Singularities capture curvature regions inaccurately, implant high aspect ratio cells in cross-flow direction and in viscous grids, they induce transverse propagation of boundary layer. In a CFD simulation, this will result in thickening of the boundary layer which in turn triggers excessive thickening and separation of the boundary layer downstream [Krishnamurthy].

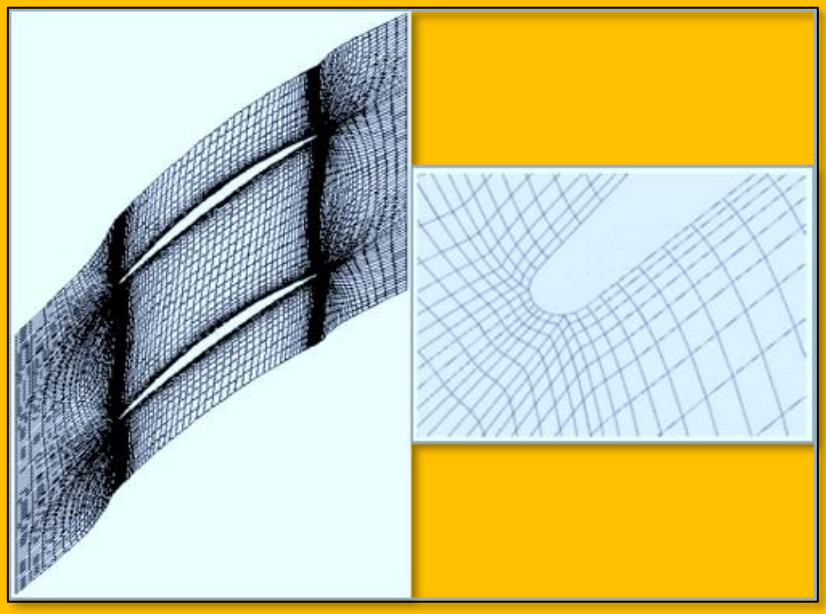


Figure 2.2.2 H-Type Topology. Image Source Reference [22]

2.2.2 The C-O Type

[10] Julian Marcon, Michael Turner, and Joaquim Peiro, "*High-order curvilinear hybrid mesh generation for CFD simulations*", Imperial College London, South Kensington Campus, London SW7 2AZ, United Kingdom.

[11] "*PADRAM*: Parametric design and rapid meshing system for turbomachinery optimization", Shahrokh Shahpar et al, Proceedings of ASME Turbo Expo June 2003, Georgia, USA.

The limitations of the H-type are overcome by C-O- type meshes. C-O–grid has good resolution at the leading and trailing edges, captures the curvature regions effectively and also provide good resolution of the wake. **Figure 2.2.3** shows an example of a C-type grid[12]. Using this topology, one can expect good quality grid for low cambers, but for blades with large camber or higher pitch, C-O-pattern end up generating skewed grids in the mid-passage region. Also, they harbor mesh singularities at the junction of the upstream/downstream boundaries and periodic surfaces. [Krishnamurthy].

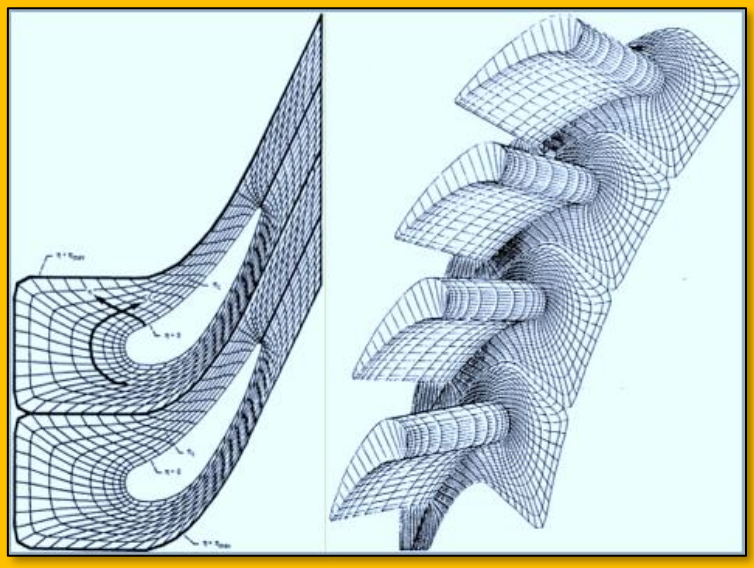


Figure 2.2.3 C-Type Topology. Image Source Reference [23]

Another example of C-O type mesh using a wing is illustrated in **Figure 2.2.5**[13]. **Figure 2.2.4** displays a C-H topology for a wing section, where red represents the C-type and blue the H-type[14].

2.2.3 The H-O-H Type

A marriage of the above two approaches results in what is known as the H-O-H approach. This brings in the positives of both the H-type and the O-type and generates an effective mesh for a large class of blade profiles. O-pattern around the blade provide orthogonal cells in the viscous regions of the blade, while the H-pattern induce orthogonality at the periodic surfaces. It does contain singularities where the H-blocks meets the O-blocks. But since they are physically placed away from the high-gradient regions of leading and trailing edges, they are benign and do not inflict any adverse reaction while running CFD codes. (see **Figure 2.3.1**).Error! Reference source not found.

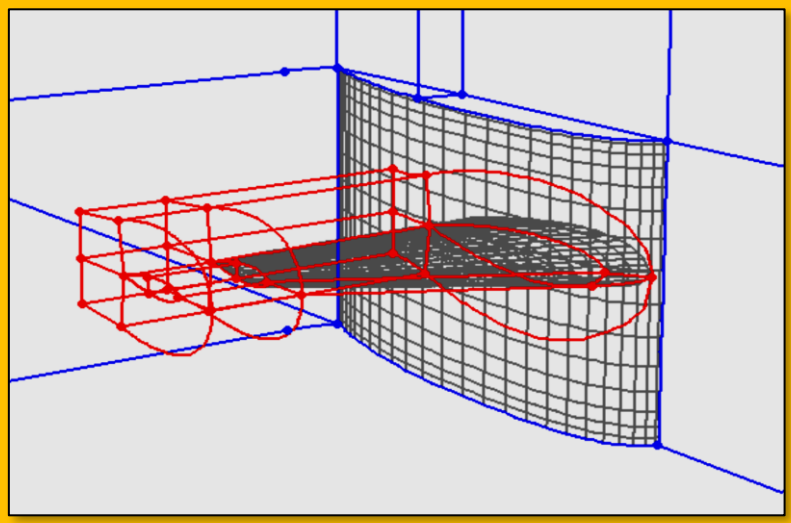


Figure 2.2.4 C-H Type Topology for a Wing Section

[12] "Generation of a composite grid for turbine flows and consideration of a numerical scheme", Y. Choo et al, NASA Technical Memorandum, Technical Report 86-C-38, November 1986.

[13] Ney R. Secco, Gaetan K. W. Kenway, Ping He, Charles A. Mader, and Joaquim R.R.A. Martins, *"Efficient Mesh Generation and Deformation for Aerodynamic Shape Optimization"*, *AIAA Journal*, 2020.

[14] D. Feszty, T. Jakubík, *" Grid Generation"*, Széchenyi University.

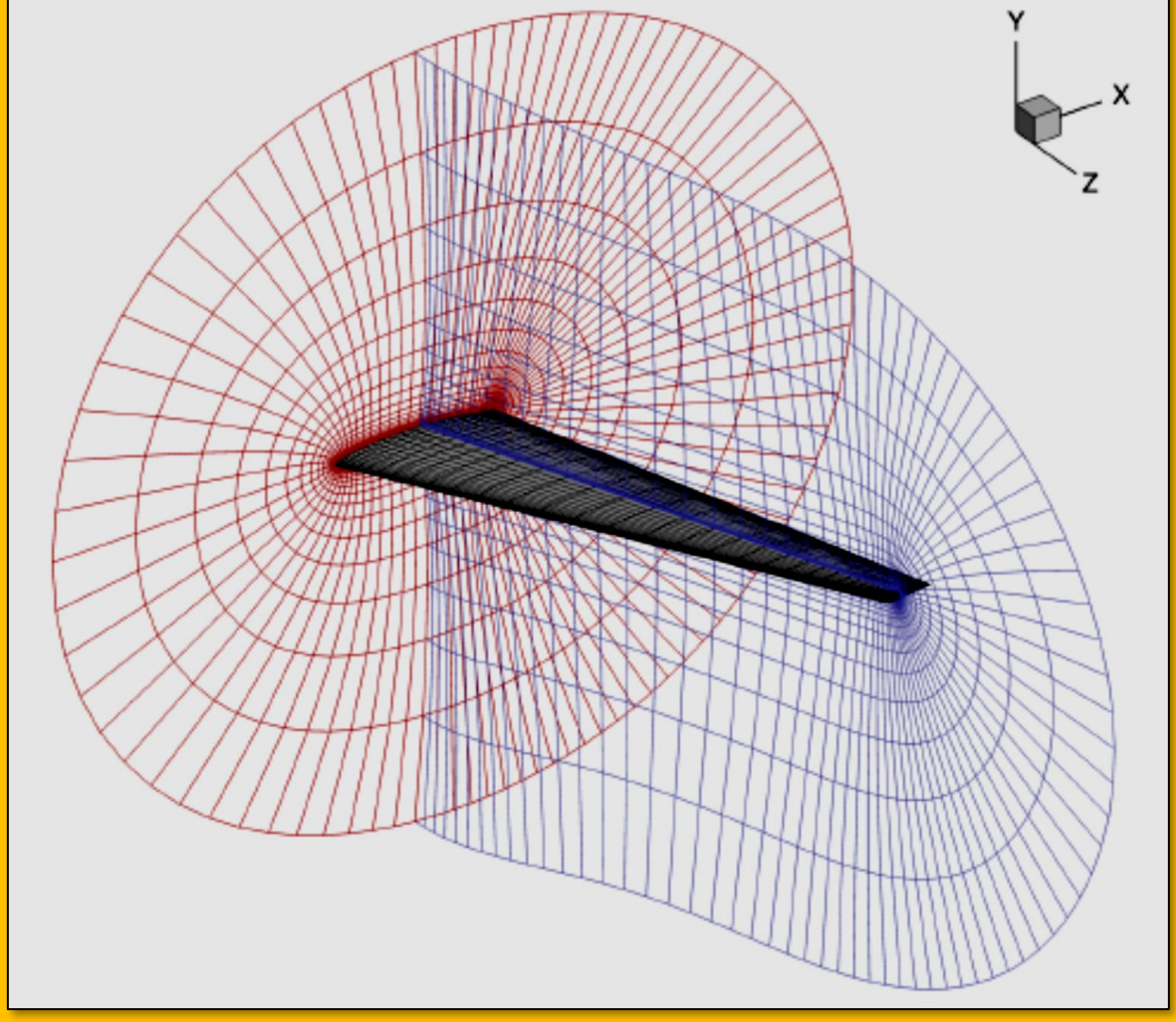


Figure 2.2.5 Example of a C-O Type Mesh (adapted from [32])

When it comes to mesh parameters, the studies show that with carefully chosen mesh spacing around the leading edge, good orthogonality and skewness factors, smooth spacing variation, and a reasonable number of nodes, excellent CFD results can be obtained from the mesh in terms of accuracy of computed functional, determined convergence order and adjoin error estimation. Now with regard to Topology of individual cells where three types are considered; Hexahedral, Tetrahedral, and Polyhedral. In essence, ***topology is a structure of blocks that acts as a framework for placing mesh elements***.

2.3 Multi-Blocking Mesh Topology

Blocks are laid out without gaps with shared edges and corners and contain same number of elements along each side. Number of blocks will dictate the skewness of the grid elements. As an example, **Figure 2.3.1** illustrates topology of a 2D representation of the multi-block mesh performed by [Alavi Moghadam et al.][15]. The O-type mesh encompassing the blade is clearly visible on the middle, while

[15] S. M. Alavi Moghadam, M. Meinke, and W. Schr¨oder, "*Analysis of Tip-Gap Size on Tip-Leakage Flow in an axial fan at design and off-design operating conditions*", 13th European conference on turbomachinery fluid dynamics & thermodynamics etc13, April 8-12, 2019; Lausanne, Switzerland.

H-types cover the extremities of the block. The mesh consists of 829 x 521 x 321 cells around the blade in the stream-wise, radial, and azimuthal direction.

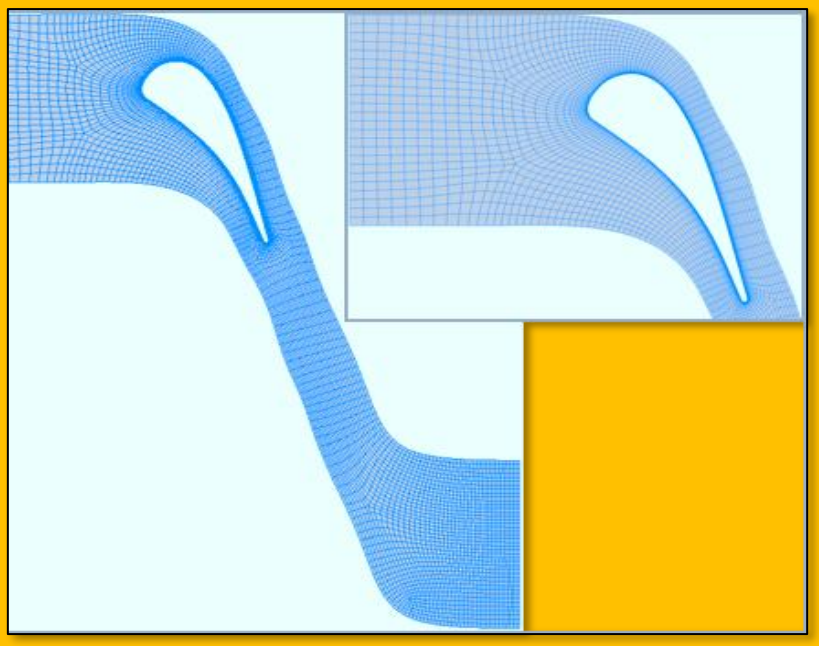


Figure 2.3.1 H-O-H Topology for a Fan Blade

2.3.1 Domain Decomposition with Multi-Blocking

This problem was largely solved by the second significant development, ***the multi block strategy***, or ***Domain Decomposition***. The basic idea, first formulated is to break up the domain into several smaller blocks (essentially an ultra-coarse mesh) and then generate separate meshes in each individual block. **Figure 2.3.2** illustrates this idea by showing a schematic of a three block decomposition for the region around a wing. In this example, one would use an H–H-mesh combination in blocks 1 and 3, and a C–H-mesh combination in block 2. A block corresponds to a sub domain that is geometrically much simpler than the full configuration and which can therefore be easily meshed either by solving a partial differential equation or, alternatively, by an algebraic method[16]. It is, in fact, common practice nowadays to create the mesh in any particular block by an algebraic method such as transfinite interpolation and then smooth the mesh by some iterations of an elliptic solver. A slightly more complicated topology of a dual Block for generic airplane configuration shown in **Figure 2.3.3**.

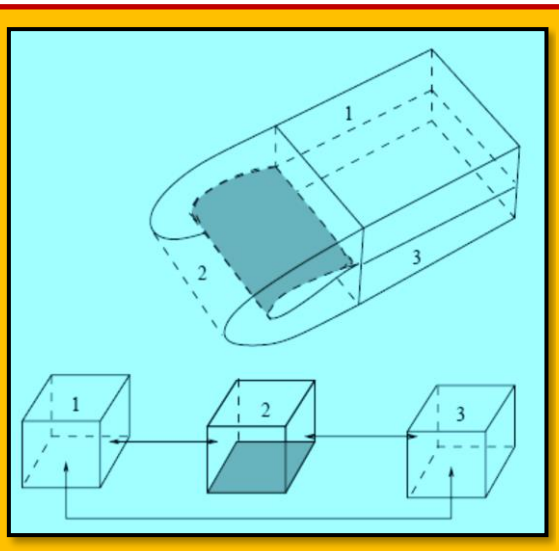


Figure 2.3.2 Multi Block Representation for C-H Mesh Around a Wing Section

An example showing a multi block conformal mapping for a M6 and Reference-H wings is illustrated in **Figure 2.3.5 (a)** and **Figure 2.3.5 (b).** Another example of multi-block structure gridding for a Turbine Blade is giving by **Figure 2.3.6.** ***GridPro©*** has developed a ***Topology Input Language (TIL)*** which can be used for similar geometries with minimal effort[17]. The domain decomposition is in essence a "***divide and conquer***" technique for arriving at the solution of problem defined over a domain from the solution of related problems posed on subdomains. The main reason is that the solution of the subdomain is qualitatively or quantitatively "easier" than the

[16] Eriksson LE,"*Generation of boundary-conforming grids around wing-body configurations using transfinite interpolation*", AIAA J 1982; 20:1313–20.

[17] *An overview of Grid Pro/az3000 for automated grid generation*.

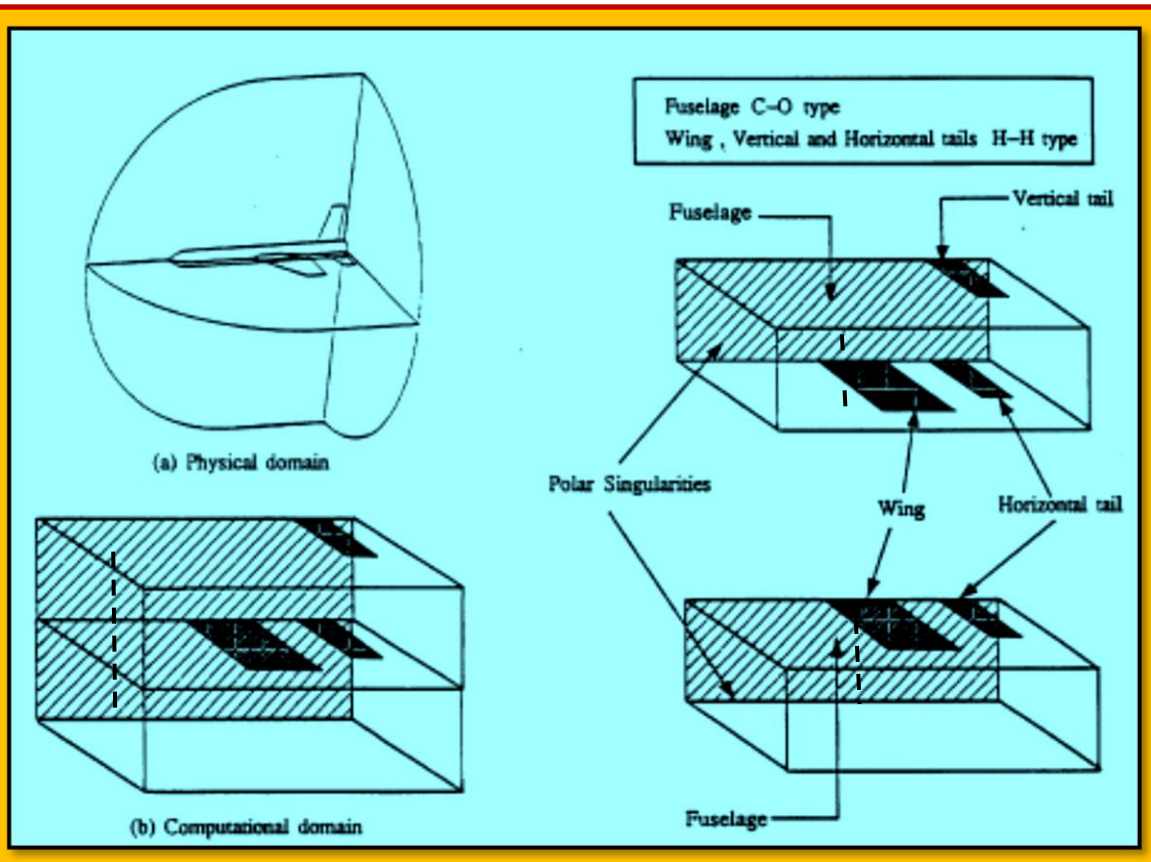


Figure 2.3.3 Dual Block Grid Topology for a Generic Wing-Fuselage Configuration

original one. Other factors are memory concern as well as that the subdomain can be solved with the aid of parallel programing. The issue of domain decomposition is vast and it involves a lot of math such as ***Schwarz concept***[18]. He purposed that simply:

- Solve the PDE in the circle with boundaries taken from interior of square.
- Solve the PDE in the square with Boundaries taken from interior of circle.

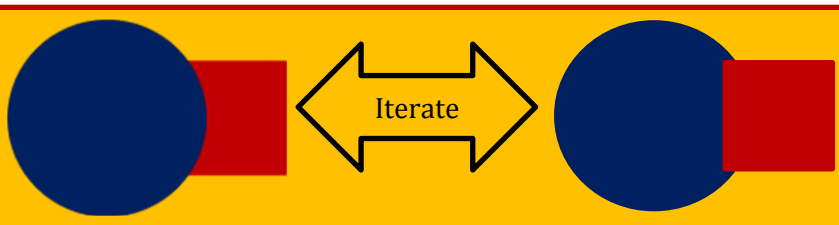


Figure 2.3.4 Schwarz Concept of Iterating Between Domains

And then iterate as depicted in **Figure 2.3.4**. These days, with aid of strong work stations with visual aids, this is running on the background. The user does not know, or cared, what algorithm in running. Some of the vendors are opted for automatic DD schemes, or at least to begin with. User has options to change the topology later. But there is no free launch! There is usually a script which should be run prior to DD. An example would be ***GridPro®*** which is runes a ***TIL (Topology Input Language)*** script, written in C. The DD obtained using a *TIL* for an M6 wing is shown in **Figure 2.3.7.** Other venders have their own scripts or input data depending. Another example is ***Pointwise®*** which uses ***Glyph*** or newer

[18] David E. Keyes, "*Domain Decomposition Methods for Partial Differential Equations*", Department of Applied Physics & Applied Mathematics Columbia University.

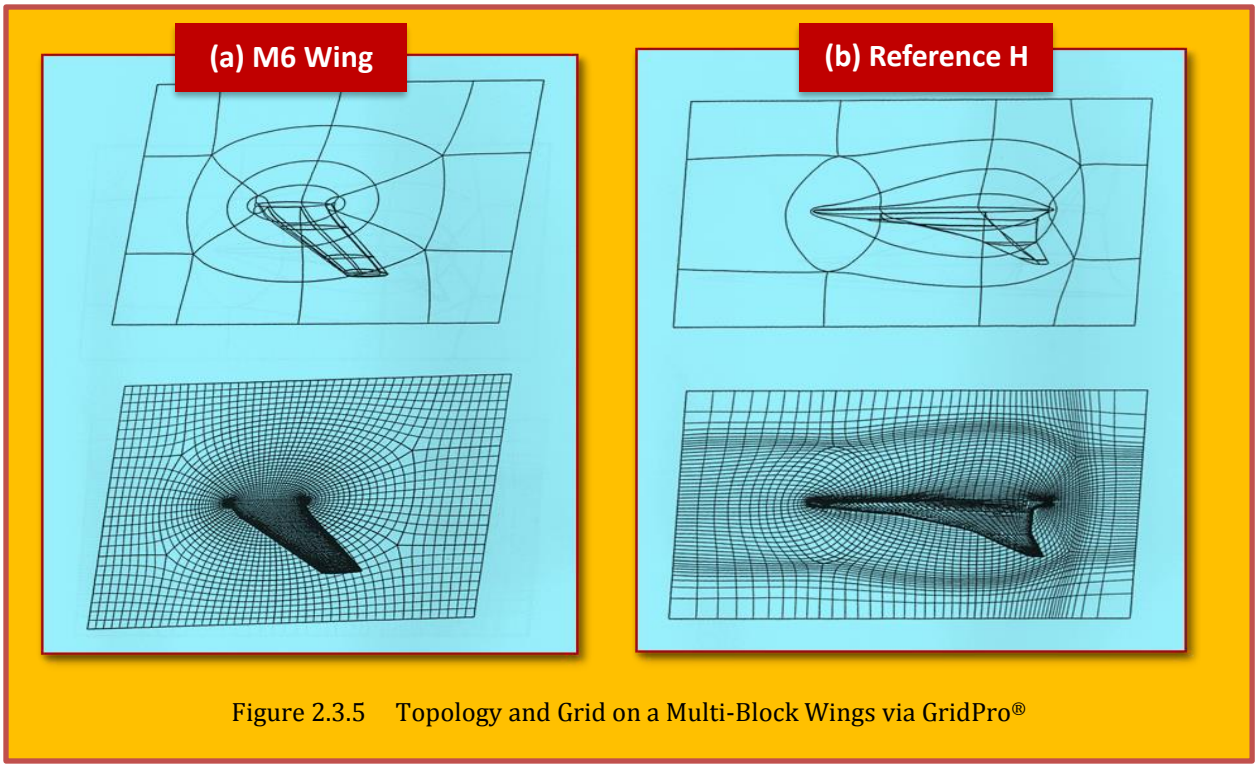


Figure 2.3.5 Topology and Grid on a Multi-Block Wings via GridPro®

Glyph2 as a scripting for the geometry. There are generally two methods for generating the grid; ***Top to Bottom (TTB)*** and inversely ***Bottom to Top (BTT).*** A new arrivals from the same vendor is ***ANSYS 19.2*** which is Mosaic technology automatically combines a variety of boundary layer meshes using high-quality polyhedral meshes. **Figure 2.3.8** displays a multi-blocking of wing fuselage geometry using *Ansys 2019-R2*. While most of unstructured mesh engines use the TTB approaches, majority of structured ones are adapted to BTT. Some might think that multi-blocking approach is too tedious which of course is true. But the reward is in complete control of grid and its quality, something which is usually hard to come by in automated unstructured grid generates.

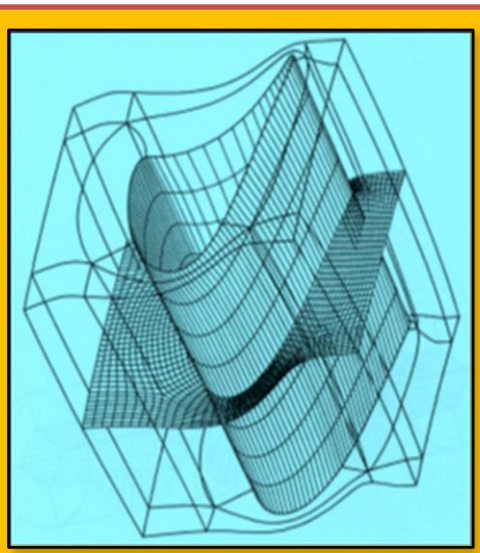


Figure 2.3.6 Multi-Block Gridding of Turbine Blade - (Courtesy of GridPro©)

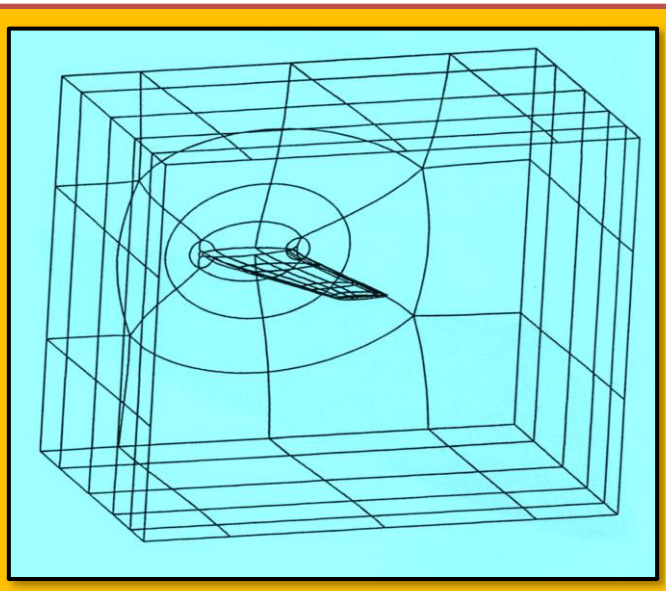


Figure 2.3.7 Domain Decomposition for M6 Wing using TIL Scripts (Courtesy of GridPro)

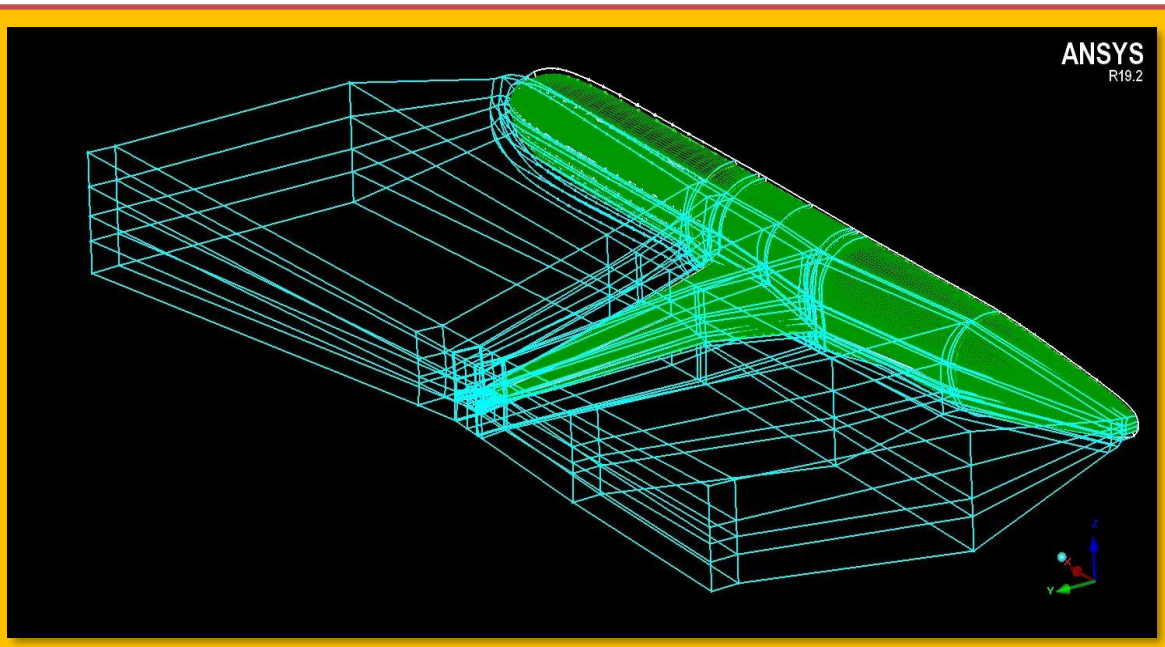


Figure 2.3.8 Multi-Blocking of a Wing-Fuselage Geometry by *Ansys 2019-R2*

2.3.2 Further Remarks on Block Topology for Multi-Block Meshing

Multi-block structured mesh generation is among the most widely used meshing techniques in flow simulations. There is essentially a fly by process and there in no specific rule, just only the users ingenuity. But to be consistent, we include the development by [Ali et al.][19] which offers some indicators in that topic. This is basically a two-stage process. In the first stage, a suitable blocking topology is generated which divides the complex domain into simple sub-domains. The resulting blocks are subsequently meshed. This structured blocking offers an efficient meshing strategy for topologically simple configurations and standard templates exist for partitioning of such domains. For example, the H-O-H type blocking is commonly used to mesh the turbine blade passage. However, the modern day design challenges demand the computational analysis of more realistic geometries. Aero-engine domains, for example, involving the fan, outlet guide vanes, gearbox shaft and nozzle coupled with wing, flap and pylon are now being used for flow simulations. Meshing such multiply linked and more diverse geometries requires significant user intervention, or writing of templates as part of a library. Thus, an automatic or semiautomatic blocking strategy can be beneficial to reduce the CFD design cycle time and could be a better alternative to the unstructured or hybrid meshing methods.

Fully automatic 3D block topology generation is a complex problem and currently ***there is no ideal block topology algorithm*** with all the desired features for structured mesh generation. However various automatic blocking approaches have been proposed with varying levels of automation and geometric complexity handling. This include approaches based on medial axis (Tam & Armstrong)[20] and (Blacker & Myers)[20], paving/plastering[21-22] and more recently methods based on cross/frame

---

[19] Zaib Ali, James Tyacke, Paul G. Tucker, Shahrokh Shahparb, "*Block topology generation for structured multi-block meshing with hierarchical geometry handling*", 25th International Meshing Roundtable (IMR25), 2016.

[20] T. Tam, C. Armstrong, *2D finite element mesh generation by medial axis subdivision*, Adv. Eng. Software (1991).

[21] T. D. Blacker, R. J. Myers, *Seams and wedges in Plastering: A 3D hexahedral mesh generation algorithm*, Engineering with Computers 2 (1993).

[22] M. L. Staten, S. J. Owen, T. D. Blacker, *Unconstrained paving and plastering: A new idea for all hexahedral mesh generation*, Proceedings of 14th International Meshing Roundtable, Sandia National Lab, 2005.

field. The ***medial axis transform (MAT)*** based algorithms for the domain decomposition have been presented. Here the medial axis is generated using the *Voronoi* based method. A subdivision is created resulting in one block for each medial vertex, medial edge and medial face. A midpoint subdivision is then used for meshing the blocks.

2.3.2.1 Medial Axis Definition

For a 2D surface, the ***medial axis*** is defined as the set of points where each point, $\hat{p}$, has a centered inscribed circle, U, that touches the boundary entities at more than a single point (Fogg et al.)[23]. Distinct parts of the medial axis are characterized by the number of touching points of $U(\hat{p})$. Medial

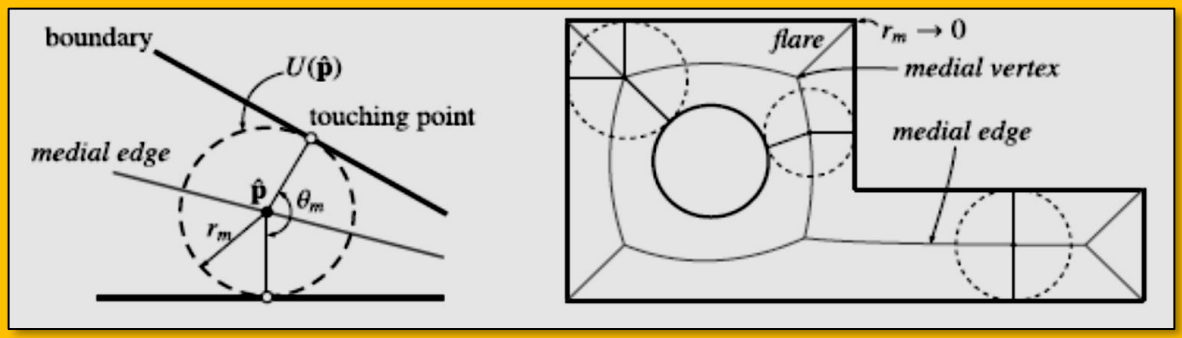


Figure 2.3.10 Medial edge features and terminology (left) and medial axis for simple surface (right)

edges connect subsets of the points with two touching points and they usually make up the majority of the medial axis. The radius of $U(\hat{p})$ is called the medial radius, r_m, and the subtended angle between the radii of $U(\hat{p})$ to the touching points is called the medial angle, θ_m. A special type of medial edge feature that frequently occurs is termination at a convex corner where r_m shrinks to zero. The corresponding medial edge is called a *flare* or *flange*. These are all illustrated in **Figure 2.3.10**. ***Medial vertices*** represent the meeting points of medial edges as $U(\hat{p})$ transitions from one boundary pair to another. In standard cases three touching points are involved. In special cases a medial vertex is equidistant to more than three touching points. There are two other special cases, the first being a medial vertex at the center of a circular arc with $U(\hat{p})$ in finite contact. The other is a curvature contact scenario, such as near the end of the major principal axis of an ellipse. There, the curvature of $U(\hat{p})$

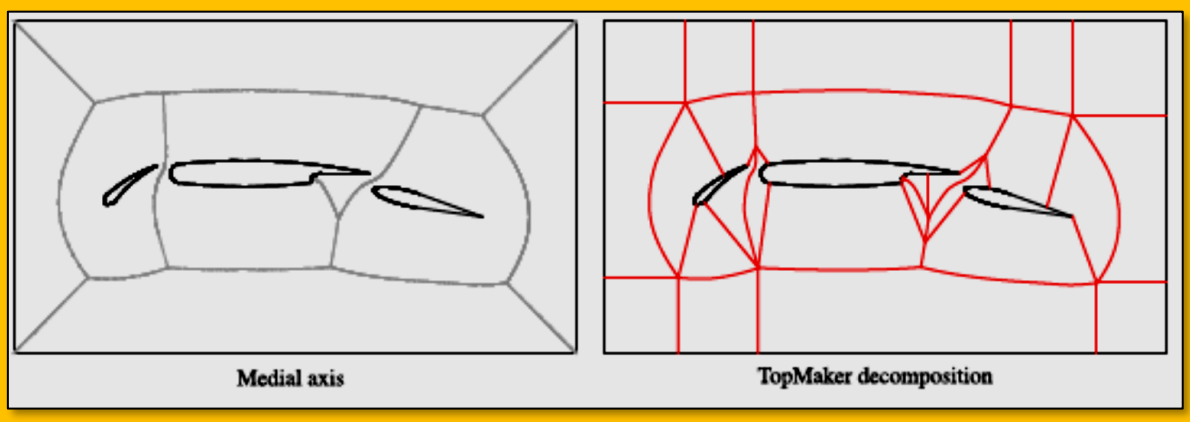


Figure 2.3.9 Multi-element Airfoil surface decomposed by Medial Axis & TopMaker [24]

[23] Fogg, H. J., Armstrong, C. G., & Robinson, T. T. (2016). Enhanced medial-axis-based block-structured meshing in 2-D. Computer-Aided Design, 72, 87-101. https://doi.org/10.1016/j.cad.2015.07.001

equals that of the boundary at the touching point, which results in a single touching point belonging to a medial vertex with θ_m equal to zero.

An alternative has been presented by [Rigby][24] called the '***TopMaker***' approach, which makes use of medial vertices and parts of medial axis to block the domain (**Figure 2.3.9**). Medial vertices are defined as the points which are equidistant from three locations form the domain boundary. Consequently, six types of medial edges and appropriate rules are defined for creating the blocks. Further enhancements have been included to produce a good quality mesh however this technique has yet to be extended for 3D.

Distance field based approaches are also widely used for the medial axis approximation and domain decomposition[25-26]. A hybrid approach called ***differential MAT*** or ***d-MAT*** approach is presented in [Xia and Tucker][27-28]. Here, the hyperbolic-natured ***eikonal***, level set equation is used to calculate the distance field[29]. Medial axis point clouds are then extracted from the Laplacian or Hessian determinant of the distance field. A thinning algorithm is then used for thinning the point clouds into curves and surfaces. For illustration of the model, please refer to [Ali et al.][30].

Recent advancements in mesh generation are the methods based on the cross-fields (frame fields in 3D). A cross field is defined by assigning a set of four unit vectors to points at the discrete locations. These unit vectors form a regular cross on the tangent plane. Thus the size and the orientation of the quadrilateral cells can be specified by the cross field. A number of approaches have been put forward for 2D and 3D cross field based domain decomposition and mesh generation. To generate the block topology, the partitioning created by connecting the cross-field streamlines to the singularities can be used. The resulting blocks of the cross field can then be mapped to a grid. The cross field approach towards domain decomposition and mesh generation is novel and efficient but quite complex and expensive. [LayTracks3D][31], is a hybrid hexahedral meshing method combining medial axis based decomposition and the advancing front method. This technique produces good quality hexahedral meshes but degenerate cells can be formed around the sharp concave features.

[Malcevic][32] presents another automated blocking strategy based on a Cartesian fitting method. While preserving the topology definition, a forward geometry simplification is performed followed by fitting the model into a Cartesian framework. The next step is blocking the domain after which the blocked model is mapped back on to the original geometry. Further operations such as removing singularities by J-grid wrapping are performed to enhance the mesh quality. This technique has been applied for meshing the end-wall cavities found in turbomachinery. This technique is very simple and but has only been demonstrated for 2D cases so far. The method sometimes produces some unnecessary mesh clustering across the block interfaces. An assessment of various automatic block

---

[24] D. Rigby, "*A technique for automatic multi-block topology generation using the medial axis"*, NASA/CR FEDSM2003-45527 (2004).

[25] P.-E. Danielsson, "*Euclidean distance mapping, Computer Graphics and image processing*", (1980).

[26] J. Vleugels, M. Overmars, *Approximating generalized Voronoi diagrams in any dimension (1995*). Technical report UU-CS-1995-14 Utretcht University.

[27] H. Xia, P. Tucker, *Finite volume distance field and its application to medial axis transforms*, International Journal for Numerical Methods in Engineering 82(1) (2010).

[28] H. Xia, P. Tucker, *Fast equal and biased distance fields for medial axis transform with meshing in mind*, Applied Mathematical Modelling (2011).

[29] P. Tucker, *Differential equation-based wall distance computation for DES and RANS*, Journal of Computational Physics 190(1) (2003).

[30] Zaib Ali, James Tyacke, Paul G. Tucker, Shahrokh Shahparb, "*Block topology generation for structured multi-block meshing with hierarchical geometry handling*", 25th International Meshing Roundtable (IMR25), 2016.

[31] W. R. Quadros, *LayTracks3D: a new approach to meshing general solids using medial axis transform*, Procedia Engineering 82 (2014).

[32] I. Malcevic, *Automated blocking for structured CFD gridding with an application to turbomachinery secondary flows*, 20th AIAA Computational Fluid Dynamics Conference, Honolulu, Hawaii, 2011.

topology generation techniques surveyed above has been performed in[33]-[34]. The comparison has been carried out using an adjoint based error analysis of the meshes generated by these block topologies. It is found that, in general, the medial axis based approaches provide optimal blocking and yields better accuracy in computing the functional of interest. Mostly, domains having internal flows were used for this assessment. However, the medial axis based methods may not always yield an optimal block topology when dealing with complex 3D geometries and external flows. To overcome this limitation, a hybrid blocking technique is illustrated which makes use of the distance field iso-surface in addition to the medial axis transform. This is demonstrated using a wing-body-tail and a jet-wing-flap configurations. In addition to that, to reduce the meshing effort, a hierarchical geometry handling approach is also defined and applied to an engine-wing-flap configuration. These approaches are described next.

2.3.3 Hybrid Blocking

Consider an aero-engine jet-wing-flap (JWF) domain with a far-field as shown in the **Figure 2.3.11**. The medial axis close to the JWF geometry is shown in the enlarged view of 2D slice of domain in the Here the distance field and the corresponding medial axis is computed with respect to the geometry and the cylindrical far-field thus avoiding effect of the inlet and exit boundaries. The solid lines represent the shock features i.e. the medial axis and the dashed lines show the expansion features.

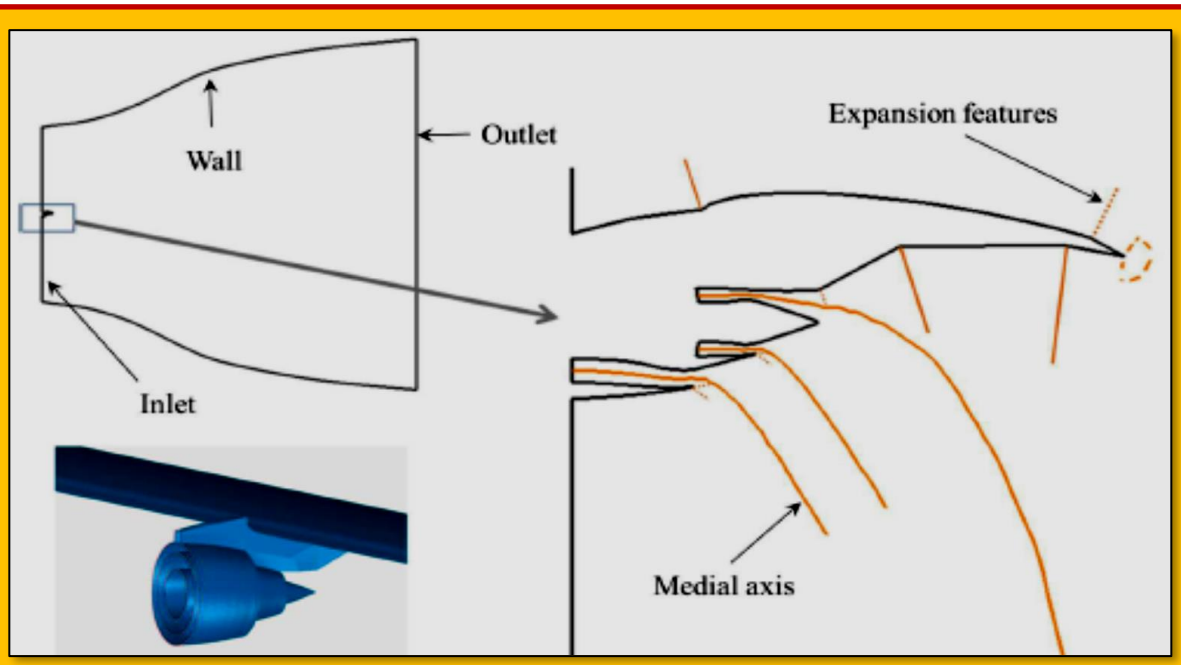


Figure 2.3.11 Jet-Wing-Flap: Medial Axis Transform (Compression Shock) and Expansion Features Close to the Geometry – Courtesy of [Ali et al]

Following this medial axis branches and even connecting the hanging and expansion features, a poor quality blocking would be achieved. This is also because, to generate small branches of the medial axis between the internal aero engine geometry parts would have to be combined with very large branches between the aero engine and the far-field.

---

[33] Z. Ali, P. G. Tucker, *Multiblock structured mesh generation for turbomachinery flows,* Proceedings of the 22nd International Meshing Roundtable, Springer, 2014.

[34] Z. Ali, *Optimal block topology generation for CFD meshing*, Ph.D. thesis, Department of Engineering, University of Cambridge, UK, 2015.

To overcome this limitation, the distance field function d can be used. An iso surface (contour in 2D) of *d* is wrapped around the geometry to facilitate the MAT based block topology generation. The wall

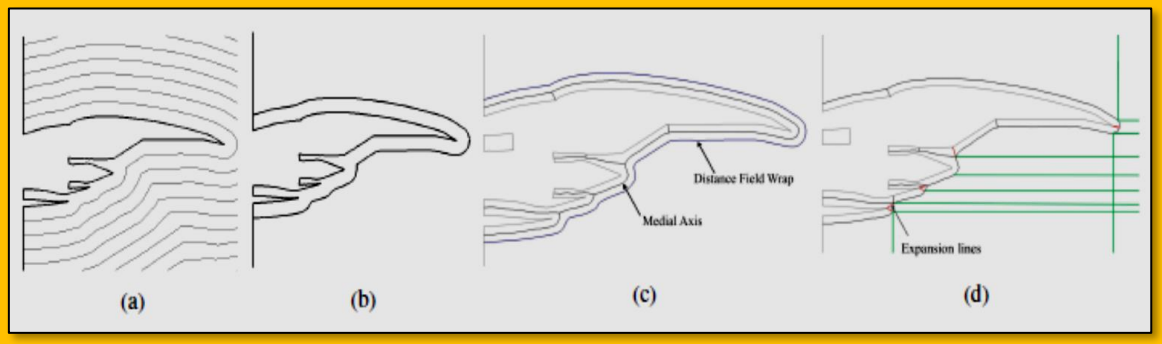


Figure 2.3.12 Two dimensional jet-wing-flap geometry: (a) the distance field; (b) distance field wrap and (c) corresponding medial axis (d) hybrid blocking around 2D geometry – Courtesy of [Ali et al]

distance computation is an intermediate step in the distance field based medial axis approximation and hence is available for use without any extra cost. Thus using an iso surface of the distance field instead of the far field for the medial axis computation can significantly improve the medial axis based blocking. The hybrid blocking procedure is described below with the help of a simple 2D JWF geometry. The extension of this methodology to the 3D cases also follows the same procedure as demonstrated later.

1. The distance field is computed around the domain of interest as shown in the **Figure 2.3.12 (a)**. An exact equation for the wall distance d is the hyperbolic ***eikonal equation*** which models the front propagation from the surface at unit velocity.

$$|\nabla d| = 1 + \Gamma \nabla^2 d$$

Eq. 2.3.1

where $\Gamma \rightarrow 0$ yielding viscosity solutions. The first arrival time of the front for a unit velocity is equal to the wall distance. The ***eikonal*** equation is solved using a fast marching method[35]. A suitable iso surface is then extracted from d. This iso surface selection is currently arbitrary but it can be linked to a criteria. For example, the width of the shear layer regions in the jet wake can dictate this selection upstream or it could be based upon the dimensionless wall distance y+ value. The iso surface acts like a virtual geometry or a wrap around the real domain (**Figure 2.3.12 (b)**).

2. The next step is approximation of the medial axis between the geometry and the distance field wrap. The Voronoi diagram based algorithm of [Dey and Zhao][36] is used here for the medial axis approximation. This algorithm provides a more stable and continuous medial axis for complex 3D domains than the voxel thinning approach. The input to this program is the point cloud data of the geometry and the distance field iso surface. It makes use of the observation that certain Voronoi facets are positioned close to the medial axis if their dual Delaunay edges tilt away from the surface or are very long. Hence, the angle condition and the ratio condition are defined to filter such tilted and long Delaunay edges and the medial axis is approximated. Let V_P be the Voronoi diagram for a dense point set P from a smooth

[35] H. Xia, P. Tucker, *Finite volume distance field and its application to medial axis transforms*, International Journal for Numerical Methods in Engineering 82(1) (2010).
[36] T. K. Dey, W. Zhao, *Approximate medial axis as a Voronoi subcomplex*, Computer-Aided Design 36 (2004).

compact surface $S \subset R3$. This Vornoi diagram is a cell complex comprising of Voronoi cells Vp $p \in P$ and their facets, edges and vertices. Also, for each point $x \in S$,

$$V_p = x \in \mathbb{R}^3 \; \|p - x\| \leq \|q - x\| \;, \qquad \forall q \neq p$$

Eq. 2.3.2

where p and q are any two points in P. Let D_P be the Delaunay triangulation of P and dual to the Voronoi complex. The Delaunay triangles incident to point p which are dual to the Voronoi edges intersected by a tangent plane at p are used to construct the criteria. All the Delaunay edges that make relatively large angle with the planes of the triangles are filtered. If the angle between the vector tpq from p to q and the normal $\mathbf{n}_{ptu}$ to a triangle ptu is less than a threshold angle $\pi/2 - \theta$ for all the triangles, then that associated Delaunay edge is filtered i.e.

$$\max \angle \; \mathbf{n_{ptu}} \;, \; t_{pq} < \frac{\pi}{2} - \theta$$

Eq. 2.3.3

gives good results. The ratio condition is based on the comparison of the length of the Delaunay edges with the circum-radii of the triangles. Thus those Delaunay edges are filtered which satisfy the criteria:

$$\min \frac{\|p - q\|}{r} > \rho$$

Eq. 2.3.4

Where $\|p - q\|$ defines the length of a Delaunay edge and ρ is the circum-radius of a triangle ptu. A value of $\rho = 8$ is normally used for dense point clouds. The medial axis is generated as a continuous surface which can be imported into the mesh generator. The medial axis for the JWF slice is shown in the **Figure 2.3.12 (c)**.

3. To complete the blocking process, additional rules as described in the Section 1 are manually used. Applying the rules, for example to the 2D JWF slice, the expansion features are connected to the nearest medial vertex or otherwise the medial axis as shown in the **Figure 2.3.12 (d)**.

Once the critical parts of the domain have been blocked using the medial axis, the far-field region can be partitioned using simple Cartesian fitting or H-type blocks. This is shown, for example, in **Figure 2.3.12 (d)** with the green lines. This resulting domain decomposition is significantly better than the one obtained initially shown in the **Figure 2.3.11**. There can still be some regions where the block topology is unsatisfactory. Such areas must be manually altered. Hence, a semi-automatic blocking process arises. The mesh is then generated in the commercial program Pointwise.

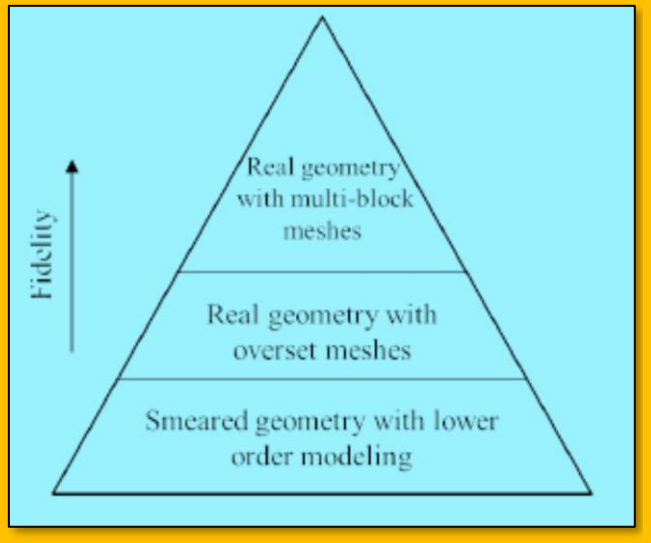


Figure 2.3.13 Hierarchical Geometry Handling Strategy

2.3.4 Hierarchical Geometry Handling

Modern day aero-engine design challenges demand performing realistic and multi-scale CFD simulations of strongly coupled

systems. This means that geometries are becoming highly complex and as a result more effort is consumed in the mesh generation and flow modeling process. This can have a strong impact on the duration of the design optimization cycle. To this end, a combination of the high and low fidelity (structured multi-block) meshing/modeling techniques can be employed using a hierarchy of the methods shown in the **Figure 2.3.13**. At the bottom of this hierarchy is the smeared representation of the geometry through the use of the lower order methods such as immersed boundary method (IBM) and body force model (BFM)[37-38]. Next in the hierarchy is the real geometry resolved by the overset meshes which present a useful option for integrating various domains together without adding extra complexity.

The information between the overlapping parts is exchanged through interpolation. At the top is the cost effective alternative to the fully resolved/meshed geometry, a combination of various high and low fidelity methods allows rapid addition and modeling of arbitrary geometry effects. This helps in reducing the design cycle time. The objective here is not to present any novel mesh generation method but to apply a zonal/hierarchical approach to handle complex domains to reduce the meshing effort. Such an approach can be efficiently used to rapidly explore the design space and can aid the development of high fidelity numerical tools.

2.3.5 Body Force Modeling

The engine-wing-flap geometry case to be presented later employs the hierarchical geometry handling approach and uses a ***body force method (BFM)*** to model the effect of the fan and outlet guide vanes. This model is outlined next. Assuming an infinite number of blades, the aerodynamic effects of a blade row are modeled using an axisymmetric flow in each infinitesimal blade passage and the body forces are added as source terms. The parallel and normal components of these forces in cylindrical coordinates system are given as:

$$F_{p,x} = \frac{k_p}{s} V_x V_{rel} \quad , \quad F_{p,\theta} = \frac{k_p}{s} V_\theta V_{rel} \quad , \quad F_{p,r} = \frac{k_p}{s} V_r V_{rel}$$

$$F_{n,x} = \frac{k_n}{s} \frac{V_\theta}{V_{rel}} f(V_x V_\theta, \alpha) \quad , \quad F_{n,\theta} = \frac{k_n}{s} \frac{V_x}{V_{rel}} f(V_x V_\theta, \alpha)$$

Eq. 2.3.5

Here, V_x, V_θ and V_r are the axial, tangential and radial velocity components. V_{rel} is the magnitude of the fluid velocity relative to the blade. K_p and K_n are calibration constants. Also, α and s are the local blade metal angle and blade pitch respectively. The above equations can also be modified to produce local blockage terms.

2.3.6 Results

2.3.6.1 NASA CRM Wing-Body-Tail

In this section, the hybrid blocking is applied to partition the domain around a 3D NASA Common Research Model (CRM) horizontal wing-body-tail configuration. This model represents a modern, transonic and commercial aircraft designed to cruise at $M_\infty = 0.85$ and $C_L = 0.5$. The geometric and aerodynamic details about the model as described. Further information can be obtained from the development by [Ali et al.][39]. The medial axis around the wing and tail also provides a block topology similar to O-type or C-type meshes. To assist the blocking, expansion features at the trailing edges of

---

[37] T. Cao, P. Hield, P. G. Tucker, *Hierarchical immersed boundary method with smeared geometry*, 54th AIAA Aerospace Sciences Meeting, AIAA Science and Technology Forum and Exposition, 2016, pp. 2016–2130.

[38] T. Cao, N. R. Vadlamani, P. G. Tucker, A. R. Smith, M. Slaby, C. T. J. Sheaf, *Fan-intake interaction under high incidence*, Proc. of ASME Turbo Expo, Seoul, South Korea, 2016. ASME Paper Number GT2016–56561.

[39] Zaib Ali, James Tyacke, Paul G. Tucker, Shahrokh Shahparb, "*Block topology generation for structured multi-block meshing with hierarchical geometry handling*", 25th International Meshing Roundtable (IMR25), 2016.

the wing and the tail are joined to the nearest medial axis. After the blocking around the geometry is

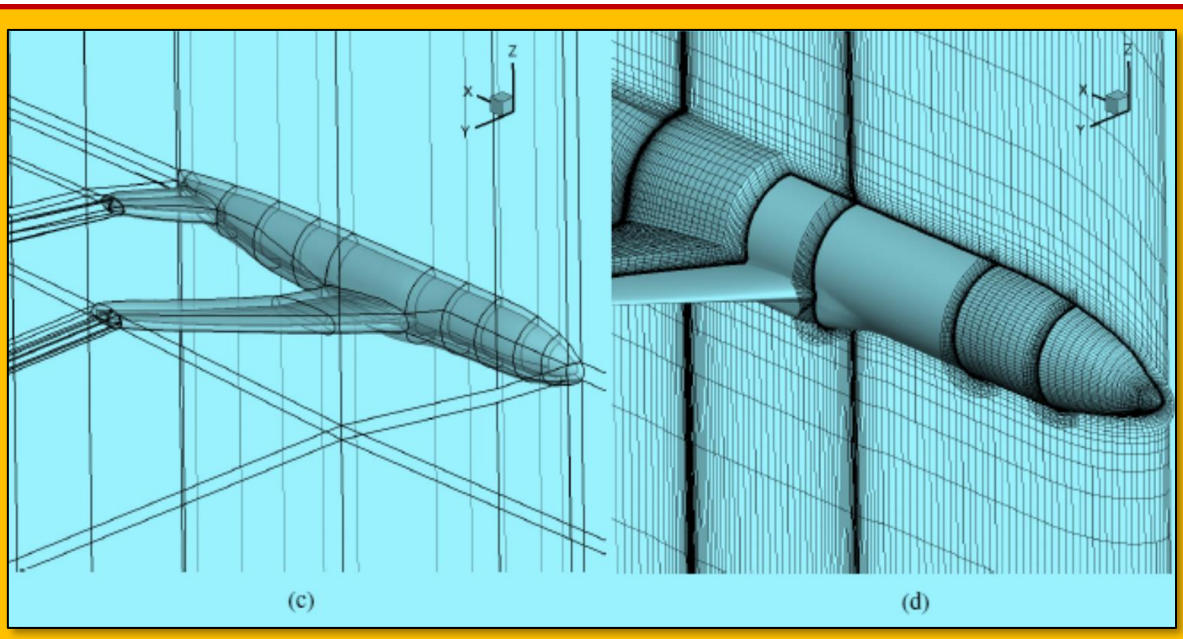


Figure 2.3.14 NASA CRM Wing-Body-Tail (c) Hybrid Blocking (d) Mesh Cut Section – Courtesy [Ali et al.]

complete, the far-field domain partitioning is carried out. The region is partitioned to create a H-type mesh. The block topology around the model is shown in the **Figure 2.3.14 (c).** The volume and the surface mesh cuts are displayed in the **Figure 2.3.14 (d)**. The NASA CRM configuration has been the test case for the 4th and 5th AIAA CFD drag prediction workshops (Vassberg et al.)[40]. The aim of the workshop is to assess the state-of-the-art in the CFD methods for aircraft aerodynamic analysis. Here, we use the same flow conditions as given in the workshop to compute the flow around the test case. The result of MAT based topology was to create 256 blocks in 2 minutes, and gridding in 6 minutes. The simulations are performed in ***HYDRA*** which is an unstructured, finite volume, edge-based and compressible flow solver using MUSCL based flux differencing.

M_∞	0.85
P_{total}	201326.9 Pa
T_{total}	310.93 k

Table 2.3.1 NASA CRM Free-Stream Conditions

The simulation is carried out at $M_\infty = 0.85$ and $C_L = 0.5$ with $Re = 5 \times 10^6$ based on the reference chord length C_{ref} = 7.00532 m. **Table 2.3.1** describes the free-stream flow conditions. A coarse mesh of approximately 4 M cells is used. The first grid node from the wall is located at y+ ≈ 1. The *Spalart Allmaras (SA)* turbulence model is used for this simulation. The flow angle for this mesh to gain $C_L = 0.5$ is $\alpha = 2.36$ deg.

2.3.6.2 Jet-Wing-Flap

In this section, the 3D jet-wing-flap case is presented. The geometry comprises of co-axial nozzle, pylon and a wing with a flap as shown in the **Figure 2.3.15 (a).** This realistic aero-engine geometry has been used for detailed computational aero-acoustics analysis. The pylon adds complexity to the otherwise cylindrical nozzle topology along with the wing and the flap. Hence, blocking such a case demands significant user insight. After wrapping the distance field, the medial axis is approximated. This is shown in the **Figure 2.3.15 (b).**

To simplify the blocking procedure, small medial axis branches are removed for this case. This is

[40] J. Vassberg, E. N. Tinoco, M. Mani, B. Rider, T. Zickuhr, D. Levy, O. P. Brodersen, B. Eisfeld, S. Crippa, R. A. Wahls, et al., *Summary of the Fourth AIAA CFD Drag Prediction Workshop (2010).* AIAA Paper No. AIAA-2010.

followed by the inner blocking aided by the rules which is shown in the **Figure 2.3.15 (c).** The far-field domain decomposition can be then be carried out at this stage. However, one of the aims for the aero-acoustic jet simulations is to properly resolve the shear layers in the far-field. This requires a good quality mesh aligned with the shear layer regions. The current block topology as shown in the **Figure 2.3.15 (c)** is non-optimal for properly resolving the shear layers produced by the bypass and

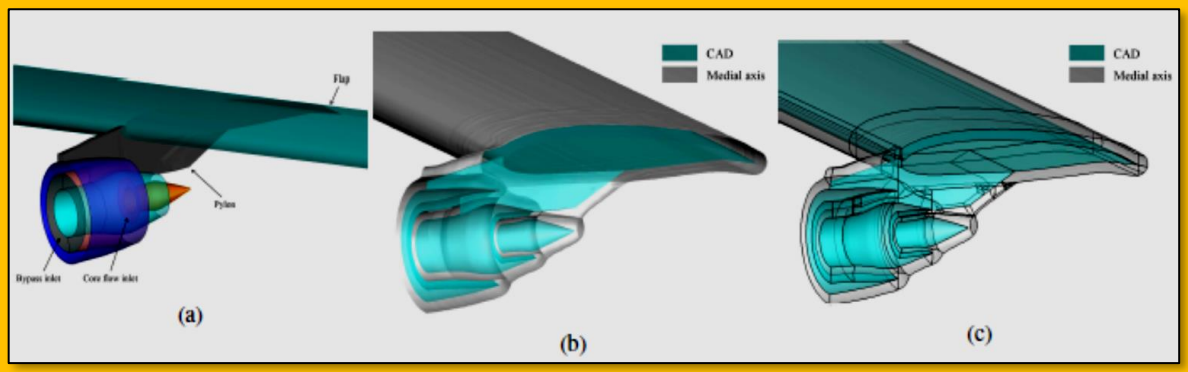


Figure 2.3.15 Jet-Wing-Flap (a) CAD, (b) CAD and the Medial Axis Cut Section, (c) Inner Hybrid Blocking – Courtesy of [Ali et al]

the core flows. Hence a manual alteration of the blocking around the pylon and the core flow exhaust is performed.

The modified inner block topology with the far-field decomposition are shown in the **Figure 2.3.16**. A RANS simulation using the SST k − ω is carried out on the mesh generated by the hybrid blocking. The first grid node from the wall is located at $y+ \approx 1$. The two cases presented above show how the hybrid blocking approach can be effectively used to decompose and mesh the complex geometries. The medial axis based domain decompositions also provide meshes having better flow alignment when compared to other partitioning methods e.g. Cartesian fitting and cross field based techniques. Hence, this technique further enhances the scope and applicability of these MAT based blocking methods. Also, the blocking templates could be generated using this approach which can speed up the mesh generation process and aid an inexperienced CFD user.

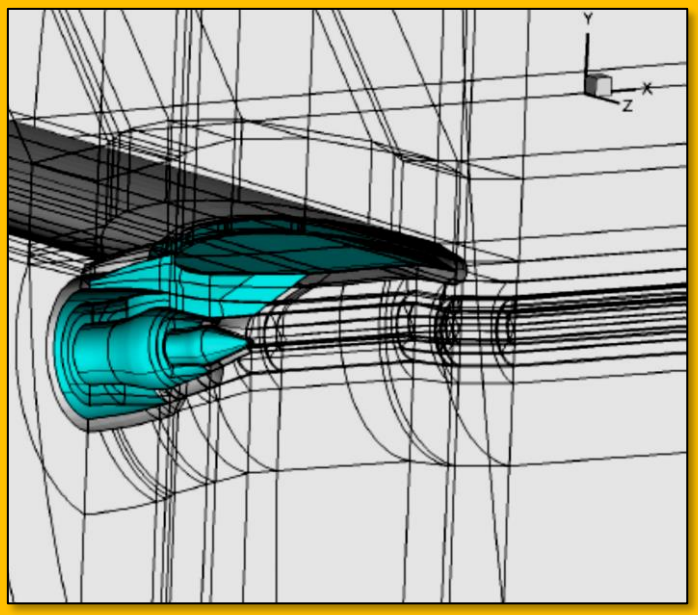


Figure 2.3.16 Jet-Wing-Flap with Modified Inner Blocking (To Accommodate Shear Layers) – Courtesy of [Ali et al]

2.3.6.3 Jet-Engine-Wing-Flap

In the last section, a coaxial nozzle with pylon and wing geometry was presented which makes the rear or downstream part of the aero-engine. To carry out a more realistic simulation, the front engine part containing the axisymmetric intake, hub and splitter geometry is added to this rear part using the overset mesh at the interface. This procedure avoids re-blocking the domains to have a cell to cell match between the two zones. Also, a smeared fan geometry is used

where the fan is modeled using the BFM (see the schematic in **Figure 2.3.17 (Top)**). The other downstream components are imprinted and treated again with the BFM but with localized sources. These internal geometry components include the downstream vanes, gearbox shaft, and the engine supporting A-frames. Thus using a hierarchical geometry handling approach, a complex domain can be readily meshed and analyzed for in a design optimization cycle. A cut section of the mesh at $z = 0$ plane is shown in the **Figure 2.3.17 (Bottom).** The total mesh size is 50 million. The internal geometry in the front part is modeled using a body force model which has been extended to include the local blockage and wakes modeling. This is done by adding local enhanced source terms to generate wake zones, which is similar to adding the source terms in the IBM for simulating geometry or boundary on a non-conformal Cartesian mesh.

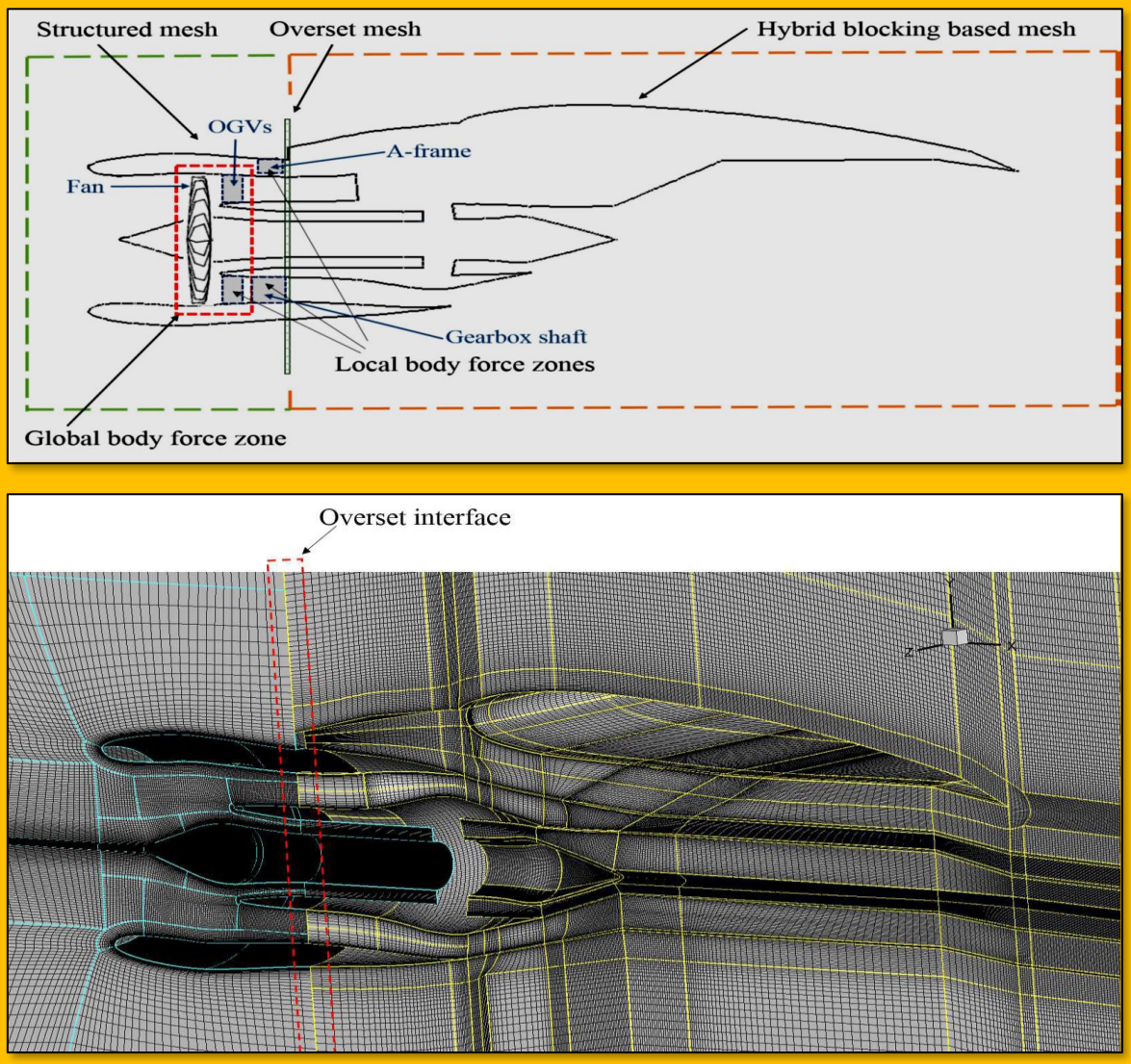


Figure 2.3.17 Engine-Wing-Flap (Top) Schematics Showing Geometric Zones and Domains with Different Block Topologies (Bottom) Cut Section at z = 0 Plane – Courtesy of [Ali et al]

2.4 A New Perspective on Structured Multi-Block Meshing for Turbine Blades From GridPro©

2.4.1 Preliminaries & Background

Meshing techniques for turbomachinery flow fields is well established and has evolved and matured over the last couple of decades[41]. Early CFD practitioners made use of various flavors in structured multi-block strategies like the H-type, C-type, O-type, etc. and were satisfied with the CFD results. However, over time, the turbine blade designs started to take on complex forms with larger twists, narrower flow passages, higher pitch, newer geometrical features like cooling holes with internal passages, cut back trailing edges, shroud cavities, snubbers etc. and the industry workhorse(in terms of meshing) i.e., structured blocking strategies were inadequate to handle these newer challenges.
With the emergence of unstructured and cartesian meshing techniques, Engineers took a detour from the structured approaches and started to embrace these newer meshing algorithms. However, many soon realized that, although the newer approaches, facilitated for faster discretization and lesser human intervention, the ultimate objective of getting accurate, reliable computational results was in jeopardy. This paved the way to look for an amalgamation of techniques, giving rise to hybrid grids. The emerging thought was, to make use of structured blocks where ever possible especially in critical flow regions and to switch to unstructured methods, in regions of geometric complexities. With this, the discretization of complex turbomachinery domains has been possible. The hybrid approach, though not an ideal solution, is largely accepted in the CFD community.
Interestingly, structured multi-block researchers haven't given up their quest for faster, reliable and robust meshing. Time and again, they have come up with newer, smarter techniques to meet the gridding challenges. Innovative blocking strategies like staggering, local enrichment, nesting and faster meshing through automated blocking, template-based approaches, etc., The approaches exploit the advantages of unstructured meshes and has made structured gridding possible even for geometries which were thought to be un-mesh able a decade ago. Structured grids are regaining back their lost prominence and are still the darling of the turbomachinery practitioners.

2.4.2 Meshing Turbine Blade Profiles vs. Airfoils ?

Turbine blades outwardly look very similar to aircraft wings and this implants a wrong notion in our minds that meshing turbine blades are similar to meshing wings. Though topologically same, meshing turbine blades are a lot trickier, even at a 2D profile level. In external aerodynamic computations such as airfoils, the domains are large and there is a lot more free space around. This

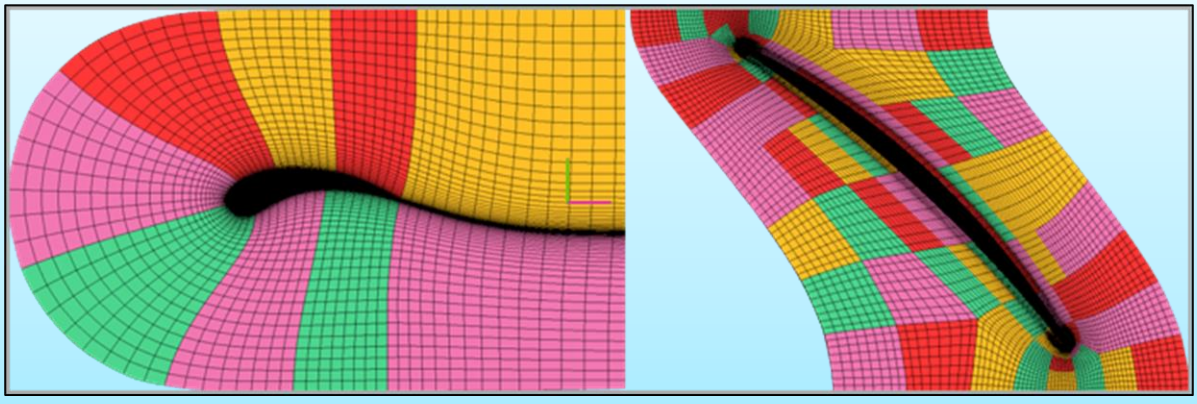


Figure 2.4.1 Structured Multi-Block Grid Around an Airfoil and a Turbine Blade

[41] Ravindra Krishnamurthy, *GridPro Blog*, March 29, 2019

gives you the freedom to easily construct the blocks which can place itself orthogonally around the body and the outer domain with almost zero skewness. The extra space provides the luxury for grid blocks to relax, adjust and re-position themselves to provide a high-quality grid. On the contrary, due to the close proximity of inlet/outlet and periodic boundaries, attaining orthogonal grids with an acceptable skewness becomes a challenge. Especially when high curvature blades with large periodicity, the flow passage shrinks and contours, which makes meshing them a lot harder. Though block creation may take less time, having them aligned to the domain boundaries and also ensuring periodicity, grid quality, especially at the LE and TE becomes an uphill task. (**Figure 2.4.1**).
In 3D, the complexity scales up multiple times with turbine blades having twists in the spanwise direction and with narrow tip clearance gaps, it becomes a nightmare. As a consequence, the turbine blades need blocking strategies which are fairly different from that needed for a wing. Interestingly, when seen from an unstructured perspective meshing an airfoil or a turbine blade are very similar. Abundance or lack of space no more becomes an issue, the unstructured grids adjust accordingly to give a healthy grid. ***If such is the flexibility offered by the unstructured methods, then why do CFD engineers still wish to mesh their blades the structured way?***

2.4.3 Why Still Live In A Hex Mesh World?

Cartesian and unstructured meshing techniques are fast, time-efficient and are more amenable for automation. But they fall short in providing the quality and solution level accuracy the Engineers are looking for. On the other hand, well-organized flow aligned hexahedral cells in structured grids provide reliable results with lower truncation and discretization errors. This results in faster solver convergence, more robustness, higher solution feature capturing and more accurate CFD predictions. Where unstructured scores are in situations when time to mesh is a critical factor. Also, structured grids lose the edge when the flow path is unknown or is rapidly changing. Further, unstructured grids can provide flexible resolution control through local clustering. More importantly, unstructured grid-generators can handle geometric complexities which most structured grid generators can't even think of. However unstructured approach loses the battle in situations where it is required to obtain almost identical grids even when the geometric profile is changing. A classic example can be the turbine blade optimization cycle, where the optimizer throws multiple variants and CFD engineer wishes to have the same set of grids points to capture the variants. This requirement is very efficiently met in a structured approach even for a highly twisted blade.
Since unstructured is fully automated, the control over the number and the shapes of the elements poses a challenge. They fail for highly twisted blades with narrow flow passages, as grid skewness will exceed acceptable thresholds. In order to preserve the prismatic topology of the unstructured region, the blade-to-blade mesh must be generated on a single section and then cloned to other sections. This will lead to excessive shearing of the prisms for highly staggered 3D blades. Also, in many cases along with the topology, the grid density needs to be kept as consistent as possible to the baseline grid in order to minimize the errors in computing the flow sensitivities. Unfortunately, this is particularly difficult to control when using unstructured grid generators as the total number of grid points cannot be fixed a priori.

2.4.4 Conventional Blocking Strategies

Structured blocking strategies for turbomachinery applications can broadly be classified into passage centered and blade centered topologies. **Figure 2.4.2** shows the two approaches. In ***passage centered*** approach, the periodic boundaries cut right across the leading and trailing edges of the blade. Though this approach results in generating good grids with less skewed cells for highly cambered blades with large pitch angles and narrow flow passages, they are more prone to trigger stability issues while running the solver. This is mainly because periodic boundaries in high gradient areas such as the leading edge and trailing edge could influence the flow field and reduce the solution accuracy for certain cases. On the other hand, in the ***blade centered*** approach, the periodic boundaries are placed away from the blade. This reduces the negative influences of the periodic boundaries on flow physics in the near vicinity of the blade and thereby improve solver stability, robustness and accuracy of CFD simulation. Moving further, the next level of classification can be made depending on the block arrangements around the blade. Three popular approaches are in usage namely, the H-type, the C/O-type and the H-O-H type.

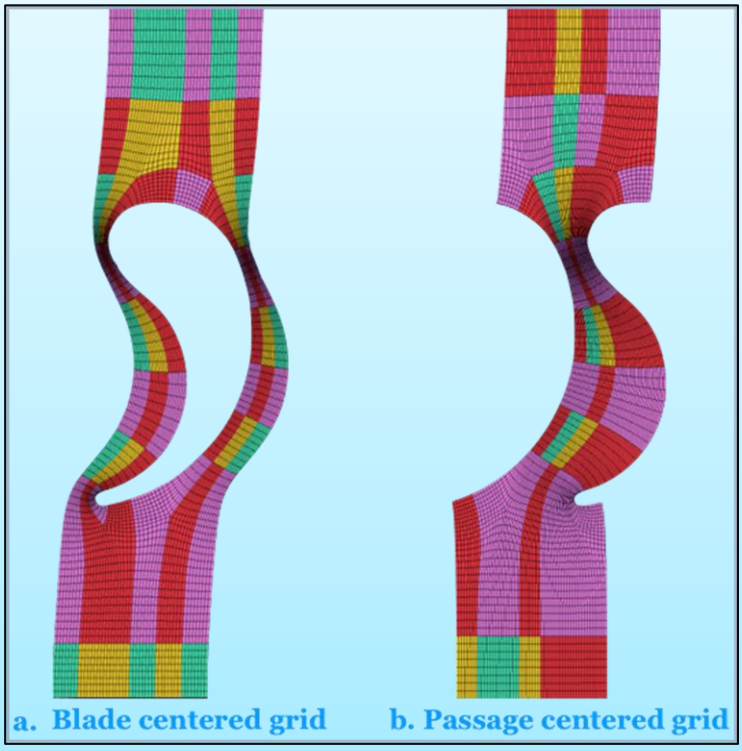


Figure 2.4.2 Structured Multiblock Grid Generated Using Blade Centered And Passage Centered Topology

2.4.5 How Should My Ideal Mesh For Blades Look Like?

For a large range of camber and pitch, just H-O-H topology generates good grids. But their limitations are exposed for cases with higher values of camber and pitch. **Figure 2.4.3** shows the ideal block flow pattern for the specified blade. In **Figure 2.4.4A** we can see that a regular H-O-H topology gives non-orthogonal blocks at the boundaries, far away from our ideal requirements. What we need is a strategy which will control, re-orient and guide the blocks in the direction we want. We need to convert the block flow pattern in A to

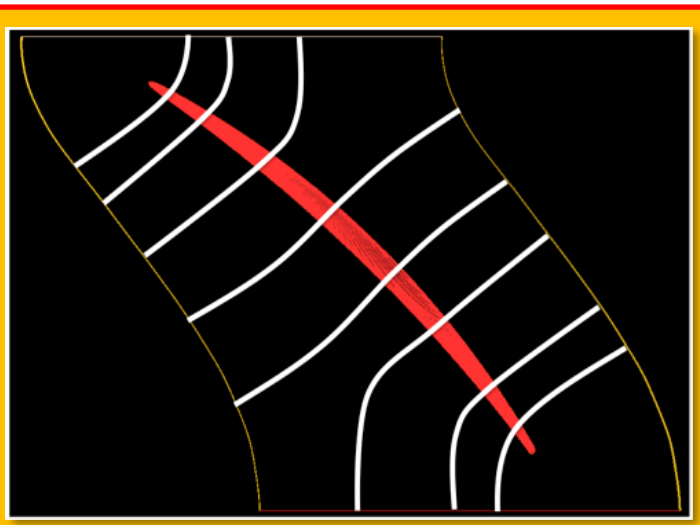


Figure 2.4.3 Ideal Block Flow Pattern For The Turbine Blade

that in B. And this can be achieved by what is called staggering.

2.4.6 How Should My Ideal Grid Look Like?

For a large range of camber and pitch, just H-O-H topology generates good grids. But their limitations are exposed for cases with higher values of camber and pitch. **Figure 2.4.3** shows the ideal block flow pattern for the specified blade. In **Figure 2.4.4 A** we can see that a regular H-O-H topology gives non-orthogonal blocks at the boundaries, far away from our ideal requirements. What we need is a strategy which will control, re-orient and guide the blocks in the direction we want. We need to convert the block flow pattern in A to that in B. And this can be achieved by what is called staggering.

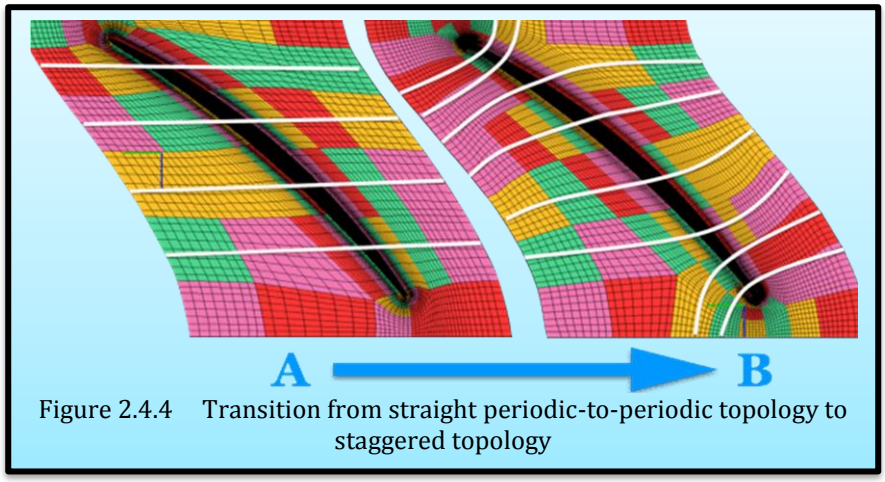


Figure 2.4.4 Transition from straight periodic-to-periodic topology to staggered topology

2.4.7 Staggered Approach

Attaining high grid quality and orthogonality becomes a stiff challenge for highly cambered blades with larger periodic angles. Reduction in space starts taking a toll on grid quality. The cell skewness, aspect-ratio starts to increase especially around leading and trailing edges. These grid issues arise due to the presence of periodic boundary condition which demands one-to-one periodic match-up of grid points. With reduced flow passage, the grid lines have difficulty to turn sharply and align themselves with the inlet and outlet boundaries.

To overcome these gridding issues, gridding engineers have developed what is called as ***staggered-grids.*** Staggering is a blocking approach where the grid-blocks do not necessarily start from inlet to outlet or from periodic to periodic. They could start at any given boundary and end on a perpendicular boundary rather than a parallel boundary. These blocking strategies in a given flow-path ensure highly orthogonal grids at every boundary ensuring an overall high-quality grid. The staggering is even possible easily in *GridPro*® because of its automatic periodic surface creating feature. This feature does contoured periodic surface itself between the passages to give the user a perfectly periodic and highly flexing boundary. The staggering can be in different flavors. Depending on the need, staggering is made from inlet/outlet-to-periodic BCs, or from periodic-to-periodic BCs, or inlet-to-outlets and in certain cases truly 3D with staggering from hub-to-shroud passing through the blade.

2.4.7.1 Case 1: Staggering from Inlet/Outlet to Periodic

Here we start with a regular H-O-H topology (**Figure 2.4.5**). As seen, the resultant mesh is non-orthogonal in the periodic boundary and stretched especially in the narrow passage between the blade. This is highlighted in **Figure** 2.4.6**.** The primary reason for this behavior is the delicate balancing needed to have grids orthogonal on the blade and the boundaries, with the additional requirement of the boundary grid points to be periodic to each other. In order to relax the tension between the blocks, the blocks on the left need to be pushed upwards while the blocks on the right need to be moved downwards as shown in **Figure 2.4.7**.

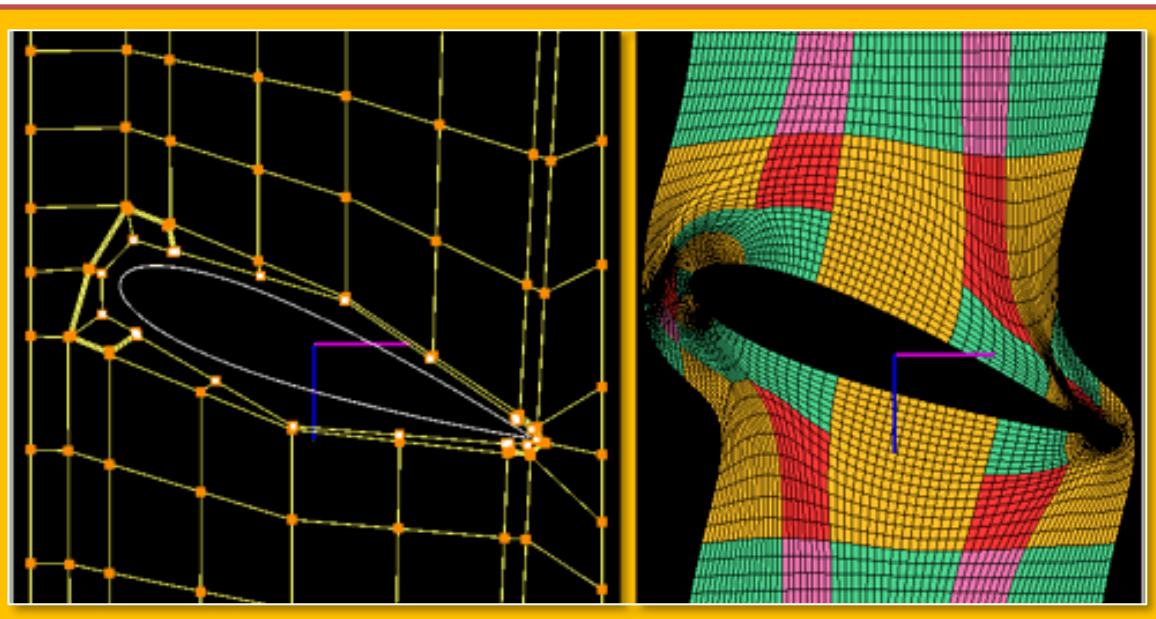


Figure 2.4.5 Basic H-O-H Topology

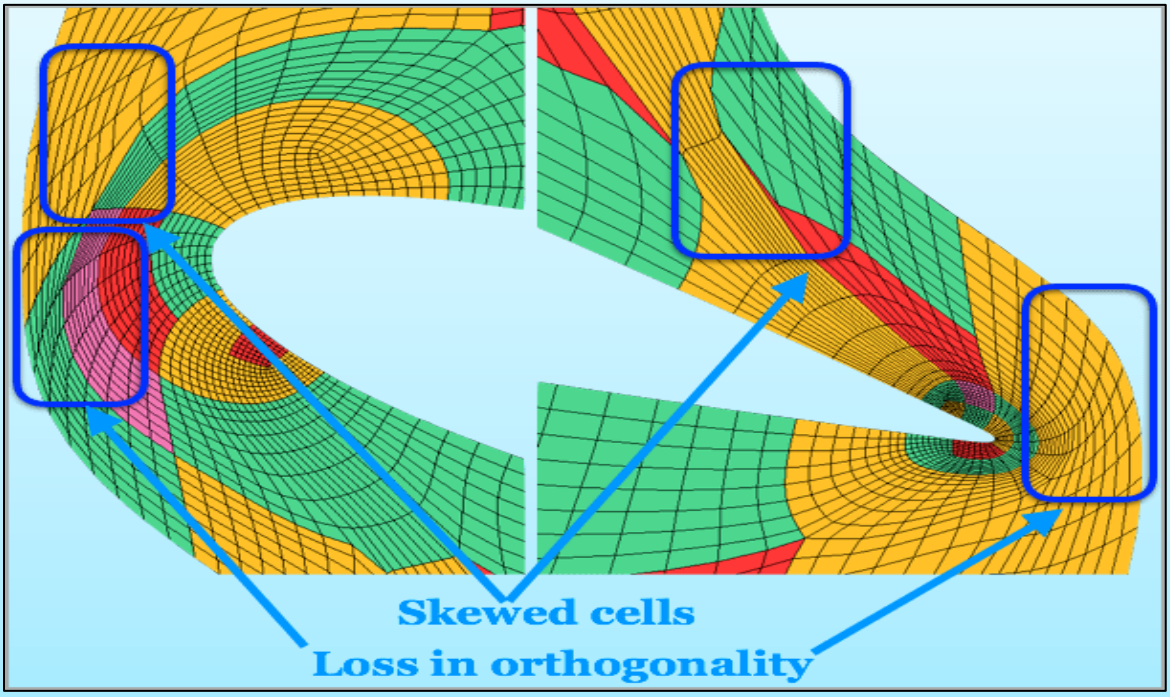


Figure 2.4.6 Skewed Cells And Lose in Orthogonality Observed Near the Leading and Trailing Edge

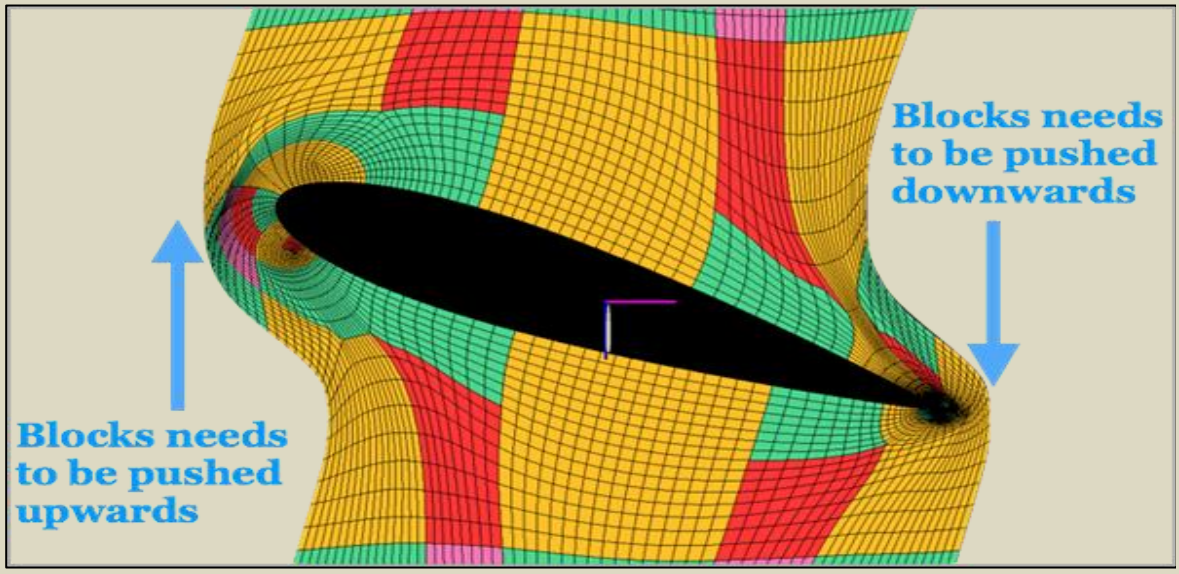


Figure 2.4.8 Block Transition Needed To Improve Grid Quality

To push the blocks and to have the grid points periodic, it becomes necessary to stagger the blocks. This can be achieved by splitting the blocks with the faces as highlighted in **Figure 2.4.7A**. The block splitting is done from the inlet to periodic and from the outlet to periodic BC. In **Figure 2.4.9** and **Figure 2.4.10** the resultant grid is obtained which satisfies all the grid quality criterion for a good grid (2D & 3D). The skewness, stretching at the LE and TE are completely eliminated and the orthogonality at the blade and periodic boundaries are enforced.

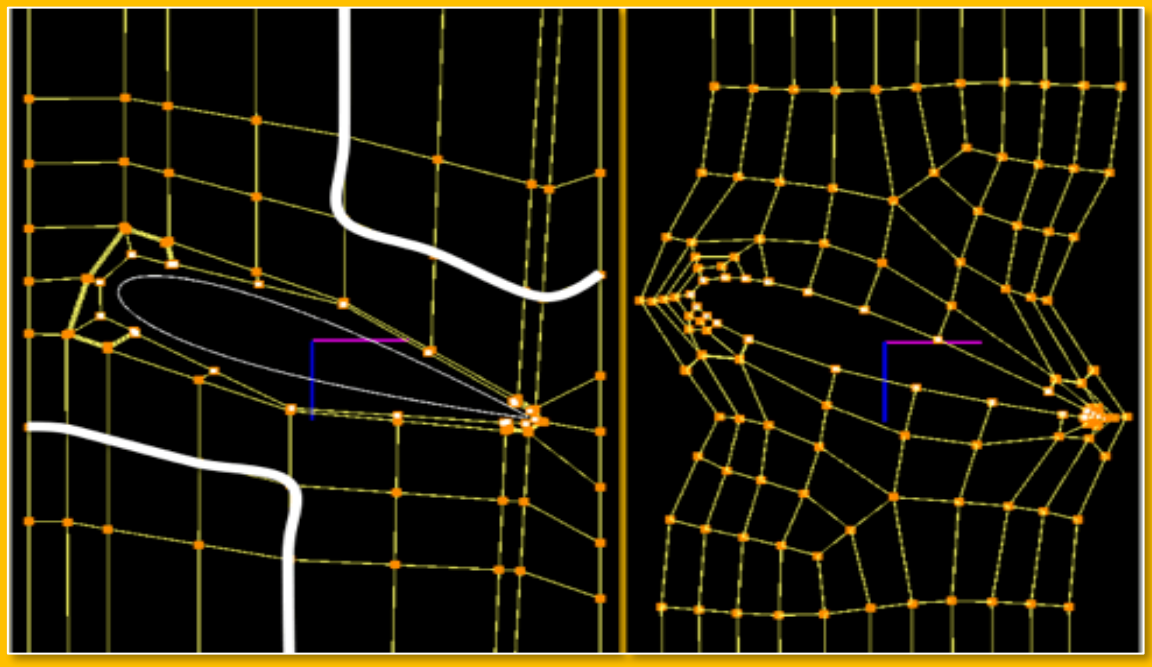


Figure 2.4.7 Staggering Achieved by Splitting : A) The Blocks Along the Path Shown from Inlet/Outlet to Periodic BC, B) Shows the *Staggered Topology.*

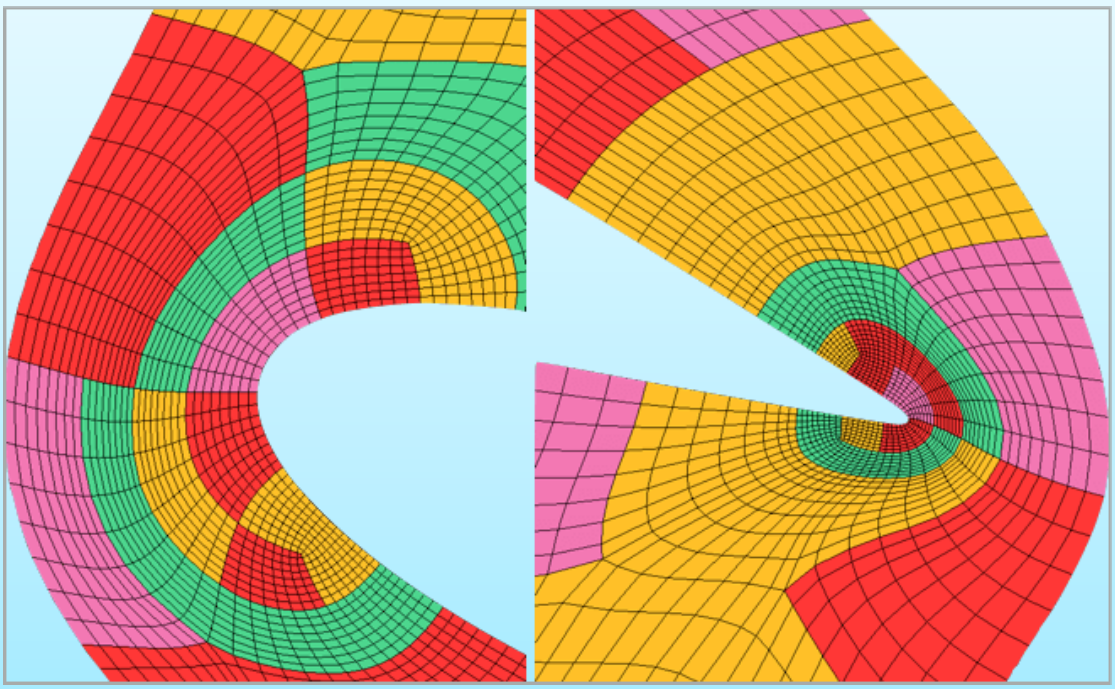


Figure 2.4.9 Grid Skewness improved and Orthogonality established at the Boundaries

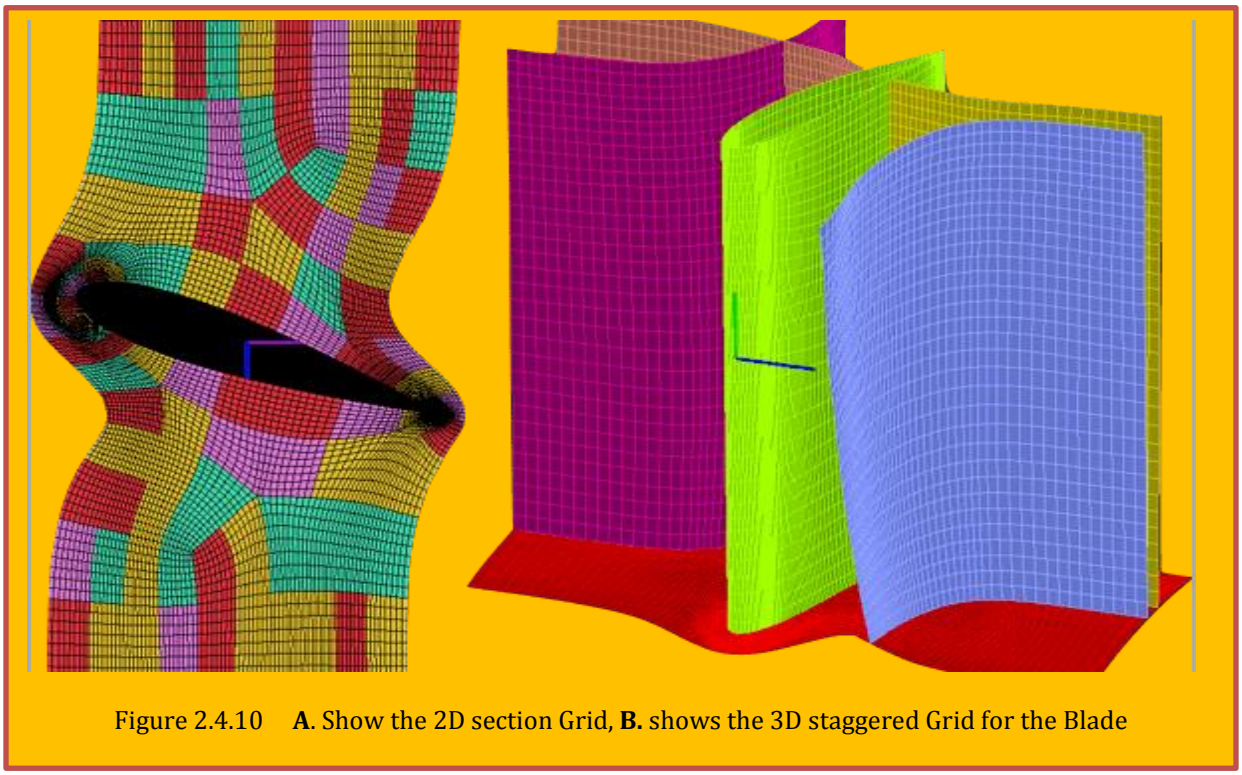


Figure 2.4.10 **A**. Show the 2D section Grid, **B.** shows the 3D staggered Grid for the Blade

2.4.7.2 Case 2 : Staggering across Blade (Two Blades)

Figure 2.4.11 - **Figure 2.4.12**, shows another staggering scenario. In **Figure 2.4.11** as the regular periodic-to-periodic blocking with non-orthogonal block-placement at the blade and periodic surfaces is shown. **Figure 2.4.11 (b)** presents how a staggering block pattern can be introduced. Staggering not only provides a novel way of approaching blade meshing but also allows the user to have additional features incorporated like the trailing edge enrichment in this case. What this means to the user is, the space constraint is managed well to resolve the meshes in regions where the resolution is needed to capture the physics. **Figure 2.4.12** shows the grids obtained for the blade with tip-clearance. Staggering need not be limited to the above scenarios alone, they could be done from hub to shroud as well. Especially in a marine propeller meshing, it eliminates the skewness and orthogonality issues particularly when they are highly twisted. The staggered blocks in the above case originate on the hub, pass through the blade and end on the shroud.

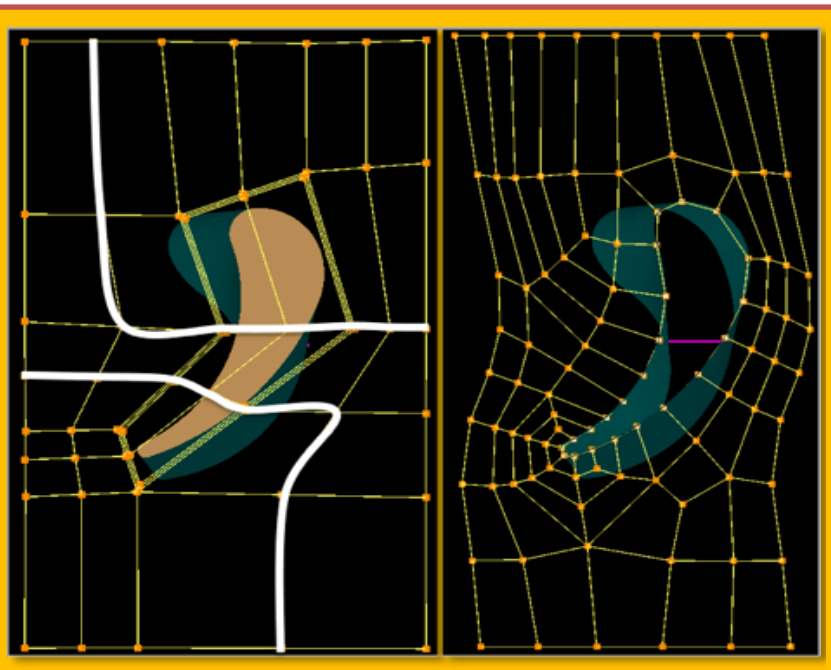


Figure 2.4.11 **A.** Shows the faces to be split to convert regular periodic-to-periodic topology to staggered topology. **b.** shows the final staggered wireframe

2.4.8 Wrapping-Up

Structured multi-block grids are the ideal grids one can think for meshing turbine blades. Usage of H-O-H topology helps to get good resolution, smooth and orthogonal grids for blades with large camber and rounded leading and trailing edges. Even blades with

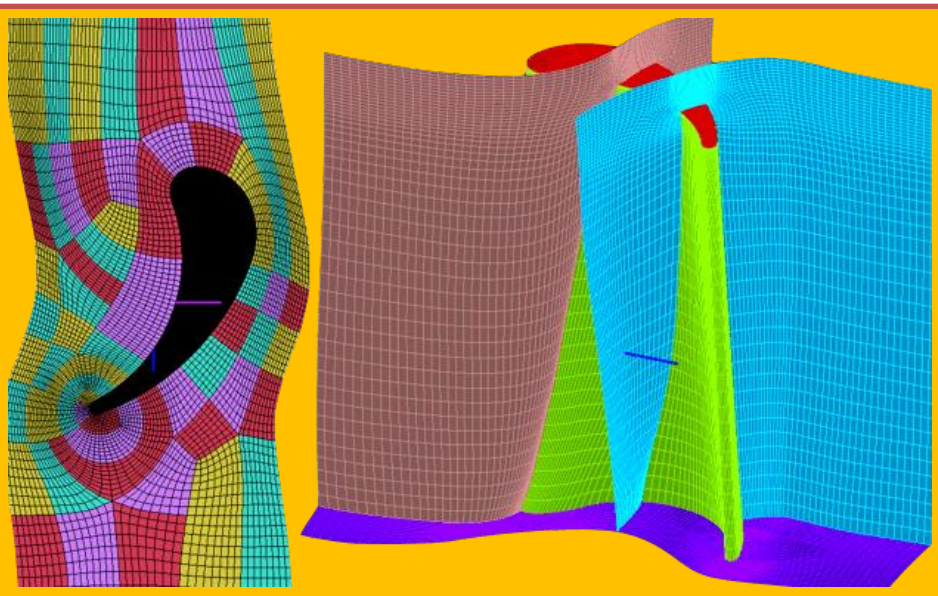
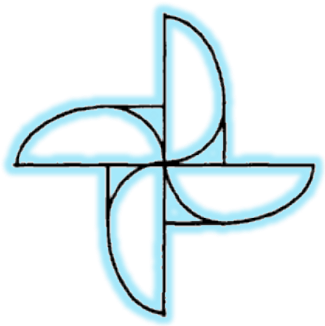


Figure 2.4.12 3D Staggered Grid With Tip Clearance

large camber and high pitch can be effectively meshed by introducing staggering. In such scenarios, the flow passages are very narrow and this lack of space results in skewed stretched cells and loss in orthogonality. Stagger smartly aligns and adjusts the blocks to get healthy grids. With this, we come to the end of this first article on the art and science of meshing turbine blades. In the next article, we will look into aspects of single topology approach for automated gridding.



3 Structured Mesh Generation

In general, decomposition of the physical domain produces several blocks. Each block is usually defined by six sides, and each side can be defined by either a surface, plane, line, or a point. If one side of a block collapses to a line or a point, then there would be a singularity in the block. In some instances, a block may have been defined by less than six surfaces. Once the surfaces are defined, the interior grid can be computed by any standard grid generation technique. The cell information stored in a 3D array, in random fashion and could be easily access.

3.1 Complex Variables

Complex variables techniques have the advantage that the transformation used are analytic as opposed to those methods that are entirely numerical. Unfortunately, complex variable method are restricted to two dimension. For this reason, the technique has limited applicability and will not be covered here. For details readers should refer to Churchill[42], Moretti, and Davis.

3.2 Algebraic Methods -Transfinite Interpolation (TFI)

Transfinite Interpolation has been used to generate the interior grid points from the boundary surfaces. In 2D (I, J), we may inscribe a linear Lagrange interpolation function as:

$$r(\xi,\eta) = \sum_{n=1}^{N} \phi_n\left(\frac{\xi}{I}\right) r(\xi_n,\eta) + \sum_{m=1}^{M} \psi_m\left(\frac{\eta}{J}\right) r(\xi,\eta_m) - \sum_{n=1}^{N}\sum_{m=1}^{M} \phi_n\left(\frac{\xi}{I}\right)\psi_m\left(\frac{\eta}{J}\right) r(\xi_n,\eta_m)$$

Eq. 3.2.1

Where now the "***blending***" functions, φ_n and ψ_m, are any functions which satisfy the cardinality conditions:

$$\phi_n\left(\frac{\xi_L}{I}\right) = \delta_{nL} \quad n, L = 1, 2,...,N \quad \text{and} \quad \psi_m\left(\frac{\eta_L}{J}\right) = \delta_{mL} \quad m, L = 1,2,....,M$$

Eq. 3.2.2

3.2.1 Blending Function

The interpolation function defined by **Eq. 3.2.2** can be thought of two unidirectional interpolation (Zhou)[43] the corner points which has been duplicated. With N = M = 2, using the ***Lagrange interpolation polynomials*** as the blending functions, is termed the transfinite bilinear interpolant. With N = M = 3, this form is the transfinite bi-cubic-interpolation. Other candidates for the blending functions are the ***Exponential, Hermit Interpolation Polynomials*** and ***Splines***. For example, for n, L = 2, **Eq. 3.2.3** shows a typical ***Exponential blending function*** as:

$$r(\xi) = Y_i + [Y_i - Y_{i-1}]\frac{\text{Exp}\left[(B_{i-1})\,\dfrac{\xi - X_i}{X_i - X_{i-1}}\right] - 1}{\text{Exp}\,(B_{i-1}) - 1} \qquad \text{for} \quad X_{i-1} \le \xi \le X_i$$

$$\text{where} \quad 0 \le X_i, Y_i \le 1 \quad , \quad 0 \le \xi, r(\xi) \le 1 \quad , \quad i = 2, , , , m$$

Eq. 3.2.3

---

[42] Churchill, R., V., "*Introduction to Complex Variables*", McGraw-Hill, New York.

[43] Quanbao Zhou, "*A Simple Grid Generation Method*", International Journal for Numerical Methods Fluids, 1998.

The integer m represents number of control points with coordinates {Xi, Yi}. The quantity B_{i-1}, called the ***stretching*** parameter, is responsible for grid density. Specifying B_1, values of $B_i \geq 2$ are obtained by matching the slopes at control points. This, guaranteeing a smooth grid transition between each region, can be accomplished using Newton's iterative scheme which is quadratically convergent. The greater the B , the less discontinuity will propagate. Similarly, a blending function could be constructed for η direction. The spline-blended form gives the smoothest grid with continuous second derivatives[44]. An example of exponential stretching (B_i = -2) is given by **Figure 3.2.1**.

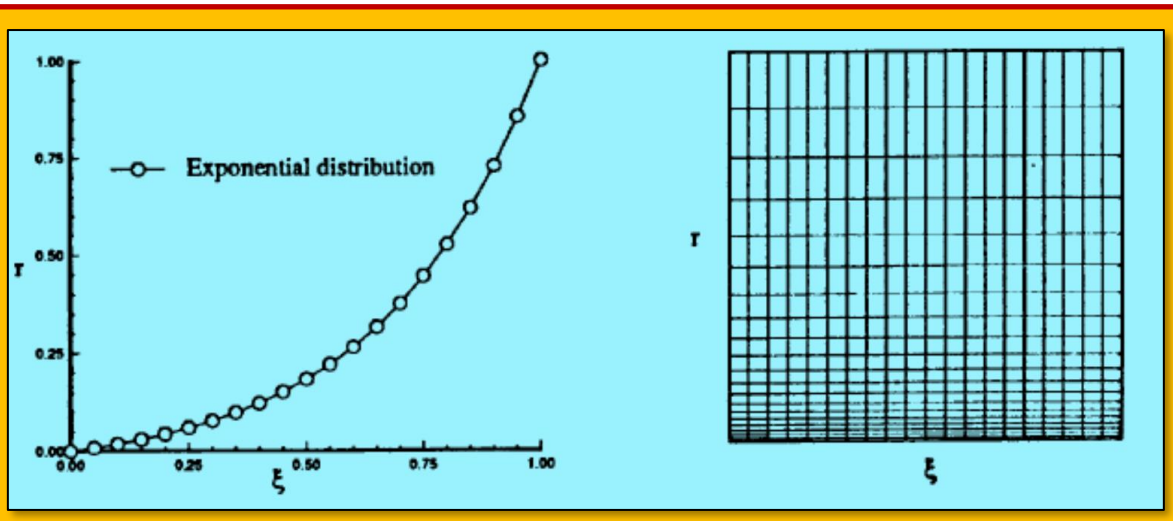


Figure 3.2.1 Exponential Distribution Functions

The exponential function, while reasonable, is not the best choice when the variation in grid spacing is large. The truncation error associated with the metric coefficients is proportional to the rate of change of grid spacing. A large variation in grid spacing, such as the one resulting from exponential function, would increase the truncation error, hence, attributing to the solution inaccuracies. A suggested alternative to exponential function has been the usage of hyperbolic sine function given as

$$r(\xi) = Y_i + [Y_i - Y_{i-1}] \frac{\sinh\left[(B_{i-1})\ \dfrac{\xi - X_i}{X_i - X_{i-1}}\right]}{\sinh\ (B_{i-1})} \qquad \text{for} \quad X_{i-1} \leq \xi \leq X_i$$

$$\text{where} \quad 0 \leq X_i\ ,\ Y_i \leq 1 \quad , \quad 0 \leq \xi\ ,\ r(\xi) \leq 1 \quad , \quad i = 2, , , , m$$

Eq. 3.2.4

The hyperbolic sine function gives a more uniform distribution in the immediate vicinity of the boundary, resulting in less truncation error. This ***criteria*** makes the hyperbolic sine function an excellent candidate for boundary layer type flows. A more appropriate function for flows with both viscous and non-viscous effect would be the usage of a hyperbolic tangent function such as

[44] Joe F. Thompson, Z. U. A. Warsi, C. Wayne Mastin, "*Numerical Grid Generation -Foundations and Applications*", North Holland, 1985.

$$r(\xi) = Y_i + [Y_i - Y_{i-1}] \frac{\tanh\left[\frac{(B_{i-1})}{2}\left(\frac{\xi - X_i}{X_i - X_{i-1}} - 1\right)\right]}{\tanh\left[\frac{B_{i-1}}{2}\right]} \quad \text{for} \quad X_{i-1} \le \xi \le X_i$$

$$\text{w here} \qquad 0 \le X_i, Y_i \le 1 \quad , \quad 0 \le \xi, r(\xi) \le 1 \quad , \quad i = 2, , , , m$$

Eq. 3.2.5

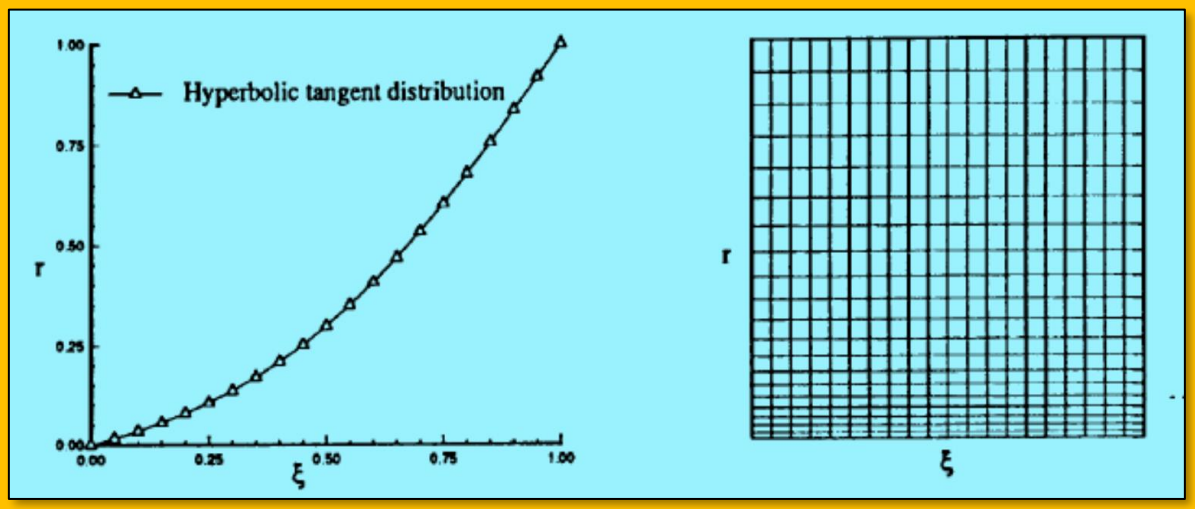


Figure 3.2.2 Hyperbolic Tangent Distribution Functions

The hyperbolic tangent, as revealed in **Figure 3.2.2**, gives more uniform distribution on the inside as well as on the outside of the boundary layer to capture the non-viscous effects of the solution. ***Such overall improvement, makes the hyperbolic tangent a prime candidate for grid point distribution in viscous flow analysis***, as publicized in **Figure 3.2.3** for a generic wing-fuselage. A numerical approximation can be used to compute the grid-point distribution on a boundary curve. This approach is widely used for complex configurations and care must be taken to insure monotonicity of the distribution. For example, the natural cubic spline is C^2 continuous and offers great flexibility in grid spacing control. However, some oscillations can be inadvertently introduced into the control function. The problem can be avoided by using a smoothing cubic spline technique and specifying the amount of smoothing as well as control points. Another choice would be the usage of a lower order polynomial such as ***Monotonic Rational Quadratic Spline (MRQS)*** which is always monotonic and smooth. Other advantage of MRQS over cubic spline is that it is an explicit scheme and does not require any matrix inversion. A sample coding in FORTRAN is given in Appendix A and the resultant grid and topology for a dual-block generic airplane geometry is displayed. A pioneering work in ***control point*** form of Algebraic Grid Generation using a univariate

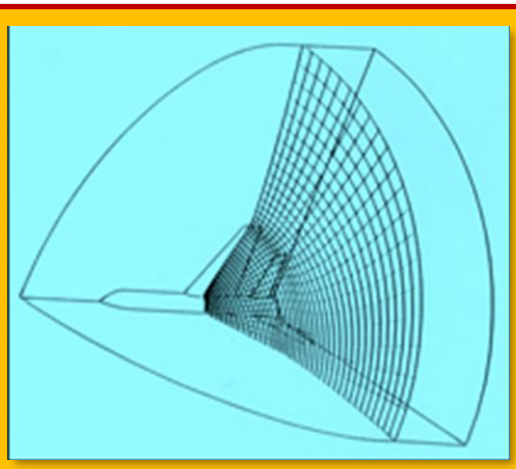


Figure 3.2.3 Grid for Dual-Block Generic Wing-Fuselage Geometry

interpolations can be attributed to [Eiseman and Smith][45].

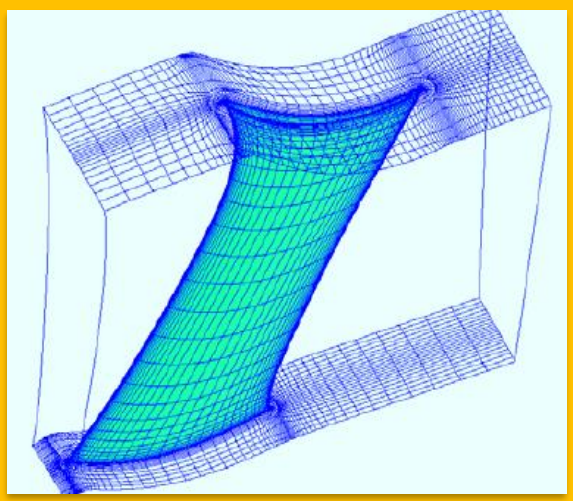
3.2.1.1 Case Study- Rapid Meshing System For Turbomachinery

PADRAM (Parametric Design and Rapid Meshing System), grid generation starts by dividing the computational blocks into sub-blocks for the purpose of generation of the algebraic grid and the control functions. O-type grid is used for the blades and H-type grids near the periodic boundary, upstream and downstream blocks, C-type grids are used for semi-infinite boundaries such as the splitter, pylon and RDF (Radial Drive Fairing). Transfinite interpolation (TFI) is used to generate the initial grid based on a linear interpolation of the specified boundaries. TFI is very easy to program, computationally efficient and the grid spacing is under direct control. *PADRAM* uses the following double clustering functions [Shahpar and Lapworth][46]:

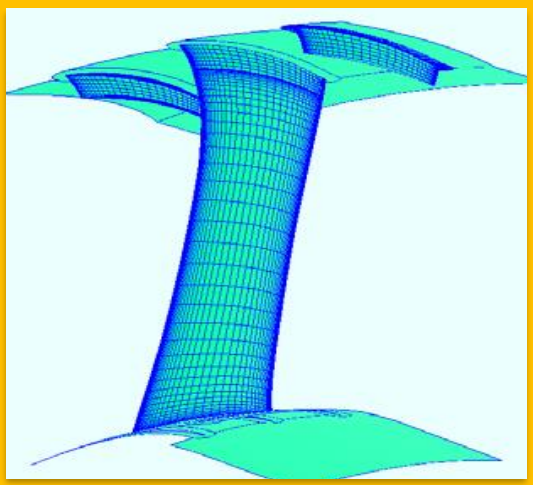
$$y = \frac{(2\alpha+\beta)\left[\frac{\beta+1}{\beta-1}\right]^{\frac{\eta-\alpha}{1-\alpha}} + 2\alpha - \beta}{(2\alpha+1)\left\{1+\left[\frac{\beta+1}{\beta-1}\right]^{\frac{\eta-\alpha}{1-\alpha}}\right\}} \qquad \text{where } 1<\beta<\infty \text{ and } 0\le\alpha\le1$$

Eq. 3.2.6

Where η is the non-dimensional grid point distribution in the computational plane which varies between zero and 1, β is the clustering function, more clustering is achieved by letting β to approach 1 and α is a non-dimensional quantity that indicates which grid location the clustering should be attracted to, e.g. a value of 0.5 ensures clustering is uniformly done at both ends. **Figure 3.2.4 (a)**



(a) Mesh Topology for Single OGV



(b) Tip Clearance

Figure 3.2.4 Single Passage *PADRAM* Mesh for the Swept Back OGV - Courtesy of [Shahpar and Lapworth]

[45] Peter Eiseman and Robert E. Smith, "*Applications of Algebraic Grid Generation*", April 1990.

[46] Shahrokh Shahpar and Leigh Lapworth, "*PADRAM: Parametric Design And Rapid Meshing System For Turbomachinery Optimization*", Proceedings of ASME Turbo Expo 2003, USA.

shows the *PADRAM* mesh topology for a single OGV mesh. Note that the upstream and downstream H-mesh can also be rotated to align the mesh with the blade inlet and exit metal angles. Tip Gaps Tip clearance is often required for shroud less rotor blades such as fans, compressors and some turbines as well as hub clearance for cantilevered stator blades. Tip leakage flows are often poorly resolved. If the number of points in the tip clearance is too low, then the complex physical phenomena that occur there cannot be accurately modelled. It has been found that the flow solution can be sensitive how the gap is modelled and in some solvers the geometry is changed to alleviate the problem, e.g. sheared H-meshes often use the so called "pinched" tip gap. However, *PADRAM* models the gap in a consistent way that meshes the blade geometry. Once the O-grid is generated with the outer domain of the blade, the grid corresponding to the solid part of the domain is constructed using the same boundary node distribution. It is important to keep the mesh spacing in the inner and outer part of the wall as close as possible to each other. **Figure 3.2.4 (b)** shows a typical tip gap mesh for a compressor rotor generated in *PADRAM* (the tip gap has been increased to 5% span for demonstration). *PADRAM* can construct a viscous mesh for the complete bypass assembly consisting of 52 different staggered and three types over and under cambered OGVS, pylon, RDF and a splitter in a matter of minutes. The details of the mesh near the OGV, Pylon, RDF and the splitter are shown in **Figure 3.2.5**. For complete details, please see the work by [Shahpar and Lapworth][47].

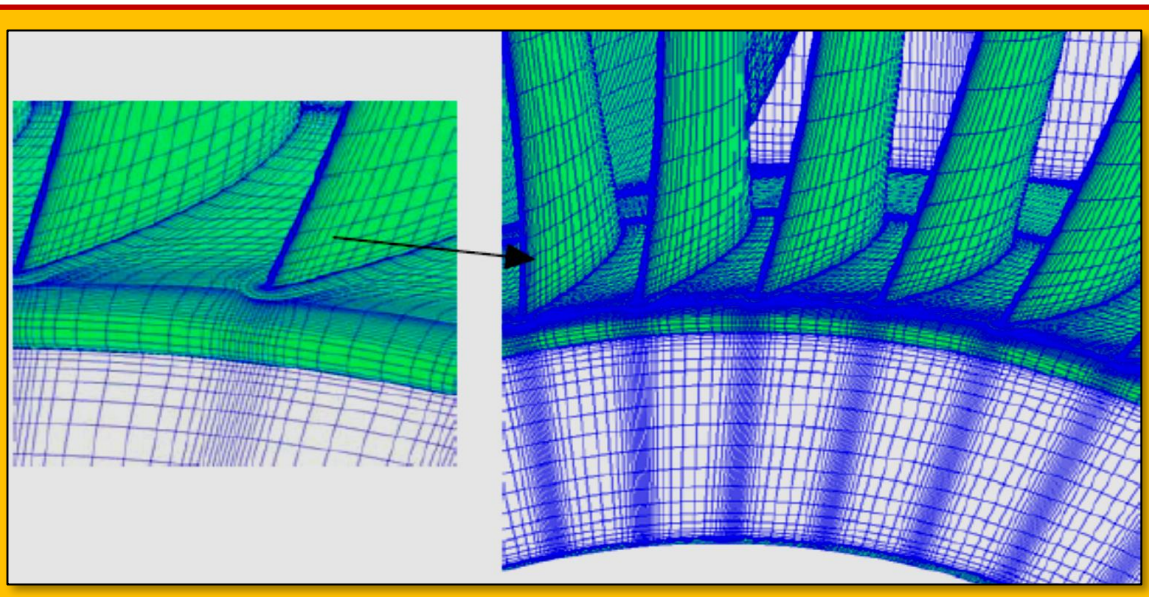


Figure 3.2.5 Detail of the Splitter C-Mesh, Hub Mesh and the Engine-Core Exit Mesh - Courtesy of [Shahpar and Lapworth]

To increasing grid data in CFD simulation, a novel method proposed by [Sun et al.][48] for compressing and saving the structured grids. In the present method, the geometric coordinates of the six logical domains of one grid block is saved instead of all grid vertex coordinates to reduce the size of the structured grid file when the grid is compressed. And all grid vertex coordinates are recovered from the compressed data with the use of the transfinite interpolation algorithm when the grid is decompressed.

---

[47] Shahrokh Shahpar and Leigh Lapworth, "*PADRAM: Parametric Design And Rapid Meshing System For Turbomachinery Optimization*", Proceedings of ASME Turbo Expo 2003, USA.

[48] Yan SUN, Ying ZHAO, Dehong MENG, Meng JIANG, A compressing and saving method for structured grid data based on block construction, Chinese Journal of Aeronautics, Volume 35, Issue 6, 2022, Pages 146-155, ISSN 1000-9361, https://doi.org/10.1016/j.cja.2021.09.022.

3.3 PDE Smoother

Like algebraic methods, differential equation methods are also used to generate grids. Grid construction can be done using all three classes of partial differential equations. The generation of field values of a function from boundary values can be done in various ways, e.g., by interpolation between the boundaries, etc., as is discussed previously. The solution of such a boundary-value problem, however, is a classic problem of partial differential equations, so that it is logical to take the coordinates to be solutions of a system of partial differential equations. If the coordinate points (and/or slopes) are specified on the entire closed boundary of the physical region, the equations must be ***elliptic,*** while if the specification is on only a portion of the boundary the equations would be ***parabolic*** or ***hyperbolic.*** This latter case would occur, for instance, when an inner boundary of a physical region is specified, but a surrounding outer boundary is arbitrary. The present chapter, however, treats the general case of a completely specified boundary, which requires an elliptic partial differential system.

3.4 Elliptic Schemes

At this stage grid is smooth enough to satisfy majority of applications, but if needed, further smoothing is obtained with solution of elliptic partial differential equations (PDE). For 2D formulation, forcing function terms are used to construct stretched layers of the cells close to the domain boundaries.

$$\xi_{xx} + \xi_{yy} = P(\xi,\eta) \quad , \quad \eta_{xx} + \eta_{yy} = Q(\xi,\eta)$$

Eq. 3.4.1

Where (ξ, η) are the coordinates in the computational domain. Control functions are computed using the boundary point spacing, r, and then interpolated to the inner points[49]. The forcing terms (P, Q) are computed as:

$$P = -\frac{\frac{\partial r}{\partial \xi}\frac{\partial^2 r}{\partial \xi^2}}{\left|\frac{\partial r}{\partial \xi}\right|^2} \quad , \quad Q = \frac{\frac{\partial r}{\partial \eta}\frac{\partial^2 r}{\partial \eta^2}}{\left|\frac{\partial r}{\partial \eta}\right|^2}$$

Eq. 3.4.2

Once P and Q are obtained at each boundary the values for the inner points are obtained using a linear interpolating along lines of constant ξ and η:

$$P(\xi,\eta) = (1-\eta)\,P_1(\xi) + \eta P_2(\xi) \qquad 0 \le \xi \le 1$$
$$Q(\xi,\eta) = (1-\xi)\,Q_1(\eta) + \xi Q_2(\eta) \qquad 0 \le \eta \le 1$$

Eq. 3.4.3

Grid control of orthogonality at boundaries is introduced adding a second term in P and Q as:

$$P = -\frac{\frac{\partial r}{\partial \xi}\frac{\partial^2 r}{\partial \xi^2}}{\left|\frac{\partial r}{\partial \xi}\right|^2} - \lambda\frac{\frac{\partial r}{\partial \xi}\frac{\partial^2 r}{\partial \eta^2}}{\left|\frac{\partial r}{\partial \eta}\right|^2} \quad , \quad Q = \frac{\frac{\partial r}{\partial \eta}\frac{\partial^2 r}{\partial \eta^2}}{\left|\frac{\partial r}{\partial \eta}\right|^2} - \lambda\frac{\frac{\partial r}{\partial \eta}\frac{\partial^2 r}{\partial \xi^2}}{\left|\frac{\partial r}{\partial \xi}\right|^2}$$

Eq. 3.4.4

---

[49] Thomas P., and Middlecoff J., *"Direct control of the Grid Point Distribution in Meshes Generated by Elliptic Equations"*, AIAA Journal Vol. 18, No. 6., 1980.

Where $0 < \lambda < 1$ is a factor that relaxes the orthogonality at the boundaries. It has been observed that the range $\lambda \in [0.4\text{-}0.7]$ produces optimal results for our configurations. The elliptic PDEs are solved using a multi-grid method and the smoother is based on a point-wise Newton solver. When the forcing terms are used the convergence of the algorithm deteriorates slightly[50]. An important property in regard to coordinate system generation is the inherent smoothness that prevails in the solutions of elliptic systems. Furthermore, boundary slope discontinuities are not propagated into the field. Finally, the smoothing tendencies of elliptic operators, and the extremum principles, allow grids to be generated for any configurations without overlap of grid lines (**Figure 3.4.1**). There are thus a number of advantages to using a system of elliptic partial differential equations as a means of coordinate system generation. A disadvantage, of course, is that a system of partial differential equations must be solved to generate the coordinate system.

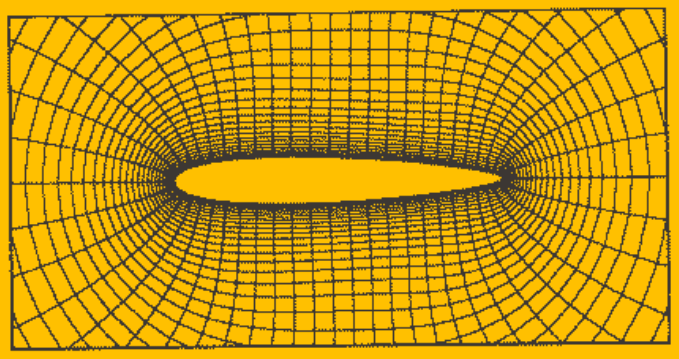


Figure 3.4.1 Typical Elliptic Grid for an Airfoil with Orthogonality Enforced on the Boundary

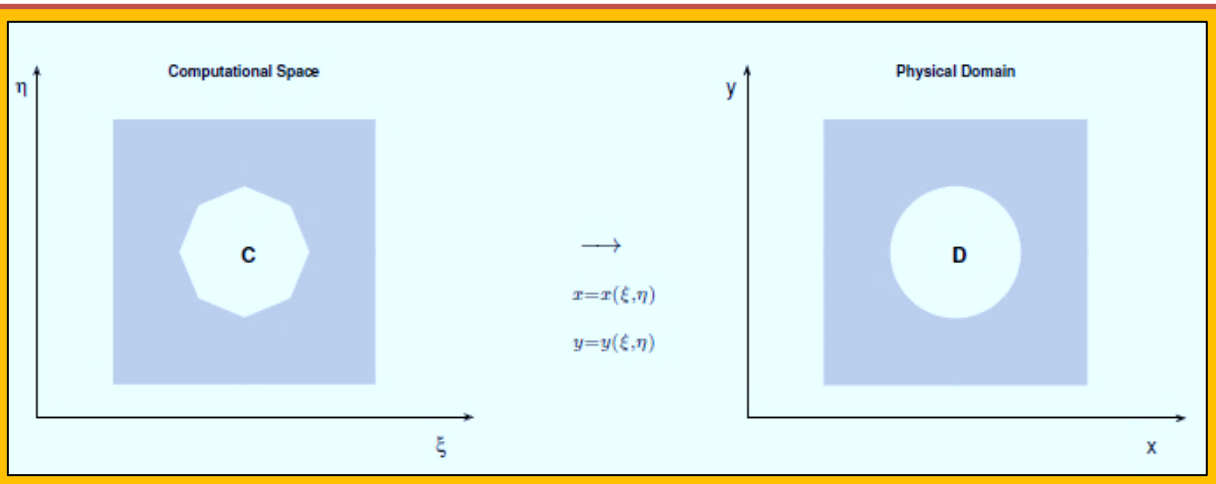


Figure 3.4.2 Winslow Equations in 2D: obtained by enforcing the computational coordinates to be harmonic, that is, the equations $\Delta\xi(x\,;y) = 0$ and $\Delta\eta\,(x\,;y) = 0$ are rewritten and solved in the computational space

3.4.1 Winslow Function

The Winslow equations are obtained by enforcing the computational coordinates to be harmonic, as illustrated in **Figure 3.4.2.** More specifically, let $D \subset R^n$ be the simply connected bounded physical domain in n space dimensions, $C \subset R^n$ the computational domain, and define the mapping $\mathbf{x} : C \rightarrow D$, where $\mathbf{x} = \mathbf{x}(\boldsymbol{\xi}) = (\mathbf{x}_1(\boldsymbol{\xi}) \,,,,,, \mathbf{x}_n(\boldsymbol{\xi}))$. Conversely, the mapping from domain D to C will be denoted by $\boldsymbol{\xi} = \boldsymbol{\xi}(\mathbf{x}) = (\boldsymbol{\xi}_1(\mathbf{x}) \,,,,,, \boldsymbol{\xi}_n(\mathbf{x}))$. In the derivations below, we use Einstein's summation notation, and follow the development in [1] closely, we define the covariant base vectors as

[50] Sorenson R. L. and Steger J. L. *Numerical Generation of Two dimensional Grids by the Use of Poisson Equations with Grid Control*, in Numerical Grid Generation Techniques, R. E Smith, ed.. NASA CP 2166, NASA Langley Research Center, Hampton, VA, USA, 1980.

$$\mathbf{g}_i = \partial_i \mathbf{x} \quad \text{for} \quad i = 1,,,,n$$

Eq. 3.4.5

Naturally, the covariant metric tensor components $\mathbf{g}_{ij}$ are defined as the inner product of the covariant base vectors, i.e,

$$\mathbf{g}_{ij} = (\mathbf{g}_i, \mathbf{g}_j) \text{ for } \quad i,j = 1,,,,n$$

Eq. 3.4.6

Furthermore, the contravariant base vectors $\mathbf{g}^i = \partial_I \xi$ satisfy

$$(\mathbf{g}^i, \mathbf{g}_j) = \delta^i_j \quad \text{for} \quad i,j = 1,,,,n$$

Eq. 3.4.7

where $\delta^i j$ is the Kronecker delta. Next, define the contravariant metric tensor components

$$\mathbf{g^{ij}} = (\mathbf{g^i}, \mathbf{g^j}) \quad \text{for} \quad i,j = 1,,,,n$$

Eq. 3.4.8

so that $\mathbf{g}^{ij}\mathbf{g}_{jk} = \delta^i{}_k$, and let g be the determinant of the covariant metric tensor $\mathbf{g} = \det(\mathbf{g}_{ij})$. Consider the arbitrary function $\phi = \phi(\boldsymbol{\xi})$ defined in the computational domain C, then ϕ is also defined in D through the mapping $\boldsymbol{\xi}(\mathbf{x})$. It is a well-known result from differential geometry that its Laplace operator can be written as (see, for example, [2])

$$\Delta_x \phi = \frac{1}{\sqrt{g}} \partial_i \left(\sqrt{g} g^{\mathbf{ij}} \right) \partial_j \phi$$

Eq. 3.4.9

which leads to the following simple form of the Winslow equations in physical coordinates:

$$g^{ij}\, \partial_i\, \partial_j x_k \;= 0 \quad \text{for } k = 1,,,,,n$$

Eq. 3.4.10

where, again, g^{ij} are defined through the relation $g^{ij}g_{jk} = \delta_{ik}$ and $gij = \partial_i x_k \partial_j x_k$. Note that **Eq. 3.4.10** form a nonlinear system, since the contravariant metric tensor depends on the unknown solution x. One of the main advantages of this particular formulation is that it allows for a highly efficient solution strategy using Picard iterations, where the components of g^{ij} are computed from an old solution and a linear problem is solved for a new improved solution [1].

3.4.1.1 References

[1] Fortunato, M., & Persson, P. (2016). High-order unstructured curved mesh generation using the Winslow equations. *J. Comput. Phys., 307*, 1-14.

[2] Kreyszig, E.: Di_erential geometry. Dover Publications, Inc., New York (1991). Reprint of the 1963 edition.

3.4.2 Case Study 1 – Orthogonal Elliptic Mesh Smoother

This is a 2D orthogonal elliptic mesh generator which works by solving the ***Winslow PDE*** [51-52]. It is capable of modifying the meshes with stretching functions and an orthogonality adjustment algorithm. This algorithm works by calculating curve slopes using a tilted parabola tangent line fitter (original discovery). ***A distinct feature of the elliptic mesh solver is that it corrects overlapping and misplaced grid line very well.*** Firstly to construct an initial mesh, the ***Transfinite Interpolation***

---

[51] Chaitanya Varier 2017.

[52] M. Farrashkhalvat and J.P. Miles, "*Basic Structured Grid Generation*", An imprint of Elsevier Science Linacre House, Jordan Hill, Oxford OX2 8DP, 200 Wheeler Rd, Burlington MA 01803, First published 2003.

algorithm is applied to the given domain constrained by the specified boundary conditions. This algorithm is implemented by mapping each point within the domain (regardless of the boundaries) to a new domain existing within the boundaries. This algorithm works by iteratively solving the parametric vector equation. At the heart of the solver is the mesh smoothing algorithm, which at a high level, works by solving the pair of ***Laplace equations***. Coordinates of every point in the target domain, mapped to a transformed, computational space using the change of variables method. This renders the calculations simpler and faster to compute. However, we wish to solve the inverse problem, where we transition from the computational space to the curvilinear solution space. Using tensor mathematics, it can be shown that this problem entails solving the equations.

3.4.2.1 Orthogonality Adjustment Algorithm

In several computational fluid dynamics applications, an orthogonal mesh is necessary in certain regions to ensure a high enough accuracy when performing calculations. However, it is not always possible to achieve a fully orthogonal solution, and thus the problem becomes finding a nearly-orthogonal solution to an arbitrarily defined domain. The implemented solution uses an iterative approach to find the angles of intersection and adjust the position of the nodes until their respective angles of intersection converge to a reasonable threshold value from 90 degrees. The exact method makes use of the linear approximation of the grid lines intersecting at each node within the mesh. (see **Figure 3.4.3**).

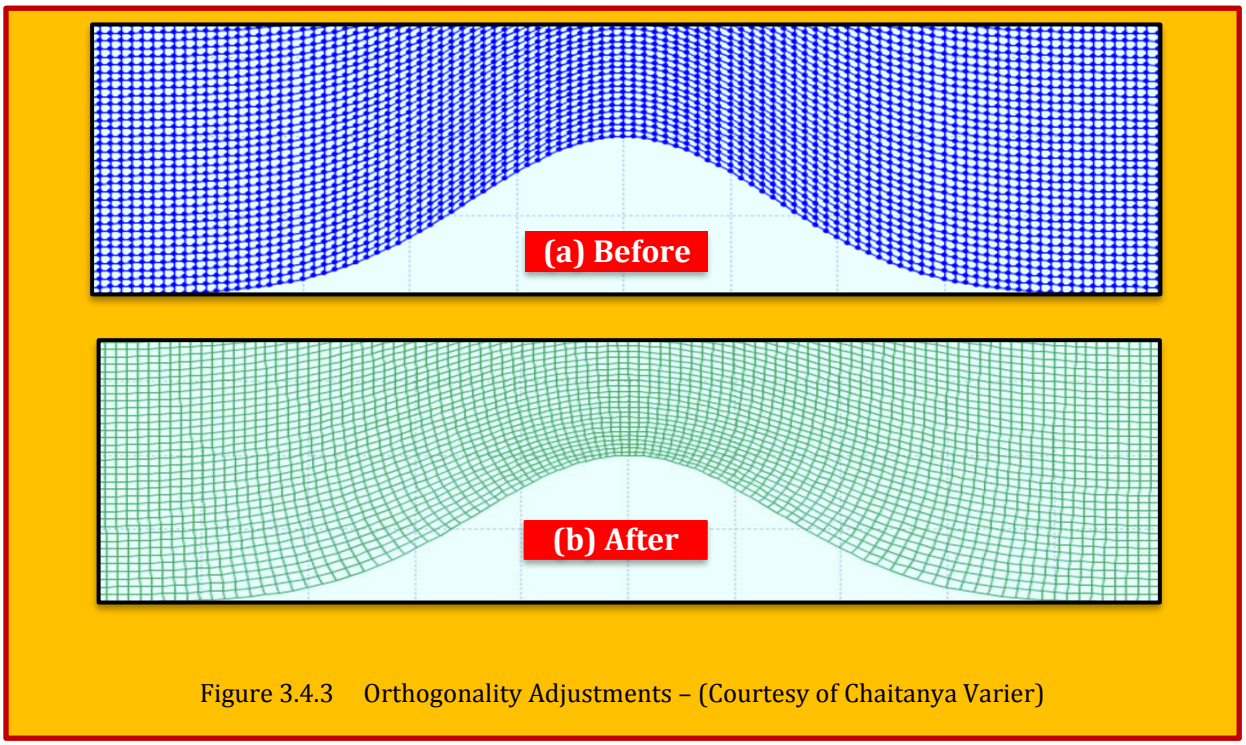


Figure 3.4.3 Orthogonality Adjustments – (Courtesy of Chaitanya Varier)

3.4.3 Case Study 2 - Boundary Orthogonality in Elliptic Mesh Generation[53]

Experience in the field of computational simulation has shown that grid quality in terms of smoothness and orthogonality affects the accuracy of numerical solutions. It has been pointed out by [Thompson et al.][54] that skewness increases the truncation error in numerical differentiation.

[53] Ahmed Khamayseh, Andrew Kuprat, C. Wayne Mastin, "*Boundary Orthogonality in Elliptic Grid Generation*", CRC Press LLC, 1999.

[54] Thompson, J.F., *A general three-dimensional elliptic grid generation system on a composite block structure*, Comp. Meth. Appl. Mech. and Eng. 1987.

Especially critical in many applications is orthogonality or near-orthogonality of a computational grid near the boundaries of the grid. If the boundary does not correspond to a physical boundary in the simulation, orthogonality can still be important to ensure a smooth transition of grid lines between the grid and the adjacent grid presumed to be across the nonphysical boundary. If the grid boundary corresponds to a physical boundary, then orthogonality may be necessary near the boundary to reduce truncation errors occurring in the simulation of boundary layer phenomena, such as will be present in a Navier–Stokes simulation. In this case, fine spacing near the boundary may also be necessary to accurately resolve the boundary phenomena.

In elliptic grid generation, an initial grid (assumed to be algebraically computed using transfinite interpolation of specified boundary data) is relaxed iteratively to satisfy a quasi-linear elliptic system of partial differential equations (PDEs). The most popular method, the [Thompson, Thames, Mastin (TTM)] method, incorporates user-specifiable ***control functions*** in the system of PDEs. If the control functions are not used (i.e., set to zero), then the grid produced will be smoother than the initial grid, and grid folding (possibly present in the initial grid) may be alleviated. However, nonuse of control functions in general leads to nonorthogonality and loss of grid point spacing near the boundaries. Imposition of boundary orthogonality can be effected in two different ways. In ***Neumann orthogonality*** no control functions are used, but boundary grid points are allowed to slide along the boundaries until boundary orthogonality is achieved and the elliptic system has iterated to convergence.

This method, which is taken up here, is appropriate for nonphysical (internal) grid boundaries, since grid spacing present in the initial boundary distribution is usually not maintained. Previous methods for implementing Neumann orthogonality have relied on a ***Newton iteration*** method to locate the orthogonal projection of an adjacent interior grid point onto the boundary. The Neumann orthogonality method presented here uses a Taylor series to move boundary points to achieve approximate orthogonality. Thus, there is no need for inner iterations to compute boundary grid point positions.

In ***Dirichlet orthogonality*** , also taken up in this chapter, control functions (called ***orthogonal control Functions***) are used to enforce orthogonality near the boundary while the initial boundary grid point distribution is not disturbed. Early papers using this approach were written by [Sorenson][55] and [Thomas & Middlecoff][56]. In Sorenson's approach, the control functions are assumed to be of a particular exponential form. Orthogonality and a specified spacing of the first grid line off the boundary are achieved by updating the control functions during iterations of the elliptic system. [Thompson][57] presents a similar technique for updating the orthogonal control functions. This technique evaluates the control functions on the boundary and interpolates for interior values. A user-specified grid spacing normal to the boundary is required.

The technique of [Spekreijse][58] automatically constructs control functions solely from the specified boundary data without explicit user-specification of grid spacing normal to the boundary. Through construction of an intermediate parametric domain by arclength interpolation of the specified boundary point distribution, the technique ensures accurate transmission of the boundary point distribution throughout the final orthogonal grid. Applications to planar and surface grids are given in[59].

---

[55] Sorenson, R.L., *A computer program to generate two-dimensional grids about airfoils and other shapes by the use of Poisson's equations*, NASA TM 81198. NASA Ames Research Center, 1980.

[56] Thomas, P.D. and Middlecoff, J.F., *Direct control of the grid point distribution in meshes generated by elliptic equations,* AIAA J. 1980, 18, pp 652–656.

[57] Thompson, J.F., *A general three-dimensional elliptic grid generation system on a composite block structure*, *Comp. Meth. Appl. Mech. and Eng*. 1987.

[58] Spekreijse, S.P., *Elliptic grid generation based on Laplace equations and algebraic transformations*, J. Com. Phys. 1995,

[59] See Previous.

Here, we present a technique similar to for updating of orthogonal control functions during elliptic iteration[60]. However, our technique does not require explicit specification of grid spacing normal to the boundary but, employs an interpolation of boundary values to supply the necessary information. However, unlike[61], this interpolation is not constructed in an auxiliary parametric domain, but is simply the initial algebraic grid constructed using transfinite interpolation.
Although this grid is very likely skewed at the boundary, the first interior coordinate surface is assumed to be correctly positioned in relation to the boundary, which is enough to give us the required normal spacing information for iterative calculation of the control functions. Ghost points, exterior to the boundary, are constructed from the interior coordinate surface, leading to potentially smoother grids, since central differencing can now be employed at the boundary in the direction normal to the boundary.
Since our technique does not employ the auxiliary parametric domain of[62], theory and implementation are simpler. The implementation of this technique for the case of volume grids is straightforward, and indeed we present an example. We mention here that [Soni][63] presents another method of constructing an orthogonal grid by deriving spacing information from the initial algebraic grid. However, unlike our method which uses ghost points at the boundary, this method does not emphasize capture of grid spacing information at the boundary.
Instead, the algebraic grid influences the grid spacing of the elliptic grid in a uniform way throughout the domain. With no special treatment of spacing at the boundary, considerable changes in normal grid spacing can occur during the course of elliptic iteration. This may be unacceptable in applications where the most numerically challenging physics occurs at the boundaries.

3.4.4 Boundary Orthogonality for Planar Grids

We assume an initial mapping $\mathbf{x}(\xi, \eta) = (x(\xi, \eta), y(\xi, \eta))$ from computational space $[0, m] \times [0, n]$ to the bounded physical domain $\Omega \subset R^2$. Here m , n are positive integers and ***grid lines*** are the lines $\xi = i$, $\eta = i$ with $0 \le i \le m$, $0 \le j \le n$ being integers. The initial mapping $\mathbf{x}(\xi, \eta)$ is usually obtained using algebraic grid generation methods such as linear transfinite interpolation. Given the initial mapping, a general method for constructing curvilinear structured grids is based on partial differential equations [Thompson et al.][64]. The coordinate functions $x(\xi, \eta)$ and $y(\xi, \eta)$ are iteratively relaxed until they become solutions of the following quasi-linear elliptic system:

$$g_{12}\left(\mathbf{X}_{\xi\xi} + P\,\mathbf{X}_{\xi}\right) - 2g_{12}\mathbf{X}_{\xi\eta} + g_{11}\left(\mathbf{X}_{\eta\eta} + Q\,\mathbf{X}_{\eta}\right) = 0$$

Eq. 3.4.11

$$g_{11} = \mathbf{X}_{\xi} \cdot \mathbf{X}_{\xi} = x_{\xi}^2 + y_{\eta}^2$$
$$g_{12} = \mathbf{X}_{\xi} \cdot \mathbf{X}_{\eta} = x_{\xi}x_{\eta} + y_{\xi}y_{\eta}$$
$$g_{22} = \mathbf{X}_{\eta} \cdot \mathbf{X}_{\eta} = x_{\eta}^2 + y_{\eta}^2$$

The control functions P and Q control the distribution of grid points. Using P = Q = 0 tends to generate a grid with a uniform spacing. Often, there is a need to concentrate points in a certain area of the grid such as along particular boundary segments in this case, it is necessary to derive appropriate values

---

[60] Thompson, J.F., *A general three-dimensional elliptic grid generation system on a composite block structure,* Comp. Meth. Appl. Mech. and Eng. 1987.
[61] Spekreijse, S.P., *Elliptic grid generation based on Laplace equations and algebraic transformations*, J. Com. Phys. 1995.
[62] See Previous.
[63] Soni, B.K., *Elliptic grid generation system: control functions revisited-I*, Appl. Math. Com. 1993.
[64] Thompson, J.F., Warsi, Z.U.A., and Mastin, C.W., *Numerical Grid Generation: Foundations and Applications*. North-Holland, New York, 1985.

for the control functions. To complete the mathematical specification of system **Eq. 3.4.11**, boundary conditions at the four boundaries must be given. (These are the ξ = 0, ξ = *m* , η = 0, and η = *n* or "left," "right," "bottom," and "top" boundaries.) We assume the orthogonality condition

$$\mathbf{X}_{\xi} \cdot \mathbf{X}_{\eta} = 0 \ , \ \xi = 0, m \ \ \& \ \eta = 0 , n$$

Eq. 3.4.12

We assume that the initial algebraic grid neither satisfies **Eq. 3.4.11** nor **Eq. 3.4.12**. Nevertheless, the initial grid may possess grid point density information that should be present in the final grid. If the algebraic grid possesses important grid density information, such as concentration of grid points in the vicinity of certain boundaries, then it is necessary to invoke "***Dirichlet orthogonality***" wherein we use the freedom of specifying the control functions P, Q in such a fashion as to allow satisfaction of **Eq. 3.4.11**, **Eq. 3.4.12** without changing the initial boundary point distribution at all, and without greatly changing the interior grid point distribution. If, however, the algebraic grid does not possess relevant grid density information (such as may be the case when the grid is an "interior block" that does not border any physical boundary), we attempt to solve **Eq. 3.4.11**, **Eq. 3.4.12** using the simplest assumption P = Q = 0. Since we are not using the degrees of freedom afforded by specifying the control functions, we are forced to allow the boundary points to "slide to allow satisfaction of **Eq. 3.4.11**, **Eq. 3.4.12**. This is "***Neumann orthogonality***." The composite case of having some boundaries treated using ***Dirichlet orthogonality,*** some treated using Neumann orthogonality, and some boundaries left untreated will be clear after our treatment of the pure Neumann and Dirichlet cases.

3.4.5 Neumann Orthogonality

As is typical, let us assume that the boundary segments are given to be parametric curves (e.g., B-splines). If we set the control functions P,Q to zero, then it will be necessary to slide the boundary nodes along the parametric curves in order to satisfy **Eq. 3.4.11**, **Eq. 3.4.12**. A standard discretization of our system is central differencing in the ξ and η directions. The system is then applied to the interior nodes to solve for $\mathbf{x}_{i,j} = (x_{i,j} , y_{i,j})$ using an iterative method. With regard to the implementation of boundary conditions, suppose along the boundary segments ξ = 0 and ξ = m the variables x and y can be expressed in terms of a parameter u as x = x (u) and y = y (u). For the ξ = 0 and ξ = m boundaries, let $(\mathbf{x}_{\eta})_{i,j}$ denote the central difference $(1/2)(\mathbf{x}_{i,j+1} - \mathbf{x}_{i,j-1})$ along the boundaries (i = 0 or i = m). Using one-sided differencing for $\mathbf{x}_{\xi}$, **Eq. 3.4.12** is discretized as

$$\left(\mathbf{X}_{\mathbf{i+1,j}} - \mathbf{X}_{\mathbf{i,j}}\right) . \left(\mathbf{X}_{\eta}\right)_{0,j} = 0 \quad , \ \text{along} \quad \xi = i = 0$$

$$\left(\mathbf{X}_{\mathbf{i,j-1}} - \mathbf{X}_{\mathbf{i,j}}\right) . \left(\mathbf{X}_{\eta}\right)_{m,j} = 0 \quad , \ \text{along} \quad \xi = i = m$$

Eq. 3.4.13

Solution of **Eq. 3.4.13** for $\mathbf{x}_{i,j}= (x_{i,j} , y_{i,j})$ in effect causes the sliding of $\mathbf{x}_{i,j}$ along the boundary so that the grid segment between $\mathbf{x}_{i,j}$ and its neighbor on the first interior coordinate curve ($\xi = 1$ or $\xi = m - 1$) is orthogonal to the boundary curve. (See **Figure 3.4.4**). To solve for $\mathbf{x}_{i,j}$ the old parameter value u_0 is used to solve for the new u to compute the new $\mathbf{x}_{i,j}$. Using the Taylor expansion of $\mathbf{x}(u)$ about u_0 to give

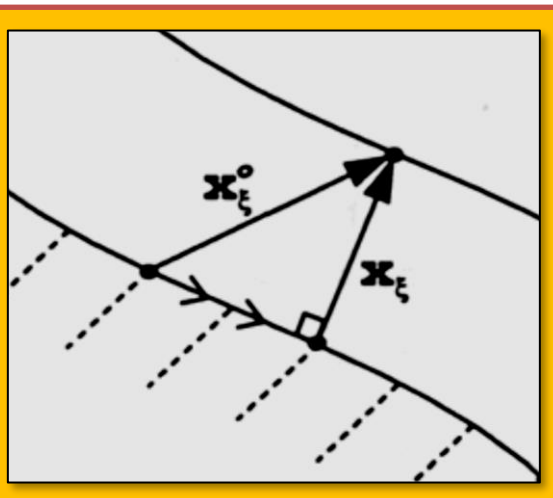


Figure 3.4.4 Change in $\mathbf{x}_{\xi}$ when Boundary Point is Re-Positioned in Neumann Orthogonality

$$x_{ij} = x(u) \approx x(u_0) + x_u(u_0)u$$

Eq. 3.4.14

substituting **Eq. 3.4.14** in **Eq. 3.4.13** implies that

$$u = u_0 + \frac{(\mathbf{x}_\eta)_{0,j}.(\mathbf{x}_{i,j} - \mathbf{x}(u_0))}{(\mathbf{x}_\eta)_{0,j}.\mathbf{x}_u(u_0)}$$

Eq. 3.4.15

to give $\mathbf{x}_{i,j} = \mathbf{x}(u)$ along the boundary $\xi = 0$. Whereas, substituting **Eq. 3.4.14** in **Eq. 3.4.13** implies that

$$u = u_0 + \frac{(\mathbf{x}_\eta)_{m,j}.(\mathbf{x}_{m-1,j} - \mathbf{x}(u_0))}{(\mathbf{x}_\eta)_{m,j}.\mathbf{x}_u(u_0)}$$

Eq. 3.4.16

to give $\mathbf{x}i_{,j}= \mathbf{x}(u)$ along the boundary $\xi = m$. Consider next the case where the boundaries are $\eta = 0$ and $\eta = n$. Orthogonality **Eq. 3.4.12** with central differencing in the ξ direction and one-sided differencing in the η direction implies

$$u = u_0 + \frac{(\mathbf{x}_\xi)_{i,0}.(\mathbf{x}_{i,1} - \mathbf{x}(u_0))}{(\mathbf{x}_\xi)_{i,0}.\mathbf{x}_u(u_0)}$$

Eq. 3.4.17

which gives along the boundary $\eta = 0$, and

$$u = u_0 + \frac{(\mathbf{x}_\xi)_{0,n}.(\mathbf{x}_{i,n-1} - \mathbf{x}(u_0))}{(\mathbf{x}_\xi)_{0,n}.\mathbf{x}_u(u_0)}$$

Eq. 3.4.18

to give $\mathbf{x}_{i,j} = \mathbf{x}(u)$ along the boundary $\eta = n$. These boundary condition equations are to be evaluated for each cycle in the course of the iterative procedure. Note that a periodic boundary condition is used in the case of doubly connected regions. Also note that during the relaxation process, "guards" must be used to prevent a given boundary point from overtaking its neighbors when sliding along the boundaries. Indeed, near obtuse corners, there is a tendency for grid points to try to slide along the boundary curves past the corners in order to satisfy the orthogonality condition. An appropriate guard would be to limit movement of each grid point so that its distance from its two boundary-curve neighbors is reduced by at most 50% on a given iteration, down to a user-specified minimum length

δ in physical space.

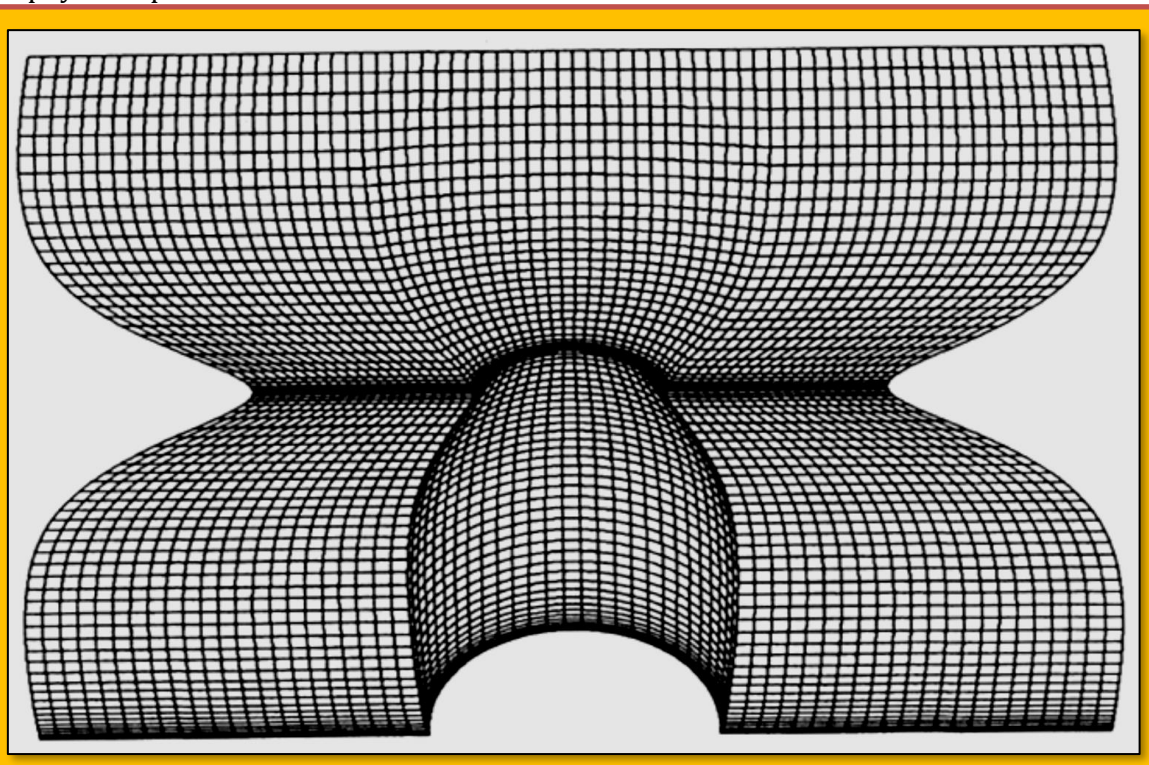


Figure 3.4.5 An Algebraic Planar Mesh on a Bi-Cubic Geometry

As an application of ***Neumann orthogonality***, consider **Figure 3.4.5,** which is an initial algebraic planar grid on a bicubic geometry. The mesh is highly nonorthogonal at certain points along the boundaries, and it possesses an undesirable concentration of points in the interior of the grid. In fact, there is folding of the algebraic grid in this central region.

Figure 3.4.6 shows an elliptically smoothed grid using ***Neumann orthogonality***. The grid is clearly seen to be smooth, boundary-orthogonal, and no longer folds in the interior. For certain applications, this grid may be entirely acceptable. However, if the bottom boundary of the grid corresponded to a physical boundary, then the results of **Figure 3.4.6** might be deemed unacceptable. This is because, although orthogonality has been established, grid point distribution (both along the boundary and normal to the boundary) has been significantly altered. In this case, the Dirichlet orthogonality technique will have to be employed.

3.4.6 Dirichlet Orthogonality

The above discussion shows how orthogonality can be imposed without use of control functions, by sliding grid points along the boundary. Orthogonality can also be imposed by adjusting the control functions near the boundary and keeping the boundary points fixed. This approach was originally developed by [Sorenson][65] for imposing boundary orthogonality in two dimensions. [Sorenson][66] and [Thompson][67] have extended this approach to three dimensions. However, as mentioned in the introduction, our approach does not require user specification of grid spacing normal to the

---

[65] Sorenson, R.L., *A computer program to generate two-dimensional grids about airfoils and other shapes by the use of Poisson's equations*, NASA TM 81198. NASA Ames Research Center, 1980.

[66] Sorenson, R.L., *Three-dimensional elliptic grid generation about fighter aircraft for zonal finite difference computations,* AIAA-86-0429. AIAA 24th Aerospace Science Conference, Reno, NV, 1986.

[67] Thompson, J.F., *A general three-dimensional elliptic grid generation system on a composite block structure*, *Comp. Meth. Appl. Mech. and Eng*. 1987.

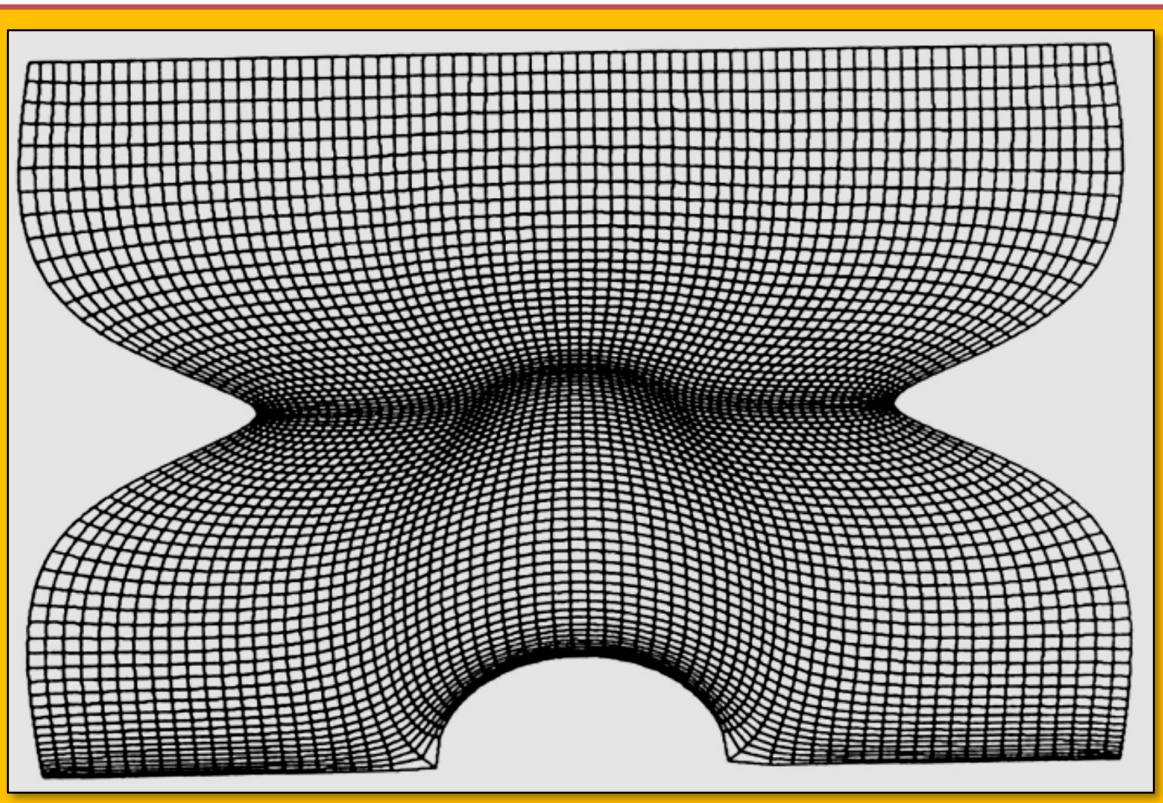


Figure 3.4.6 An Elliptic Planar Mesh on a Bi-Cubic Geometry with Neumann Orthogonality

boundary. Instead, our technique automatically derives normal grid spacing data from the initial algebraic grid. Assuming boundary orthogonality **Eq. 3.4.12**, substitution of the inner product of $\mathbf{x}_\xi$ and $\mathbf{x}_\eta$ into **Eq. 3.4.11** yields the following two equations for the control functions on the boundaries:

$$P = -\frac{\mathbf{X}_\xi - \mathbf{X}_{\xi\xi}}{g_{11}} - \frac{\mathbf{X}_\xi - \mathbf{X}_{\eta\eta}}{g_{22}} \quad , \quad Q = -\frac{\mathbf{X}_\eta - \mathbf{X}_{\eta\eta}}{g_{11}} - \frac{\mathbf{X}_\eta - \mathbf{X}_{\xi\xi}}{g_{22}} \qquad \textbf{Eq. 3.4.19}$$

These control functions are called the ***orthogonal control functions*** because they were derived using orthogonality considerations. They are evaluated at the boundaries and interpolated to the interior using linear transfinite interpolation. These functions need to be updated at every iteration during solution of the elliptic system. We now go into detail on how we evaluate the quantities necessary in order to compute P and Q on the boundary using **Eq. 3.4.19**. Suppose we are at the "left" boundary $\xi = 0$, but not at the corners ($\eta \neq 0$ and $\eta \neq n$). The derivatives $\mathbf{x}_\eta$, $\mathbf{x}_{\eta\eta}$ and the spacing $g_{22} = ||\mathbf{x}_\eta||^2$ are determined using centered difference formulas from the boundary point distribution and do not change. However, the g_{11}, $\mathbf{x}_\xi$, and $\mathbf{x}_{\xi\xi}$ terms are not

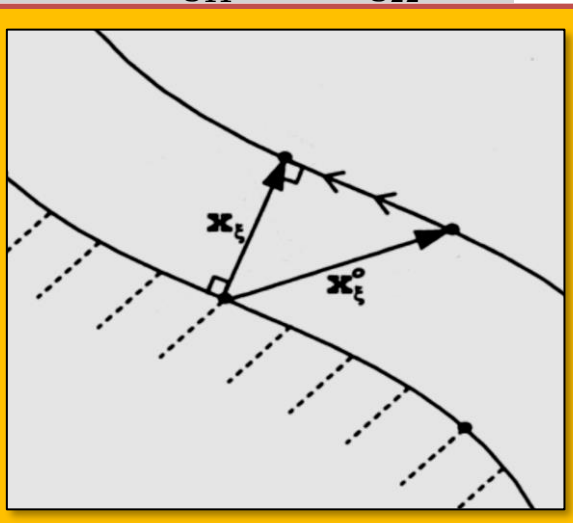


Figure 3.4.7 Projection of Interior Algebraic Mesh Point to Orthogonal Position

determined by the boundary distribution. Additional information amounting to the desired grid spacing normal to the boundary must be supplied.

A convenient way to infer the normal boundary spacing from the initial algebraic grid is to assume that the position of the first interior grid line off the boundary is correct. Indeed, near the boundary, it is usually the case that all that is desired of the elliptic iteration is for it to swing the intersecting grid lines so that they intersectthe boundaries orthogonally, *without* ***changing the positions*** of the grid lines parallel to the boundary. This is shown graphically in **Figure 3.4.7**, where we see a grid point, from the first interior grid line, swung along the grid line to the position where orthogonality is established. The effect of forcing all the grid points to swing over in this fashion would thus be to establish boundary orthogonality, but still leave the algebraic interior grid line unchanged. The similarity of **Figure 3.4.4** and **Figure 3.4.7** seems to indicate that this process is analogous to, and hence just as "natural" as, the process of sliding the boundary points in the Neumann orthogonality approach with zero control functions.

Unfortunately, this preceding approach entails the *direct specification* of the positions of the first interior layer of grid points off the boundary. This is not permissible for a couple of reasons. First, since they are adjacent to *two* different boundaries, the points $\mathbf{x}_{1,1}$, $\mathbf{x}_{m-1,1}$, $\mathbf{x}_{1,n-1}$, and $\mathbf{x}_{m-1,n-1}$ have contradictory definitions for their placement. Second, and more importantly, the direct specification of the first layer of interior boundary points together with the elliptic solution for the positions of the deeper interior grid points can lead to an undesirable "kinky" transition between the directly placed points and the elliptically solved for points. (This "kinkiness" is due to the fact that a perfectly smooth boundary-orthogonal grid will probably exhibit some small degree of nonorthogonality as soon as one leaves the boundary even as close to the boundary as the first interior line. Hence, forcing the grid points on the first interior line to be *exactly* orthogonal to the boundary cannot lead to the smoothest possible boundary-orthogonal grid.)

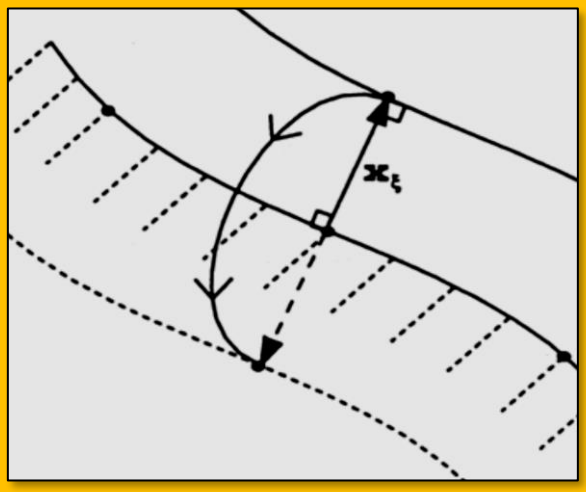


Figure 3.4.8 Reflection of Orthogonalized Interior Mesh Point to Form External Ghost Point

Nevertheless, our "natural" approach for deriving grid spacing information from the algebraic grid can be modified in a simple way, as depicted in **Figure 3.4.8**. Here, the orthogonally-placed interior point is reflected an equal distance across the boundary curve to form a "ghost point." Repeatedly done, this procedure in effect forms an "exterior curve" of ghost points that is the reflection of the first (algebraic) grid line across the boundary curve. The ghost points are computed at the beginning of the iteration and do not change. They are employed in the calculation of the normal second derivative $\mathbf{x}_{\xi\xi}$ at the boundary and the normal spacing off the boundary; the fixedness of the ghost points assures that the normal spacing is not lost during the course of iteration, as it sometimes is in the Neumann orthogonality approach. Conversely, all of the interior grid points are free to change throughout the course of the iteration, and so smoothness of the grid is not compromised. More precisely, again at the "left" $\xi = 0$ boundary, let $(\mathbf{x}\eta)_{0,j}$ denote the centrally differenced derivative $(\tfrac{1}{2})(\mathbf{x}_{0,j+1} - \mathbf{x}_{0,j-1})$. Let $(\mathbf{x}_{o\xi})_{0,j}$ denote the one-sided derivative $\mathbf{x}_{1,j} - \mathbf{x}_{0,j}$ evaluated on the initial algebraic grid. Then condition **Eq. 3.4.12** implies that if $\mathbf{a}$ is the unit vector normal to the boundary, then

$$\mathbf{a} \equiv \frac{\mathbf{X}_\xi}{\|\mathbf{X}_{\xi\xi}\|} = \frac{y_\eta\ , -x_\eta}{\sqrt{x_\eta^2 + y_\eta^2}} = \frac{y_\eta\ , -x_\eta}{\sqrt{g_{22}}}$$

Eq. 3.4.20

Now the condition from **Figure 3.4.7** is

$$\mathbf{x}_\xi = P_a\left(x_\xi^0\right)$$

Eq. 3.4.21

where $\mathbf{Pa} = \mathbf{a}\mathbf{a}^T$ is the orthogonal projection onto the one-dimensional subspace spanned by the unit vector **a**. Thus we obtain

$$\mathbf{x}_\xi = \mathbf{a}\left(\mathbf{a}.x_\xi^0\right) = \frac{y_\eta\ ,\ x_\eta}{g_{22}}\left(y_\eta x_\xi^0 - x_\eta y_\xi^0\right)$$

Eq. 3.4.22

Finally, the reflection operation of **Figure 3.4.8** implies that the fixed ghost point location should be given by

$$\mathbf{x}_{-1,j} = x_{0,j} - \left(x_\eta\right)_{0,j}$$

Eq. 3.4.23

This can also be viewed as a first-order Taylor expansion involving the orthogonal derivative $(\mathbf{x}_\xi)_{0,j}$:

$$\mathbf{x}_{-1,j} = x_{0,j} - \Delta\xi\left(x_\eta\right)_{0,j}$$

Eq. 3.4.24

with $\Delta\xi = 1$. The orthogonal derivative $(\mathbf{x}_\xi)_{0,j}$ is computed in **Eq. 3.4.22** using only data from the boundary and the algebraic grid. Now in **Eq. 3.4.19** the control function evaluation at the boundary, the second derivative $\mathbf{x}_{\xi\xi}$ is computed using a centered difference approximation involving a ghost point, a boundary point, and an iteratively updated interior point. The metric coefficient g_{11} describing spacing normal to the boundary is computed using **Eq. 3.4.22** and is given by

$$(g_{11})_{0,j} = \left(\mathbf{x}_\xi\right)_{0,j}.\left(\mathbf{x}_\xi\right)_{0,j}$$

Eq. 3.4.25

Finally, note that the value for $(\mathbf{x}_\xi)_{0,j}$ used in **Eq. 3.4.19** is not the fixed value given by **Eq. 3.4.22**, but is the iteratively updated one-sided difference formula given by

$$\left(x_\xi\right)_{\mathbf{0,j}} = \mathbf{x}_{i,j} - \mathbf{x}_{0,j}$$

Eq. 3.4.26

Evaluation of quantities at the $\xi = m$ boundary is similar. Note, however, that the ghost point locations are given by

$$\mathbf{x}_{m+1,j} = \mathbf{x}_{m,j} - \left(\mathbf{x}_\xi\right)_{0,j}$$

Eq. 3.4.27

where $(\mathbf{x}_\xi)_{m,j}$ is evaluated in **Eq. 3.4.22** which is also valid for this boundary. On the "bottom" and "top" boundaries $\eta = 0$ and $\eta = n$, it is now the derivatives x_η, $x_{\eta\eta}$, and the spacing g_{11} that are evaluated using the fixed boundary data using central differences. Using similar reasoning to the "left" and

"right" boundary case, we obtain that for the "bottom" boundary the ghost point location is fixed to be

$$\mathbf{x}_{i-1,j} = \mathbf{x}_{i,0} - (\mathbf{x}_{\eta})_{i,0} \quad \text{where} \quad x_{\eta} = \frac{(-y_{\xi}, x_{\xi})}{g_{11}}(-y_{\xi}x^{0}_{\eta} + x_{\xi}\, y^{0}_{\eta})$$

Eq. 3.4.28

Here, $(-y_{\eta}, x_{\eta})$, g_{11} is evaluated using central differencing of the boundary data, and $(x_{o\eta}, y_{o\eta})$ represents a one-sided derivative $\mathbf{x}_{i,1} - \mathbf{x}_{i,0}$ evaluated on the initial algebraic grid. The metric coefficient $(g_{22})_{i,0} = (\mathbf{x}_{\eta})_{i,0} \cdot (\mathbf{x}_{\eta})_{i,0}$ is now computed using **Eq. 3.4.28**, and $\mathbf{x}_{\eta\eta}$ is computed using a ghost point, a boundary point, and an iteratively updated interior point. The value of $(\mathbf{x}_{\eta})_{i,0}$ used in **Eq. 3.4.19** is not the fixed value given in **Eq. 3.4.28**, but is the iteratively updated one-sided difference formula given by

$$(\mathbf{x}_{\eta})_{i,0} = \mathbf{x}_{i,1} - \mathbf{x}_{i,0}$$

Eq. 3.4.29

Finally, the "upper" $\eta = n$ boundary is similar, and we note that the ghost-point locations are given by

$$(\mathbf{x}_{\eta})_{i,0} = \mathbf{x}_{i,1} - \mathbf{x}_{i,0}$$

Eq. 3.4.30

with $(\mathbf{x}_{\eta})_{i,n}$, evaluated using **Eq. 3.4.28**. Quantities for the four corner points, $\mathbf{x}_{0,0}$, $\mathbf{x}_{m,0}$, $\mathbf{x}_{0,n}$ and $\mathbf{x}_{m,n}$, are computed somewhat differently in that no orthogonality considerations or ghost points are used. Indeed, the values $\mathbf{x}_{\xi}$, $\mathbf{x}_{\xi\xi}$, $\mathbf{x}_{\eta}$, $\mathbf{x}_{\eta\eta}$, g_{11}, g_{22} are all evaluated once using one-sided difference formulas that use the specified boundary values and do not change during the course of iteration. We forego imposition of orthogonality at the corners, because at the corners ***conformality*** is more important than ***orthogonality***. In other words, orthogonality at the corners should be sacrificed in order to ensure that the resulting grid does not spill over the physical boundaries in the neighborhood of the corners. For the case of highly obtuse or highly acute corners, it may in fact be necessary to relax orthogonality in the regions that are within *several* grid lines of the corners. One way to do this is to construct ghost points near the corners with the orthogonal projection operation **Eq. 3.4.21** omitted (i.e., constructed by simple extrapolation), and to use a blend of these ghost points and the ghost points derived using the orthogonality assumption. To further ensure that the elliptic system iterations do not cause grid folding near the boundaries, "guards" may be employed, similar to those mentioned in the previous section on Neumann orthogonality. In practice, however, we have found these to be unnecessary for Dirichlet orthogonality.

3.4.7 Blending of Orthogonal and Initial Control Functions

The orthogonal control functions in the interior of the grid are interpolated from the boundaries using linear transfinite interpolation and updated during the iterative solution of the elliptic system. If the initial algebraic grid is to be used only to infer correct spacing at the boundaries, then it is sufficient to use these orthogonal control functions in the elliptic iteration. However, note that the orthogonal control functions do not incorporate information from the algebraic grid beyond the first interior grid line. Thus if it is desired to maintain the entire initial interior point distribution, then at each iteration the orthogonal control functions must be smoothly blended with control functions that represent the grid density information in the whole algebraic grid. These latter control functions we refer to as "initial control function," and their computation is now described. The elliptic system **Eq. 3.4.11** can be solved simultaneously at each point of the algebraic grid for the two functions P and Q by solving the following linear system:

$$\begin{bmatrix} g_{22}\mathbf{x}_{\xi} & g_{11}\mathbf{x}_{\eta} \\ g_{22}\mathbf{y}_{\xi} & g_{11}\mathbf{y}_{\eta} \end{bmatrix} \begin{bmatrix} P \\ Q \end{bmatrix} = \begin{bmatrix} R_1 \\ R_2 \end{bmatrix} \quad \text{where} \begin{cases} R_1 = 2g_{12}\mathbf{x}_{\xi\eta} - \mathbf{g}_{22}\mathbf{x}_{\xi\xi} - \mathbf{g}_{11}\mathbf{x}_{\eta\eta} \\ R_2 = R_1 \end{cases}$$

Eq. 3.4.31

Where the derivatives here are represented by central differences, except at the boundaries where one-sided difference formulas must be used. This produces control functions that will reproduce the algebraic grid from the elliptic system solution in a single iteration. Thus, evaluation of the control functions in this manner would be of trivial interest except when these control functions are smoothed before being used in the elliptic generation system.

This smoothing is done by replacing the control function at each point with the average of the nearest neighbors along one or more coordinate lines. However, we note that the *P* control function controls spacing in the ξ-direction and the Q control function controls spacing in the η-direction. Since it is desired that grid spacing normal to the boundaries be preserved between the initial algebraic grid and the elliptically smoothed grid, we cannot allow smoothing of the *P* control function along ξ-coordinate lines or smoothing of the Q control function along η-coordinate lines. This leaves us with the following smoothing iteration where smoothing takes place only along allowed coordinate lines:

$$P_{i,j} = \frac{1}{2}\left(P_{i,j+1} + P_{i,j-1}\right) \quad , \quad Q_{i,j} = \frac{1}{2}\left(Q_{i+1,j} + Q_{i-1,j}\right)$$

Eq. 3.4.32

Smoothing of control functions is done for a small number of iterations. Finally, by blending the smoothed initial control functions together with orthogonal control functions, we will produce control functions that will result in preservation of grid density information throughout the grid,

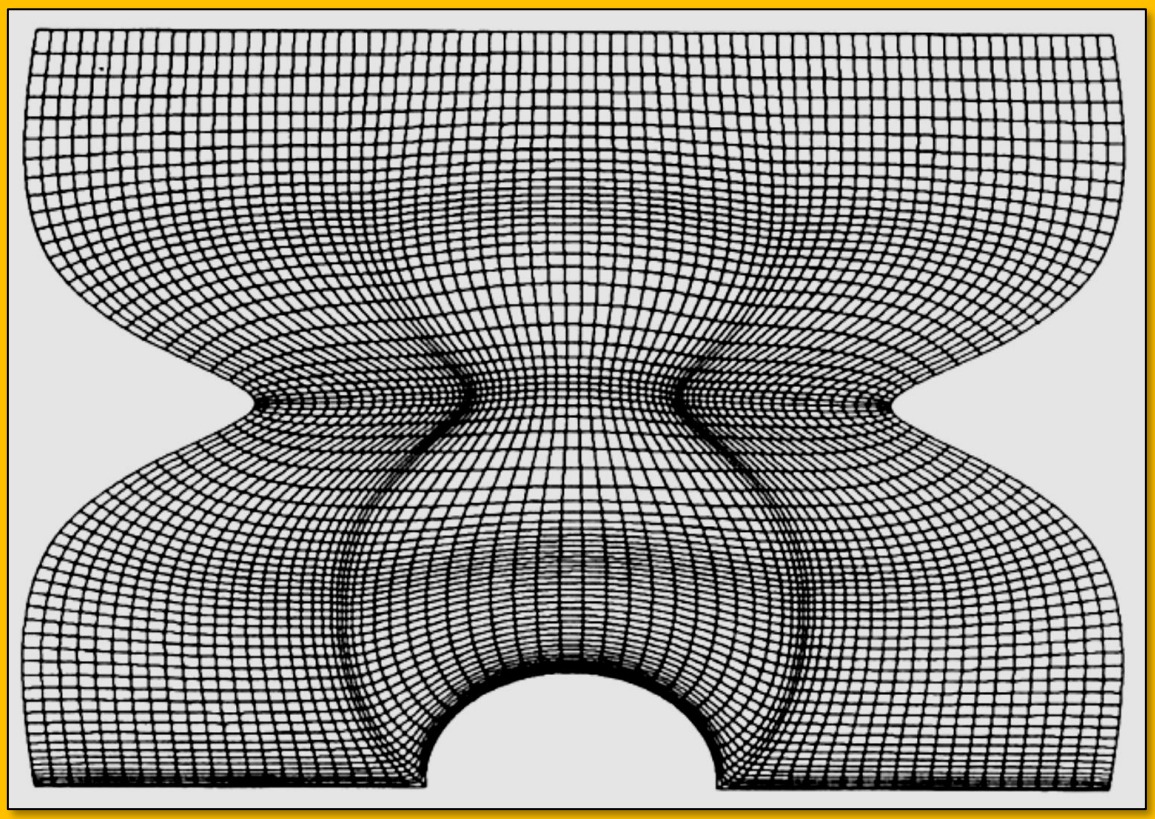


Figure 3.4.9 An Elliptic Planar Mesh on a Bi-Cubic Geometry with Dirichlet Orthogonality

along with boundary orthogonality. An appropriate blending function (b_{ij}) for this purpose is exponential blending function as described before. Now the new blended values of the control functions are computed as follows:

$$P_{i,j} = b_{i,j}P_0(i.j) + (1 - b_{i,j})P_1(i,j)$$
$$Q_{i,j} = b_{i,j}Q_0(i.j) + (1 - b_{i,j})Q_1(i,j)$$

Eq. 3.4.33

where P_0 and Q_0 are the orthogonal control functions from **Eq. 3.4.19**. P_I and Q_1 are the smoothed initial control functions computed using **Eq. 3.4.31** and **Eq. 3.4.32.**

As an application of Dirichlet orthogonality, in **Figure 3.4.9** we show the results of smoothing the algebraic grid of **Figure 3.4.5** using orthogonal control functions only. Like the grid produced using Neumann orthogonality, the grid is smooth, boundary-orthogonal, and no longer folds in the interior. However, unlike the grid of **Figure 3.4.6**, we see that the grid of **Figure 3.4.9** ***preserves*** the grid point density information of the algebraic grid at the boundaries. The effect of smoothing near the boundaries has been essentially to slide nodes along the coordinate lines parallel to the boundaries, without affecting the spacing between the coordinate lines normal to the boundary.

We note that if the user for some reason wished to preserve the *interior* clustering of grid points in the algebraic grid, then the above scheme given for blending initial control functions with orthogonal control functions would have to be slightly modified. This is because the fact that the algebraic grid is actually folded in the interior makes the evaluation of the initial control functions using **Eq. 3.4.31** ill-defined.

This is easily remedied by evaluating the initial control functions using **Eq. 3.4.31** at the boundaries only using one-sided derivatives, and then defining them over the whole mesh using transfinite interpolation. Since there is no folding of the algebraic grid at the boundaries, this is well-defined. (The interpolated initial control functions will reflect the grid density information in the interior of the initial grid, because the interior grid point distribution of the initial grid was computed using the same process transfinite interpolation of boundary data.) Then we proceed as above, smoothing the initial control functions and blending them with the orthogonal control functions.

Finally we note that if the algebraic initial grid possesses folding *at the boundary*, then using data from the algebraic grid to evaluate either the initial control functions or the orthogonal control functions at the boundary will not work. In this case, one could reject the algebraic grid entirely and manually specify grid density information at the boundary. This would however defeat the purpose of our approach, which is to simplify the grid generation process by reading grid density information off of the algebraic grid. Instead, we suggest that in this case the geometry be subdivided into patches sufficiently small so that the algebraic initial grids on these patches do not possess grid folding at the boundaries.

3.4.8 Boundary Orthogonality for Surface and Volume Meshes (3D)

Now we turn our attention to applying the same principles of the previous section to the case of surface grids. Our surface is assumed to be defined as a mapping $\mathbf{x}(u, v)$: $R^2 \rightarrow R^3$. The (u, v) space is the ***parametric space***, which we conveniently take to be [0,1] × [0,1]. The parametric variables are themselves taken to be functions of the computational variables ξ, η, which live in the usual [0, m] × [0, n] domain. Thus

$$\mathbf{x} = (x, y, z) = \big(x(u, v), y(u, v), z(u, v)\big) \quad \text{and} \quad (u, v) = [u(\xi, \eta), v(\xi, \eta)]$$

Eq. 3.4.34

The mapping $\mathbf{x}(u, v)$ and its derivatives $\mathbf{x}_u$, $\mathbf{x}_v$, etc., are assumed to be known and evaluable at reasonable cost. It is the aim of surface grid generation to provide a "good" mapping (u(ξ, η), v(ξ, η)) so that the composite mapping $\mathbf{x}(\mathbf{u}(\xi, \eta), \mathbf{v}(\xi, \eta))$ has desirable features, such as boundary

orthogonality and an acceptable distribution of grid points. A general method for constructing curvilinear structured surface grids and its derivates are presented in [Khamayseh][68] and will not be repeated here. Interested renders, should consult the above mentioned source, as well as the [Khamayseh & Mastin][69], [Warsi][70].

3.4.9 Stretching Functions

In order to further improve the quality of the mesh, one can introduce univariate stretching functions to either compress or expand grid lines in order to correct non-uniformity where grid lines are more or less dense. These functions are arbitrarily chosen and only reflect the distribution of grid lines. We can derive a new set of equations by combining our previously established differential model for grid generation and a set of univariate stretching functions of our choice. In order to do so in a straightforward manner, we can transform our Cartesian coordinates to a new set of coordinates which exists in a different space, called the parameter space. Then, we define our stretching functions as onto and one-to-one univariate functions of ξ and η respectively. For additional info, please consult the work by [71]-[72]-[73].

3.4.10 Extension to 3D

If we wished to extend the elliptic solver to 3D, we would need to develop equations for transitioning from a curvilinear coordinate system (ξ, η, ς) to a 3D Cartesian coordinate system (x, y, z). Using the same elliptic model as before, we get the 3D version of the Winslow equations where each of the metric tensor coefficients is determined by taking the cofactors of the contravariant tensor matrix. The contravariant tensor matrix is used to obtain the coefficients for the Winslow equations, which are the inverse of the Laplace equations as stated before. In general, if we wish to extend our elliptic mesh solver to n dimensions, then we will have n sets of equations each with $n!/(2(n-2)! + n$ terms. This renders the problem gradually more and more difficult to solve for higher dimensions with the existing elliptic scheme, implying that a different type of PDE might be needed in these cases. Another complication that arises in higher dimensions is adjusting grid lines to enforce orthogonality. Using the aforementioned algorithm for adjusting grid lines to achieve either complete or partial orthogonality on the boundary, we would need to iteratively solve three sets of two linear equations for each node in the mesh, as well as solve three trigonometric equations per iteration to compute the tangents. An excellent short web-cast from ***Pointwise©*** (right click here) is available regarding their development in structured meshing for an axial rotor blade.

3.4.11 Mesh Quality Analysis

In order to determine the quality of the resulting mesh, it was necessary to construct an objective means of quality measurement. Therefore, several ***statistical procedures*** were implemented in the program to produce a ***meaningful mesh quality analysis report***. The metrics which are presented are divided into the following categories:

- Orthogonality Metrics
  - Standard deviation of angles

---

[68] Ahmed Khamayseh, Andrew Kuprat, C. Wayne Mastin, "*Boundary Orthogonality in Elliptic Grid Generation*", CRC Press LLC, 1999.

[69] Khamayseh, A. and Mastin, C W., *Computational conformal mapping for surface grid generation, J. Com. Phys. 1996.*

[70] Warsi, Z.U.A., *Numerical grid generation in arbitrary surfaces through a second-order differential geometric model*, *J. Cot. Phys*. 1986.

[71] Chaitanya Varier 2017.

[72] M. Farrashkhalvat and J.P. Miles, "*Basic Structured Grid Generation*", An imprint of Elsevier Science Linacre House, Jordan Hill, Oxford OX2 8DP, 200 Wheeler Rd, Burlington MA 01803, First published 2003.

[73] Joe F. Thompson, Z. U. A. Warsi, C. Wayne Mastin, "*Numerical Grid Generation – Foundations and Application*", Mississippi State, Mississippi, January 1985.

- Mean angle
- Maximum deviation from 90 degrees
- Percentage of angles within x degrees from 90 degrees (x can be set as a constant in the code)

- Cell Quality Metrics
  - Average aspect ratio of all cells
  - Standard deviation of all aspect ratios

3.5 Case Study 3 - *PADRAM*: Parametric Design and Rapid Meshing System for Turbomachinery Optimization

Authors : *Shahrokh Shahpar and Leigh Lapworth*
Affiliations : *Rolls-Royce plc, Derby, U.K.*
Citation : *Shahpar, S, & Lapworth, L. "PADRAM: Parametric Design and Rapid Meshing System for Turbomachinery Optimisation." Proceedings of the ASME Turbo Expo 2003, collocated with the 2003 International Joint Power Generation Conference. Volume 6: Turbo Expo 2003, Parts A and B. Atlanta, Georgia, USA. June 16–19, 2003. pp. 579-590. ASME. https://doi.org/10.1115/GT2003-38698*
Source : *GT2003-38698 - Proceedings of ASME Turbo Expo 2003, Power for Land, Sea, and Air, June 16–19, 2003, Atlanta, Georgia, USA*

3.5.1 Abstract

A parametric design system suitable for inclusion in an automatic optimization process is presented. The system makes use of a multi-block structured grid generation system specially designed for the rapid meshing of two-dimensional, quasi three-dimensional, and three-dimensional single passage as well as multi-passage, multi-row turbomachinery blades. Full annulus viscous meshes of the order of five to ten million mesh points for the complete bypass assembly of the low pressure compression (LPC) system can be generated in a matter of minutes. *PADRAM* offers a major new design capability where the optimization of multi-passage three-dimensional blades and its circumferential pattern is done simultaneously in one system. Successful usage of *PADRAM* in a number of design, optimization and analysis applications has recently been demonstrated and reported herein.

Nomenclature

a, b, c	elliptic partial-differential-equation coefficient
m'	non-dimensional streamwise coordinate
r, θ, *z*	physical, polar co-ordinates: radial, circumferential and axial direction
P, Q	elliptic grid generator forcing functions
y	normal to the wall distance
y+	non-dimensional wall distance, = $y(\tau_w/\nu^2\rho)^{1/2}$
x, y, z	Cartesian coordinates
α , β	clustering function coefficient
ξ , η	coordinates in the computational plane
θ	azimuthal angle
ν	kinematic viscosity
ρ	density
τ_w	wall shear stress

3.5.2 Introduction

As Computational Fluid Dynamics (CFD) codes have steadily evolved into everyday analysis tools, so attention is now focused on integrating the analysis codes with design tools paving the way to carry-out automatic optimization. Automatic optimization tools and methodologies have proved able to significantly reduce the manual design time and simultaneously improve the quality of the designs, e.g. see references [1] and [2]. Traditionally the CFD pre-processing tools are embedded within a Graphical User Interface (GUI) environment. The user-friendly environment which a GUI provides to engineers is deemed a necessary and essential part of the CFD process. However, automatic optimization strategies require a parametric definition of the geometry and the transfer of geometry from CAD or a database, such as blade definition file, to the mesh generator to be seamless with no user intervention. GUIs must be scriptable, otherwise they become bottlenecks in the optimization process.

In the design by analysis approach, spending a few days to set up a new geometry interactively and spending a couple of hours, using a very good mesh generator to produce a suitable computational grid has been regarded as reasonable. However, this is totally unacceptable for inclusion in an optimization loop, as the design process cannot be automated and the overall optimization time would be too long to be of use to the designer.

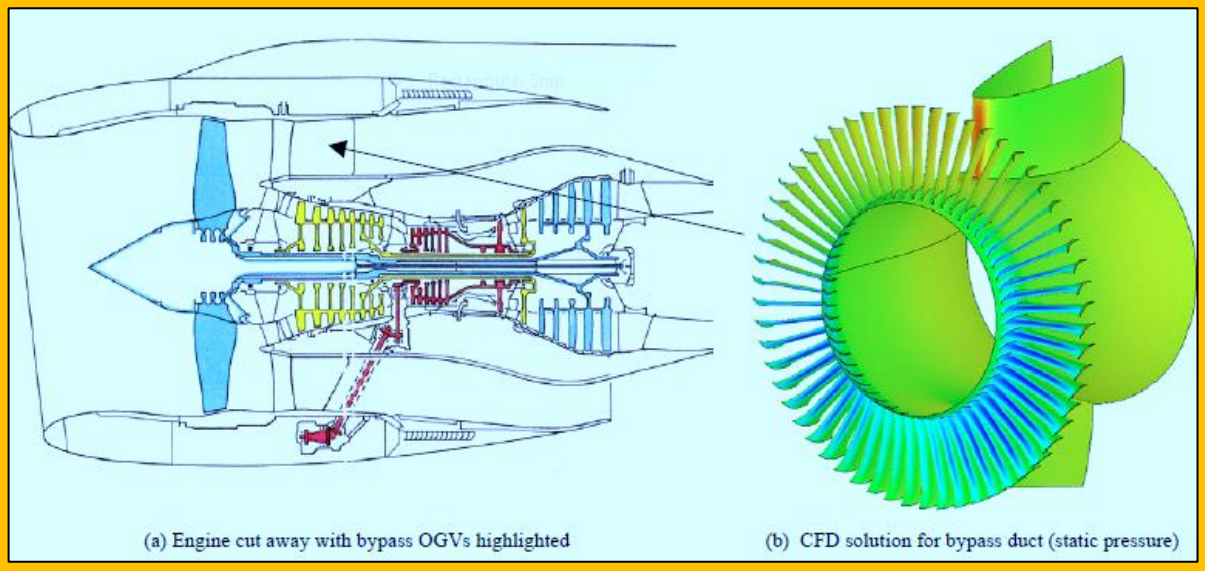


Figure 3.5.1 Bypass duct components, Outlet Guide Vane (OGV), Pylon and Radial Drive Faring (RDF)

The grid generator presented in this paper arose not only from the need to perform rapid mesh generation within an optimization scheme; but, also the need to mesh rings of vanes where each vane can potentially be different from all the others. Error! Reference source not found. shows an engine cut-away along with a CFD representation of the bypass duct. Downstream of the Bypass Outlet Guide Vanes (OGVs) there are the engine pylon and Radial Drive Faring. These create large asymmetric pressure distortions which propagate upstream through the OGVs and interact with the fan rotor. The level of pressure distortion has a direct influence on the forced response levels of the fan rotor. By *tailoring* the OGVs (i.e. making them non-circumferentially uniform) they can be used to attenuate the pressure distortion and reduce/eliminate fan forcing. If the tailoring of the OGV geometry is to be performed by an optimization system (see Shahpar et al., [2]), particularly one that uses heuristic optimizers, then there can be strong variations in the geometry of adjacent OGVs during the design cycle.

Grid generation for turbomachinery is relatively well evolved for generating simple blade-to-blade mesh sections that are then *stacked* radially to create a 3D mesh. The typical types of meshes used in the field of turbomachinery CFD are block structured [3-4], unstructured [5-6], overset or the so-called Chimera [7-9], hybrid [10-11] and Cartesian grids [12]. In the following discussion, we concentrate on mesh generation for turbomachinery blades. A more detailed discussion of various grid generation techniques is beyond the scope of this paper, interested readers should refer to a number of good books published in this field, e.g. Thompson *et al*. [13]. Among the grid generation techniques, the block-structured is the most powerful and is the most established, in particular for external aerodynamics CFD. Complex geometries can either by grided by unstructured grids or by structured multi-block grids.

Unstructured grids typically have memory and CPU overheads due to the need store mesh connectivity data, but offer the greatest geometrical flexibility. On-the-other-hand multi-block structured grids are very efficient and can fill-up topologically complex domain by decomposing the

geometry into simple blocks. Multi-block meshes have successfully been used in the context of MDO (Multi-Disciplinary Optimization) sensitivity study [14].

The single–block structured H-mesh topology is illustrated in **Figure 3.5.2** and **Figure 3.5.3**. The *sheared* H-mesh [15] shown in **Figure 3.5.2** leads to a very fast CFD solver because the solver exploits the fact that the circumferential mesh lines have a constant axial co-ordinate. This approach is clearly too restrictive for the current work where the axial position of each OGV may vary circumferentially around the annulus. The *letter-box* H-mesh [16] shown in **Figure 3.5.3** uses a curvilinear mesh topology and is more-flexible than the sheared H-mesh, but the high aspect ratio cells in the cross-passage direction at the leading and trailing edge would also restrict the geometric flexibility of the optimizer. A single–block structured C-mesh topology is shown in **Figure 3.5.4**. For single passage meshes this has good resolution of the leading and trailing edges and also the vane wake; but, has a mesh singularity at the junction of the upstream and periodic boundaries. The C-mesh has more flexibility for movement of the OGVs than the H-meshes, but is difficult to interface with the downstream meshes for the pylon and RDF. The multi-block structured H-O-H mesh topology [17] is shown in **Figure 3.5.5**. Here, the flexibility to vary the axial position of the OGVs around the annulus is limited by the requirement to have degenerate nodes at the junction of the O and H-meshes with the periodic boundaries. The periodicity means one O-block cannot be moved independently of its neighbor. An unstructured mesh topology [18] is shown in **Figure 3.5.6**. This has a structured O-mesh around the vane and an unstructured mesh of triangles in the freestream and wake regions. The triangles become prisms when the mesh is stacked in 3D. This style of mesh can easily accommodate a wide range of perturbations to the OGV geometry around the annulus. The difficulty of this approach is that to preserve the prismatic topology of the unstructured region, the

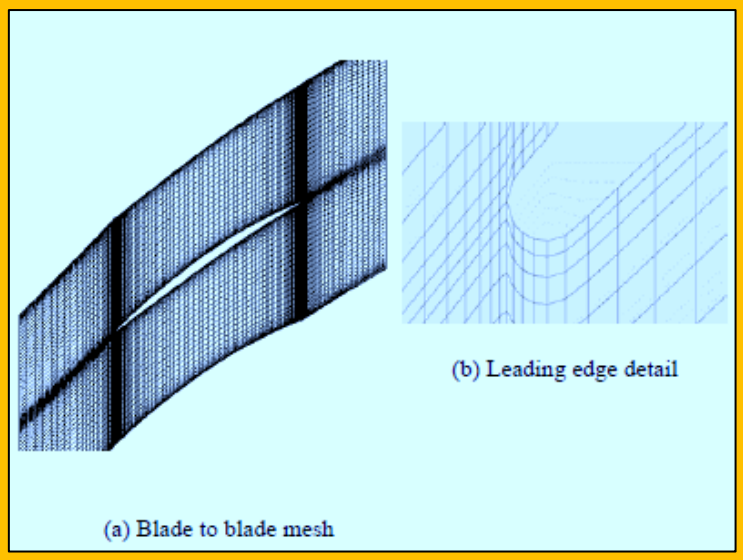


Figure 3.5.2 Sheared H style mesh for a compressor blade, with leading edge detail

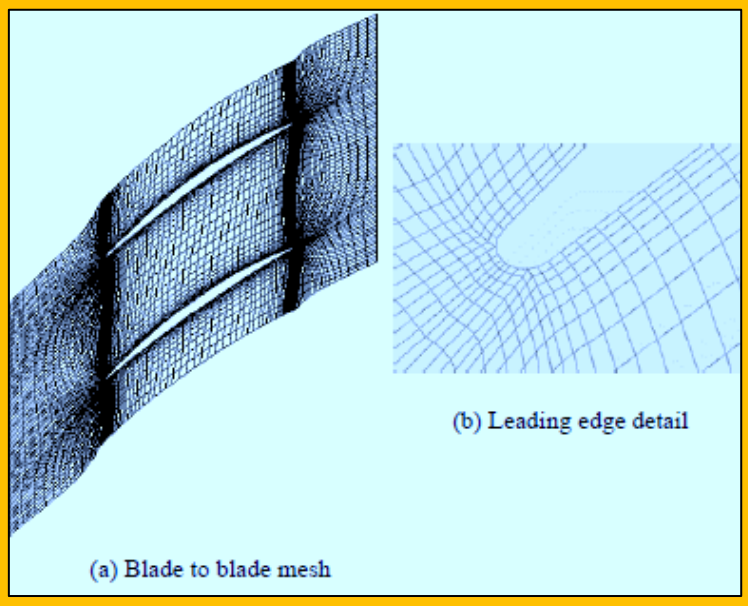


Figure 3.5.3 Letter-box style mesh for a compressor blade, with leading edge detail

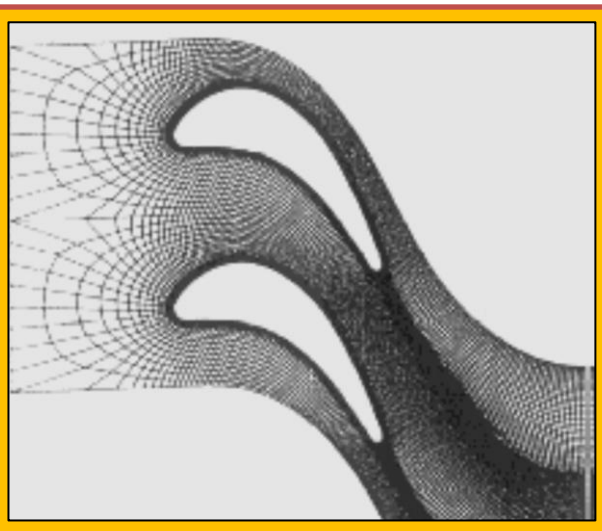


Figure 3.5.4 C-type mesh for a Turbine blade

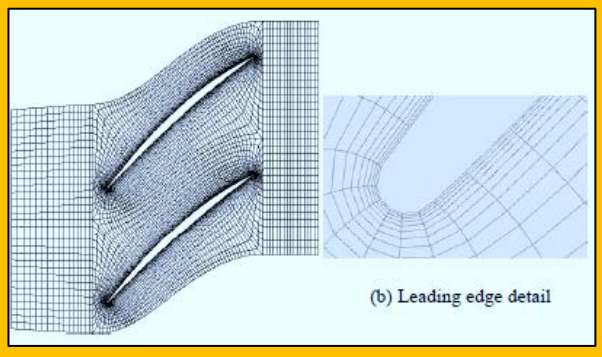


Figure 3.5.5 H-O-H grid for a compressor blade with leading edge detail of the O-mesh

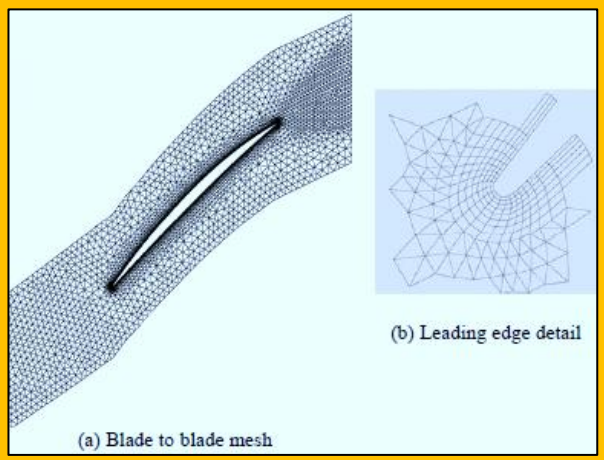
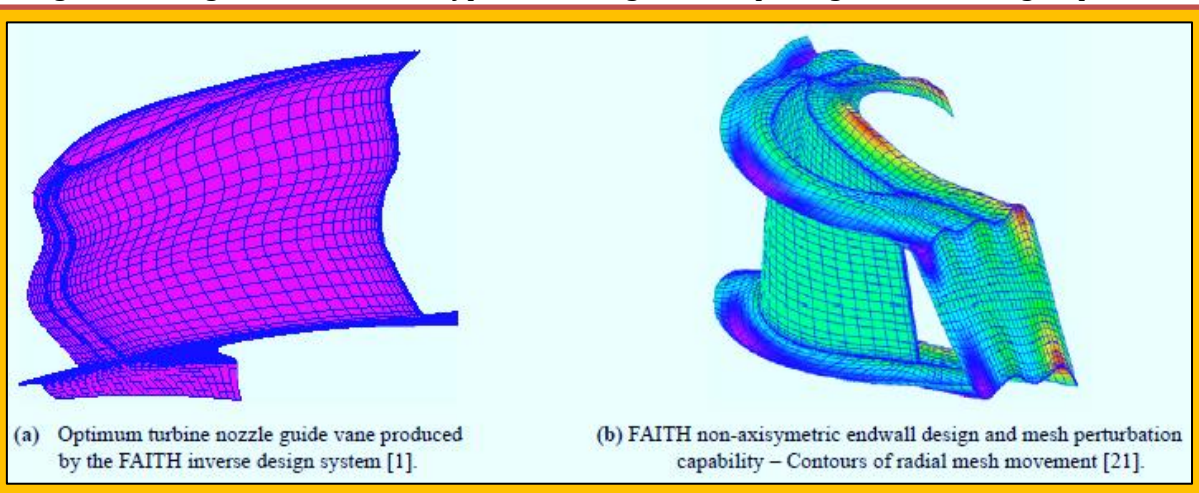


Figure 3.5.6 Hybrid-mesh for a compressor blade with leading edge detail

blade-to blade mesh must be generated on a single section and then cloned to the other sections. This can lead to significant shearing of the prisms if the optimizer generates strongly three dimensional blades. When generating new meshes not only the topology but also the grid density needs to be kept as consistent as possible to the original datum geometry in order to minimize the errors in computing the flow sensitivities. This is particularly difficult to control when using unstructured grid generators as the total number of grid points cannot be fixed *a priori*. Fully unstructured meshes are too time consuming to be generated within the optimization cycle, if only because they are working in 3D and not in a stacked 2D mode. Chimera or overset meshes [7-9] are a flexible way to apply structured mesh solution techniques to geometrically complicated multi-body configurations. Novel applications have been reported for dynamic systems like store release or helicopter rotor flow simulation. Overset meshes can easily accommodate large variations in the OGV geometry around the annulus, but require *care* with interpolation schemes in the overlap regions. Wadia *et al.* [19] reported that the Chimera grid generation process was extremely complicated for the analysis of their 16 vanes and 1 strut configuration.

The Chimera technique was used to create a mesh for the sixteen variably-staggered OGVs, strut and splitter. Their process started with untrimmed initial surface grids for each component that was generated from CAD data using the *ICEM CFD* program. A series of Fortran programs, shell scripts and various grid tools were used to rotate components to their respective positions, find surface-to-surface intersections, and construct the final surface grids. Volume grids for the strut and vanes were generated using a hyperbolic grid generator while an algebraic grid generator was developed for the other components. An additional program was used to perform the Chimera hole-generation and determine the grid-to-grid boundary interpolation coefficients. The final grid configuration consisted of sixteen-vane grids overlaid on an

outer grid, a strut grid, and core and bypass section grids, comprising of 2.7 million grid points.



(a) Optimum turbine nozzle guide vane produced by the FAITH inverse design system [1].

(b) FAITH non-axisymetric endwall design and mesh perturbation capability – Contours of radial mesh movement [21].

Figure 3.5.7 Application of mesh perturbation within design

The majority of optimizers begin with a CFD solution for base or reference geometry. Since this is upstream of the optimization cycle, more time can be taken in generating the base mesh. Indeed, time is generally taken at this stage to define default mesh generation parameters that are then fixed throughout the optimization process. Tschirner *et al.* [20] have recently reported viscous 2D analysis of 31 OGVs, 6 fairings and a pylon. They have reported an automated procedure using the *FLUENT* grid generator *GAMBIT* and several UNIX scripts. The manual grid generations necessary to perform the design and analysis with *GAMBIT* was reported to be extremely time consuming. Hence, a combination of several UNIX shell scripts and libraries were used in conjunction with an interactive user interface for data input by the user. Re-staggering and re-blading of the OGVs were done in a different, in-house system that designs blades in a single passage environment. If mesh generation is time consuming, then an alternative approach is to use a mesh movement algorithm to *morph* the mesh onto each new geometry generated by the optimizer.

Figure 3.5.7 show a successful application mesh perturbation techniques to both aerofoil [1] and end wall design [21]. Although, this approach is quite popular, it is the authors' experience that producing good quality viscous meshes in three-dimensions for complicated geometry is very difficult and time consuming. This is particularly true when the geometry is changing significantly from its datum shape for example during a heuristic optimization run. It has been found [22] that the approach of changing the geometry gradually and perturbing the meshes can lead to erroneous geometry (see **Figure 3.5.8**) and meshes (if geometry changes abruptly); and, also can take up to ten hours of CPU on a compute server which clearly makes the strategy of the mesh perturbation impractical to include in an automatic optimization cycle.

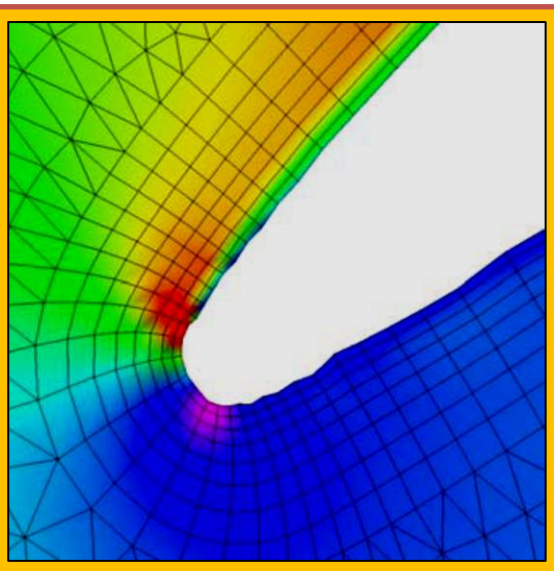


Figure 3.5.8 Erroneous geometry and mesh generated by perturbing the datum 3D BOGV in a ring of 52 OGVs

In this paper, a unique design system is presented that allows the design of multi-passage 3D blades and their circumferential pattern to be carried out simultaneously in one system. A novel method is presented that allows consistent good quality viscous meshes to be generated very rapidly for multiple passages of three-dimensional blades requiring no more than a few minutes of CPU time. Since the mesh generation is extremely fast and occur in real time, the users can tune the default parameters that control the quality of the 3D mesh upstream of the optimization runs interactively and easily.

This system is called *PADRAM* (*Parametric Design and Rapid Meshing System*). *PADRAM* can automatically and parametrically change the 2D, quasi-3D and 3D blade geometry and produce very rapidly good quality viscous mesh for the multi-passage, multi-stage turbomachinery passages. *PADRAM* makes uses of both transfinite interpolation and elliptic grid generators to generate hybrid C-O-H meshes. An orthogonal body-fitted O mesh is used to capture the viscous region in the vicinity of the aerofoil whilst an H mesh is used near the periodic boundaries and where stretched cells are required, for example in the wake.

The H-meshes also *take up the slack* from the relative movement in adjacent O-meshes as the OGVs are being redesigned. The mesh is independently generated for every stream section, hence three-dimensional meshes are produced easily from stacked two-dimensional blade section meshes with no mapping required to transfer the meshes radially. This avoids the mesh morphing, and ensures that good quality meshes are created at every height even if the geometry is changing considerably from hub to tip. Mesh generation is extremely fast, requiring no more than a few seconds to generate 2D meshes for the complete bypass OGV or a single passage 3D mesh; and, a few minutes to generate a mesh of the order of 6 million points for the complete bypass assembly (on Sun Ultra 10 workstations).

3.5.3 Methodology

3.5.3.1 Geometry Modelling

Turbomachinery blade geometry is defined at a number of radial stream sections These radial sections lie on a three dimensional surfaces defined in polar coordinates as $S_i(r, \theta, z)$, where i is the section index. Using non-dimensional parametric coordinates, m' and θ, a typical surface can be defined as:

$$r_s = r(m, \theta) \quad , \quad \theta_s = \theta(m, \theta) \quad , \quad z_s = z(m, \theta)$$

Eq. 3.5.1

Where,

$$\acute{m} = \int_{z_0}^{z} \frac{\sqrt{dr^2 + dz^2}}{r(z)} \quad , \quad r = r(z)$$

Eq. 3.5.2

Where z_0 is an arbitrary reference and *m'* is a nondimensional distance along a stream section which will be zero at z_0. The starting point for *PADRAM* system is to transform the radial stream sections into parametric two-dimensional planes, using the co-ordinates θ and *m'*. As *r* is greater than zero for any *z* coordinates, *m'* is a monotonic function of *z*, hence a unique inverse function exists to map the computational coordinates back to the physical, three-dimensional polar coordinates. The advantage of the above transformation is that the angles are preserved and the mesh-generation procedure deals with plane sections only.

3.5.3.2 Grid Generation

PADRAM grid generation starts by dividing the computational blocks into sub-blocks for the purpose of generation of the algebraic grid and the control functions. O- type grid is used for the blades and H-type grids near the periodic boundary, upstream and downstream blocks, C-type grids are used for semi-infinite boundaries such as the splitter, pylon and RDF (Radial Drive Fairing). ***Transfinite***

interpolation (TFI) [13] is used to generate the initial grid based on a linear interpolation of the specified boundaries. TFI is very easy to program, computationally efficient and the grid spacing is under direct control. *PADRAM* uses the following double clustering functions:

$$y = \frac{(2\alpha+\beta)\left[\frac{\beta+1}{\beta-1}\right]^{\frac{\eta-\alpha}{1-\alpha}} + 2\alpha - \beta}{(2\alpha+\beta)\left\{1+\left[\frac{\beta+1}{\beta-1}\right]^{\frac{\eta-\alpha}{1-\alpha}}\right\}} \quad , \quad 1<\beta<\infty \ , \quad 0\le\alpha\le 1$$

Eq. 3.5.3

Where η is the non-dimensional grid point distribution in the computational plane which varies between zero and 1, β is the clustering function, more clustering is achieved by letting β to approach 1 and α is a non-dimensional quantity that indicates which grid location the clustering should be attracted to, e.g. a value of 0.5 ensures clustering is uniformly done at both ends. *PADRAM* solves a system of ***elliptic partial differential equations (PDEs)*** to smooth the algebraically generated grids in each block. For each radial height, the following equations are solved:

$$am_{\xi\xi} - 2bm_{\xi\eta} + cm_{\eta\eta} = P(\xi,\eta)$$
$$a\theta_{\xi\xi} - 2b\theta_{\xi\eta} + c\theta_{\eta\eta} = Q(\xi,\eta)$$

Where,

$$a = m_\eta^2 + \theta_\eta^2 \quad , \quad b = m_\xi m_\eta + \theta_\xi\theta_\eta \quad , \quad c = m_\xi^2 + \theta_\xi^2$$

Eq. 3.5.4

P and Q are the forcing functions to ensure orthogonality and grid clustering at the surfaces. The grids presented in this paper were generated with the forcing functions switched off which saves significant computational time (typically 50%). However, the grid orthogonality near the surfaces was ensured by using a hyperbolic-type grid generation to generate the O grids, that is using a marching procedure to grow the grid normal to the boundaries. **Eq. 3.5.4** can be solved by an iterative solver such as the ***Gauss-Siedel scheme*** or a ***point successive over relaxation scheme*** (PSO). *PADRAM* makes use of the latter scheme to elliptically smooth the ***TFI*** generated grids. **Figure 3.5.9** shows the *PADRAM* mesh topology for a single OGV mesh. Note that the upstream and downstream H-mesh can also be rotated to align the mesh with the blade inlet and exit metal angles.

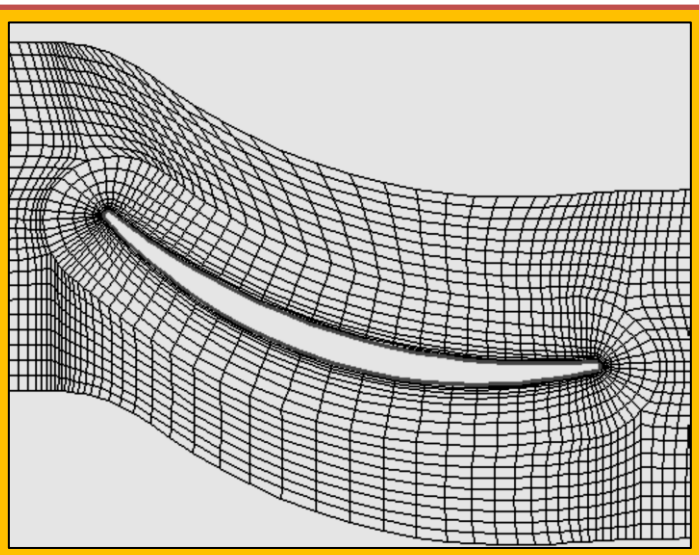


Figure 3.5.9 PADRAM mesh for a compressor blade H-O-H topology

3.5.3.3 Tip Gaps

Tip clearance is often required for shroud less rotor blades such as fans, compressors and some turbines as well as hub clearance for cantilevered stator blades. Tip leakage flows are often poorly

resolved. If the number of points in the tip clearance is too low, then the complex physical phenomena that occur there cannot be accurately modelled. It has been found that the flow solution can be sensitive how the gap is modelled and in some solvers the geometry is changed to alleviate the problem, e.g. sheared H-meshes often use the so called "pinched" tip gap shown in **Figure 3.5.10**. However, *PADRAM* models the gap in a consistent way that meshes the blade geometry. Once the O-grid is generated with the outer domain of the blade, the grid corresponding to the solid part of the domain is constructed using the same boundary node distribution. It is important to keep the mesh spacing in the inner and outer part of the wall as close as possible to each other. **Figure 3.5.11** shows a typical tip gap mesh for a compressor rotor generated in *PADRAM* (the tip gap has been increased to 5% span for demonstration). The work of Van Zante [23] has shown the importance of extra mesh clustering close to the shroud in meshes with a tip gap. This can be achieved in *PADRAM* using specified clustering parameters.

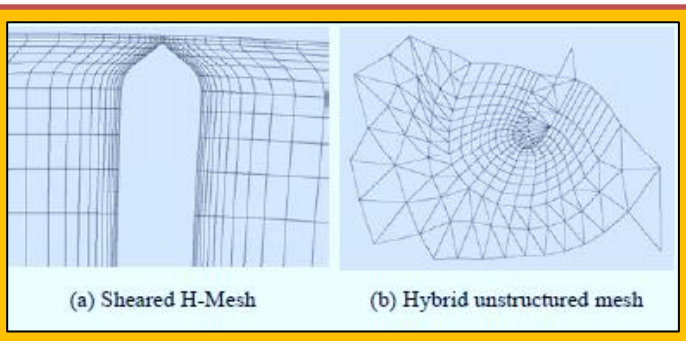


Figure 3.5.10 Tip clearance models

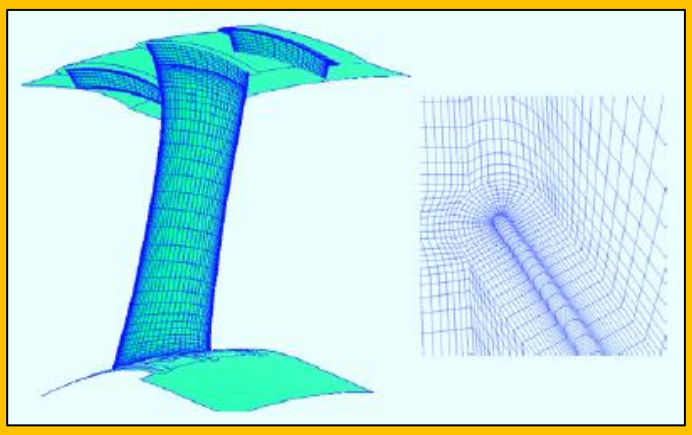


Figure 3.5.11 PADRAM's multi-passage tip clearance model (5% gap for demonstration)

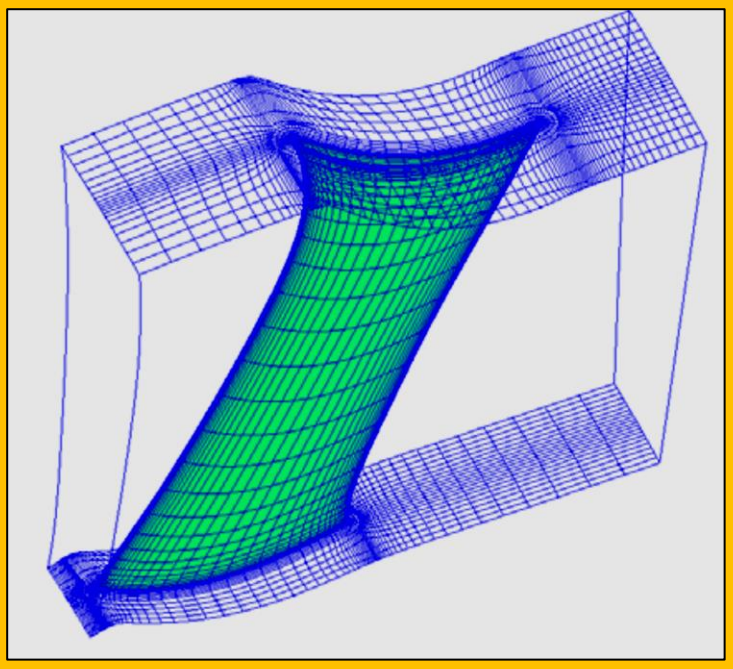


Figure 3.5.12 Single passage PADRAM mesh for the swept back OGV

3.5.4 LP Compression Design System

3.5.4.1 Design Considerations

In order to reduce or eliminate the effects of downstream struts or pylons, the bypass outlet guide vanes (BOGVs) can be reconfigured, e.g. circumferentially re-staggered or re-cambered. A detailed review of various design methods is reported in reference [2] and is not repeated here. However, most of the reported analyses in the literature are 2D or 3D single passage calculations. Long computational

time and generating suitable CFD meshes for the complex three-dimensional full annulus geometry are two main factors deterring engineers to carry out routine analysis of reconfigured BOGVs. *PADRAM* provides a unique design system that allows the design of multi-passage 3D blades and their circumferential pattern to be carried out simultaneously in one system.

3.5.4.2 Bypass Duct Meshing

One key feature of *PADRAM* is the fact that it is intimately linked with the geometry definition. Within the ring of the BOGVs the geometry of each OGV can be specified independently of the others either by using a different geometry definition file or by using one of the several design options within *PADRAM* (see below). Hence, *PADRAM* is easily able to produce meshes for variable stagger OGVs with a fixed base geometry; or, to design for sophisticated OGV patterns.

Another important feature of *PADRAM* is that it is a true multi passage meshing system. The geometry of each OGV in the annulus can be varied independently and a good viscous mesh for the complete ring of OGVs is generated in one step. This is both more efficient and more robust than the practice of generating a symmetric ring of identical OGVs and then morphing the mesh to match the actual asymmetric geometry of the OGVs.

PADRAM can construct a viscous mesh for the complete bypass assembly consisting of 52 different staggered and three types over and under cambered OGVS, pylon, RDF and a splitter in a matter of minutes. The details of the mesh near the OGV, Pylon, RDF and the splitter are shown in **Figure 3.5.12-Figure 3.5.15**. **Figure 3.5.1 b** shows a typical CFD solution.

3.5.4.3 Design/Optimization System

In order to perform design iterations, *PADRAM* requires the reference geometry for the OGVs, pylon/RDF and end walls. The reference OGV geometry may include one or more distinct blade shapes in a prescribed circumferential pattern. Each OGV is then assigned a set

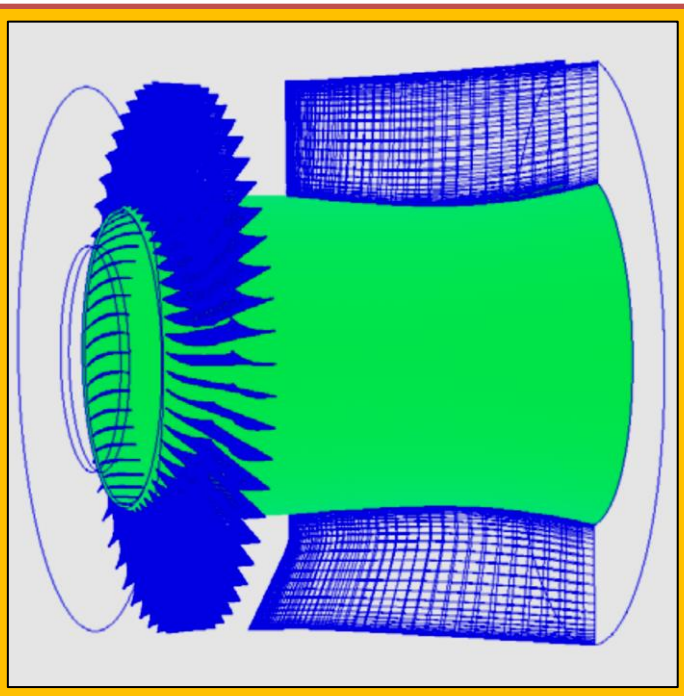


Figure 3.5.13 PADRAM mesh for the 52 bypass OGVs, pylon, RDF, and splitter consisting of 119 blocks

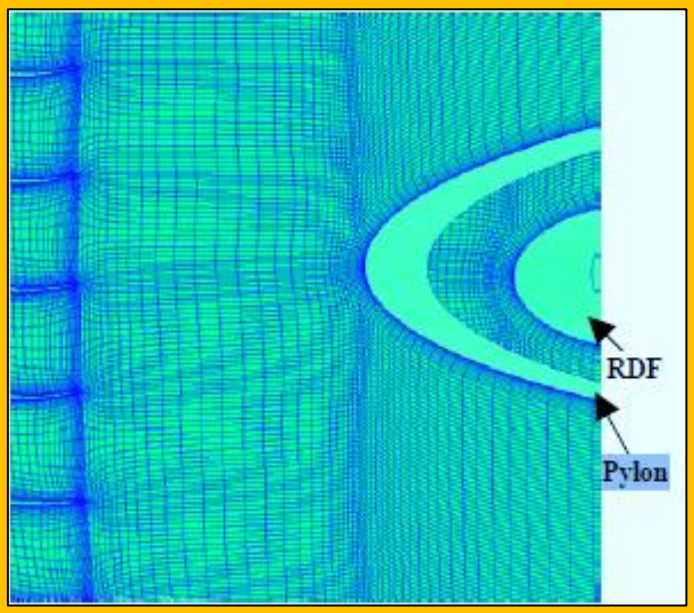


Figure 3.5.14 Detail of the C-mesh near the Pylon and RDF – top View

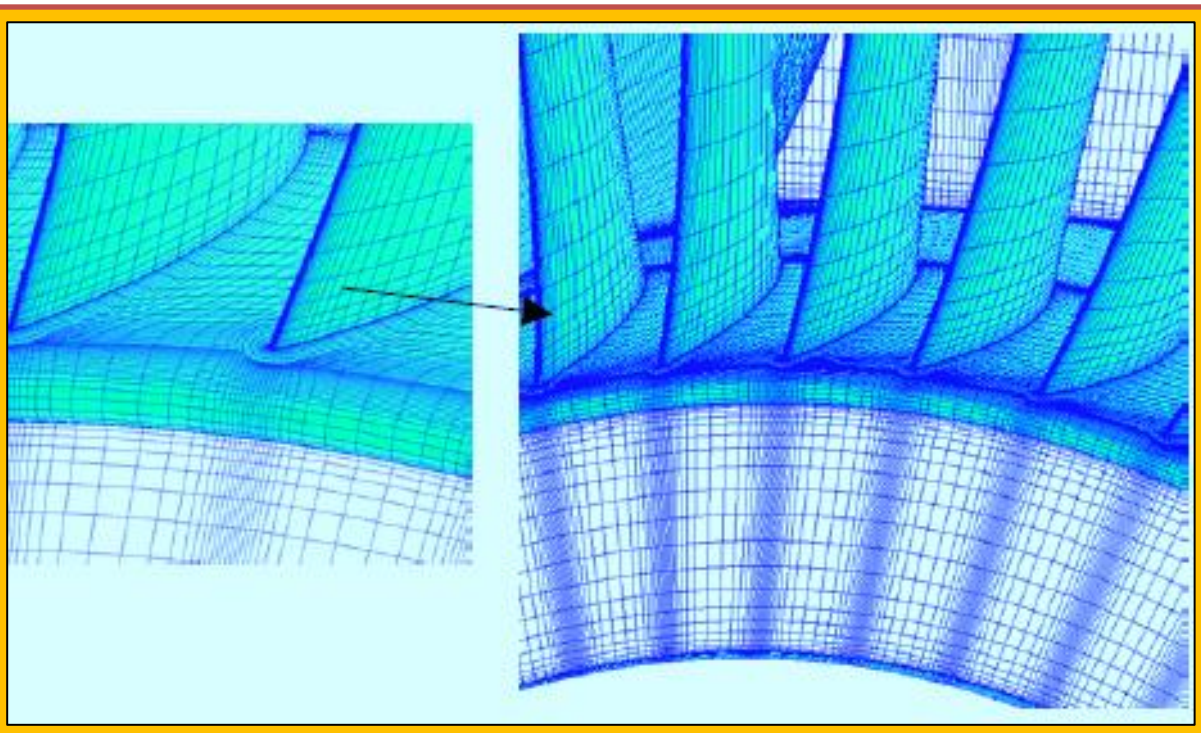


Figure 3.5.15 Detail of the splitter C-mesh, hub mesh and the engine-core exit mesh

of design parameters (see next section) which can be varied as a function of %span. Hence, the design system is able to, simultaneously, construct new 3D blade shapes and new circumferential patterns. The mesh generation parameters are tuned for the reference geometry and then held fixed for the design iterations. Since, the mesh generation parameters are parametrically related to the blade geometry, this system ensures consistent meshes are generated even if the geometry changes considerably during the optimization.

3.5.4.4 Parametric Design Space

The *PADRAM* design space includes standard turbomachinery design parameters such as lean (or pitch variation in 2D multi-passage simulation), sweep (or axial variation of sections), leading-edge and trailing-edge re-camber. A sample of design parameters and *PADRAM* meshing capability are shown in **Figure 3.5.16** and **Figure 3.5.17**. The mesh is generated after the new leant geometry has been constructed, this will produce a better mesh than the mesh perturbation approach. **Figure 3.5.16 b** shows a differential 0 to 2° leant geometry and mesh generated by *PADRAM* for a bypass OGV. Successful automatic design and optimization of the bypass OGVs in the presence of the Pylon and RDF using *PADRAM* is reported in reference [2] and is therefore not repeated here.

3.5.4.5 Remote Optimization

Another feature of the *PADRAM* system is that it presents a means of making use of remote computer resources. *PADRAM* is written in ANSI 'C' for complete portability. The mesh generation parameters can be defined locally, where the ability to view the mesh is essential. The geometry files, *PADRAM* input file and pre-processing scripts, which are only a few Kbytes in size, can be sent electronically to the remote machine. Since *PADRAM* is extremely fast, the mesh generation and pre-processing can then be done remotely, avoiding the need to send mesh files of many tens or hundreds of megabytes to the remote machine via CD or DAT tape. Salient details from the solution can also be extracted

remotely before deciding whether the full CFD solution should be returned on CD/DAT tape to be examined locally.

3.5.5 Further Applications of *PADRAM*

Although *PADRAM* was initially developed for the parametric design and rapid meshing of the LP compression system, its robust mesh generator and its design space have been used on a number of other project applications. The direct link between the geometry and mesh make it an ideal tool for design studies whether those are based on an optimizer or on *design by analysis.*

3.5.5.1 Single Stage Research Fan

PADRAM has also been used to set up a suitable mesh for a single stage research fan to demonstrate its ability to provide meshes for mixing and sliding plane analyses. This single stage fan is a very severe test for a mixing plane analysis because the rotor and stator are closely coupled at the hub, but have a very wide spacing at the tip. This represents problems both in terms of mesh generation and stability of the CFD calculation. Using typical civil fan default values for the *PADRAM* mesh generation parameter produced a rotor mesh where the outer boundary for the O-mesh extended past the exit boundary.

The capability in *PADRAM* to independently set the mesh clustering parameters at hub and casing was crucial to getting an acceptable mesh for this test case. In fact, it was necessary to add independent control of the clustering at mid-height (with the clustering parameters linearly interpolated between the hub, mid and tip values) to get a fully satisfactory mesh. The final mesh is shown in **Figure 3.5.18** and consists of 334k nodes in the rotor mesh and

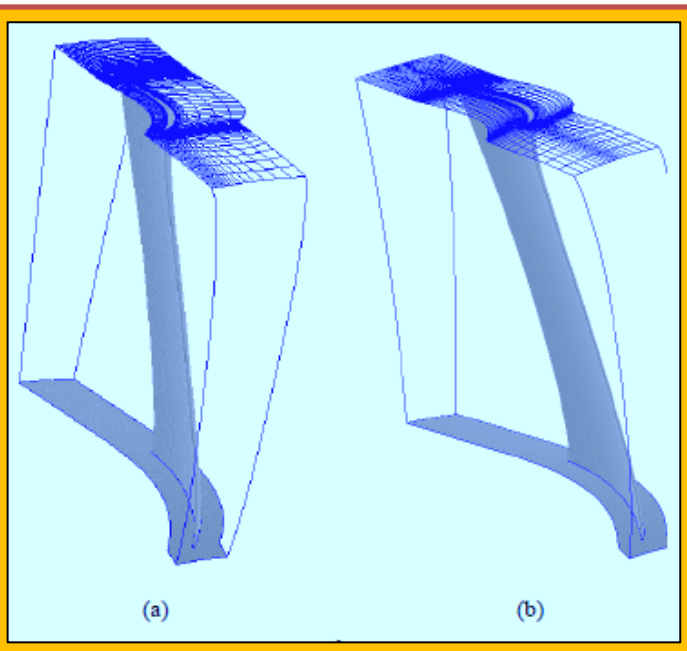


Figure 3.5.16 Differential 0 to 2o lean, (a) perturbed mesh with Fixed periodic boundaries, (b) PADRAM mesh

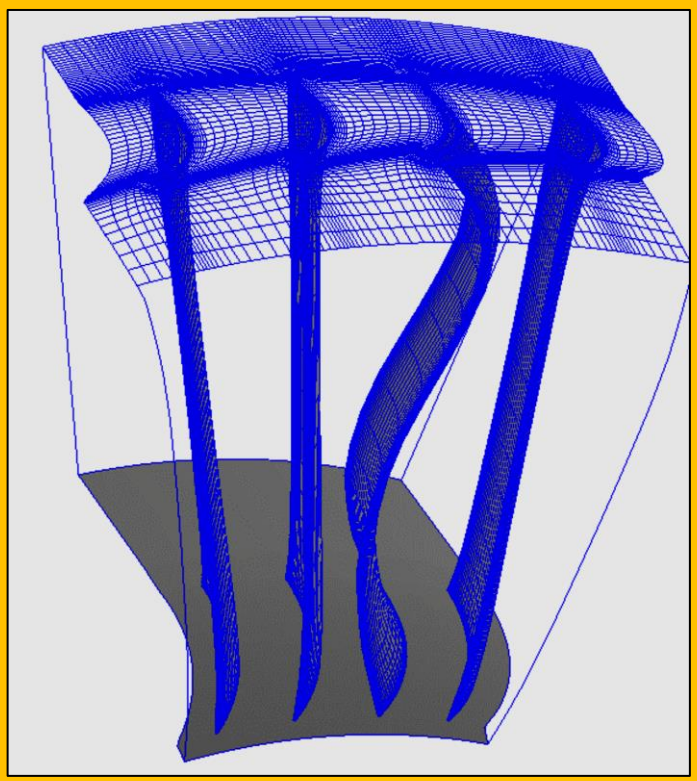


Figure 3.5.17 An example of the PADRAM's flexible design system to vary an OGV's lean in a row of four OGVs

212k nodes in the stator mesh. **Figure 3.5.18** also shows the mesh detail at the hub in the vicinity of the mixing plane. The very tight control over the O-meshes on the rotor and stator is clearly shown.

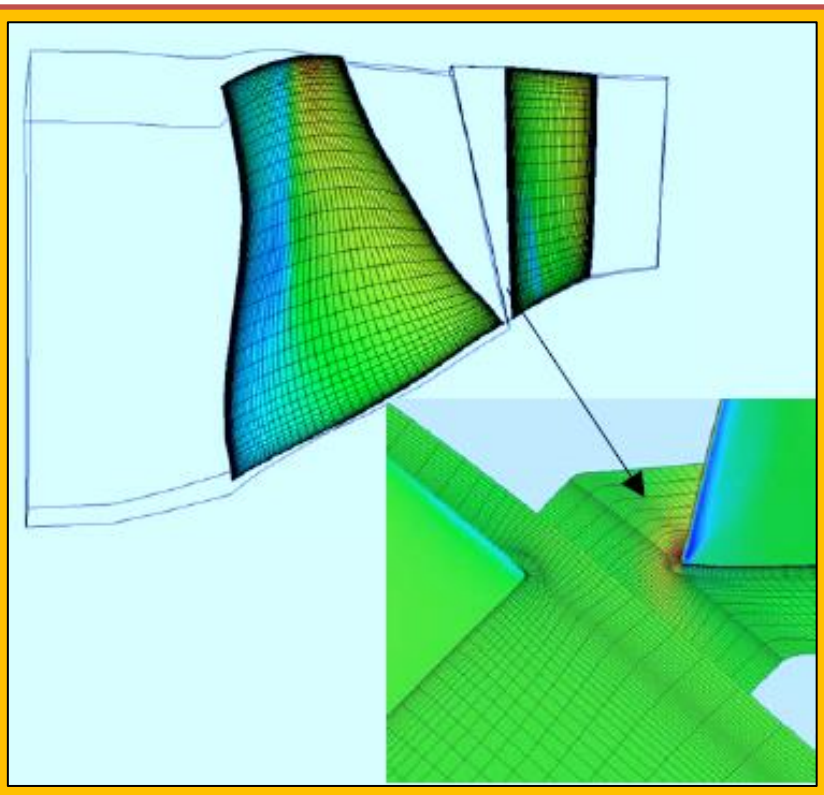


Figure 3.5.18 PADRAM mesh for a single stage fan with rotor TE and stator LE detail near the hub

3.5.5.2 Multi-Stage Research Compressor

PADRAM has also been used to generate meshes for multistage compressors and turbines. **Figure 3.5.19** shows the CFD solution from a *PADRAM* mesh for a 5 stage research compressor. Of interest is the fact that the 2nd, 3rd and 4th stage stators are cantilevered and the *PADRAM* tip gap model has been successfully applied to both stator hubs and rotor casings.

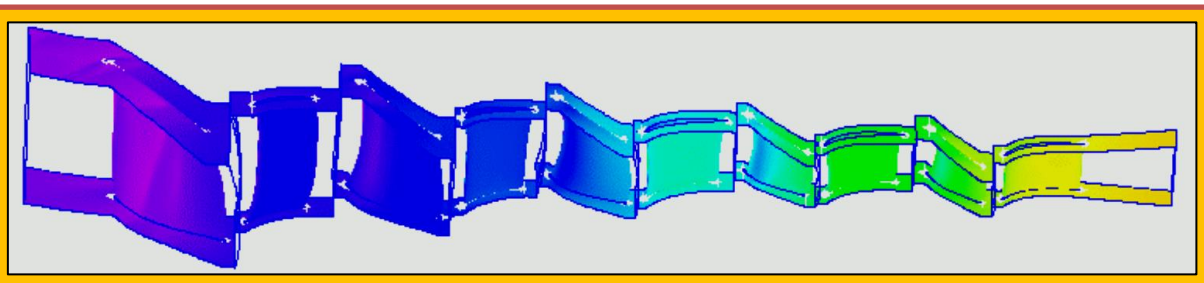


Figure 3.5.19 PADRAM mesh for a 5 stage compressor, showing contours of static pressure

3.5.5.3 Waterjet Impeller

Figure 3.5.20 shows a *PADRAM* mesh for a 6-bladed waterjet impeller. This is a challenging case to mesh since there is a considerable change in the geometry from hub to tip; and, the radius changes significantly from leading to trailing edge. The geometry has been represented using the same style of blade geometry definition as used above and so all of the blade design parameters in *PADRAM* are available for a redesign of this component.

3.5.6 Mesh Quality

The need for the meshing system to be both rapid and based on parametric geometry has been described above. For it to be used successfully within an optimization cycle it must also be robust when presented with large geometry perturbations. This is particularly likely to occur with heuristic optimizers. The robustness of the mesh generator must also be matched by the robustness of the CFD solver. For example, as the optimizer tries to push up incidence angle, the CFD solver must respond with a realistic loss loop in order that the optimizer converges to a design with *acceptable* incidence.

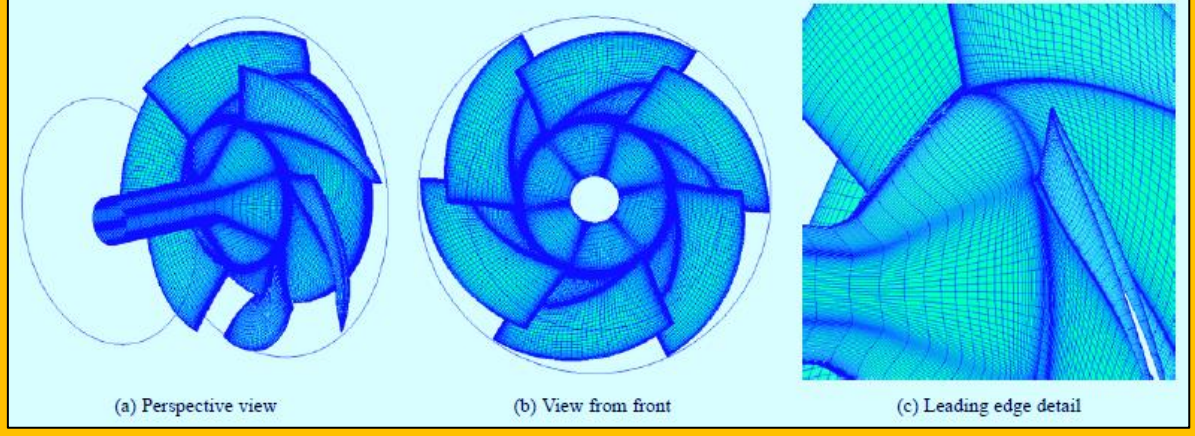


Figure 3.5.20 PADRAM mesh for a 6 bladed waterjet

In this section, two examples of the importance of mesh quality within the design system are presented.

3.5.6.1 CFD Solver

The CFD solver used with the *PADRAM* system is the *Rolls-Royce plc. HYDRA*. *HYDRA* is a suite of non-linear, linear and adjoint CFD solvers developed collaboratively by Rolls-Royce plc. and its University partners. *HYDRA* represents the *PADRAM* meshes as hybrid unstructured meshes using an efficient edge-based data structure [24]. Convergence to steady state is accelerated through the use of an element collapsing multigrid algorithm [25]. The *HYDRA* solver has been parallelized using the domain decomposition method and runs efficiently on both shared and distributed memory computers. The parallel multigrid approach is essential to generating CFD solutions in the elapsed time needed for effective use within a design/optimization system.

3.5.6.2 Leading Edge Separations

It is almost inevitable in an optimization cycle that the sensitivity to incidence will be explored. Here, both the mesh topology and CFD solution must respond physically to changes in incidence. **Figure 3.5.21** shows the Mach number contours near the leading-edge of a high-pressure compressor stator. A relatively coarse grid was used in this study. It has been found that the boundary-layer is better represented by the *PADRAM* mesh than the letter-box style mesh. The *corner grid points* in the letter-box mesh, near the blade leading-edge, thicken the boundary-layer

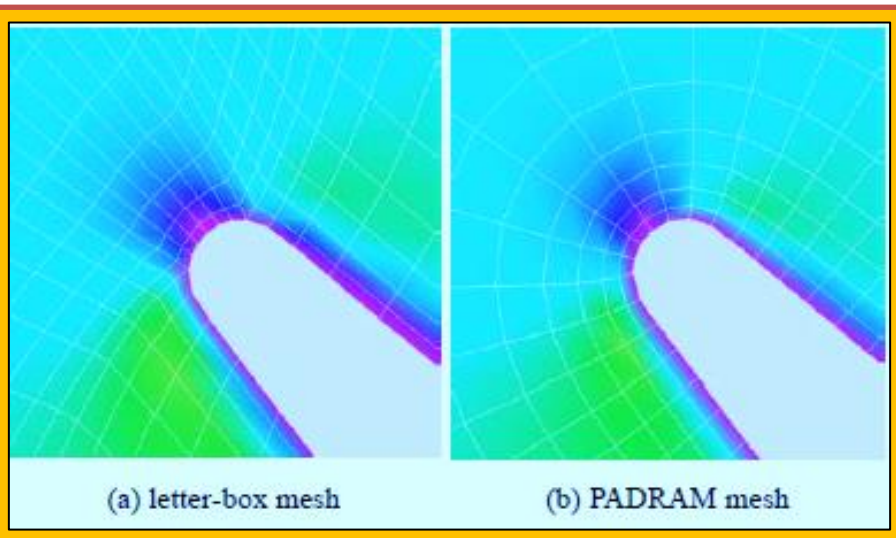


Figure 3.5.21 HYDRA solution for a compressor blade – contours of Mach number shown

leading to excessive thickening and separation of the boundary-layer downstream. The *PADRAM* mesh gives much more reasonable leading edge behavior – due to the fact that the O-mesh around the blade does not contain corner points. The *PADRAM* mesh does contain singularities in the mesh, where the O- and H-meshes meet, but these are away from the high gradient regions at the leading and trailing edges.

Both meshes in **Figure 3.5.21** have a relatively coarse resolution of the leading edge and it is likely that the consistency of the solution of the 2 meshes will improve as the mesh gets finer. However, the successful application of an optimizer is often a judicious choice between the level of mesh resolution for rapid run times and that needed for good quality CFD solutions. Care in needed to ensure good quality mesh is used in the area of high-gradient flow regions.

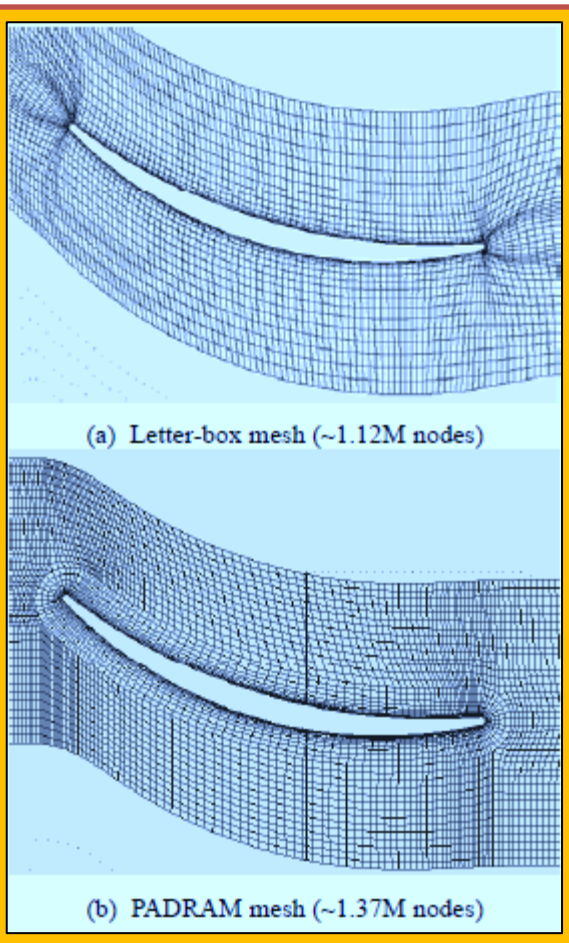


Figure 3.5.22 Letter-box and PADRAM meshes for the bypass OGV noise calculation

3.5.6.3 Bypass OGV Noise Simulation

PADRAM has been used to generate suitable meshes for the CFD analysis of a representative Fan/bypass OGV interaction. The OGV noise generation calculation uses the linearized unsteady *HYDRA* solver, with the incoming wake amplitude and phase defined from a separate non-linear steady *HYDRA* calculation. Initial calculations for the bypass OGV used a letter box style mesh. **Figure 3.5.22 a** shows the letterbox mesh that was sized to 1.12M to maintain the propagation of the expected acoustic waves at 2BPF (blade passing frequency) and 3BPF in the linear unsteady calculation. With this mesh the non-linear steady *HYDRA* calculation yielded a substantial pressure-surface flow separation. The problem was traced to excessive thickening of the leading edge boundary layer due to the distorted cells in that region of the letterbox mesh. This was a case run at part shaft speed with significant negative incidence onto the OGV. For the linear unsteady calculation the separated flow led to substantial flow instability and very high unsteady blade response to the incoming wake.

Figure 3.5.22 b shows the alternative *PADRAM* O-H mesh generated for this OGV, which has 1.37M nodes. With this mesh the steady flow was now well attached on the OGV and the linear unsteady blade response and resulting noise generation were much reduced. **Figure 3.5.23** illustrates the levels of unsteady velocity response for the two mesh styles. **Figure 3.5.24** compares the amplitude of the predicted midspan unsteady pressure in 2BPF for the two mesh styles and illustrates the reductions obtained with the *PADRAM* mesh. Using linear *HYDRA* and the *PADRAM* system the effects of different numbers off (65 vs. 58) and different OGV geometries are being studied.

3.5.7 Conclusions

A flexible, integrated multi-passage blading design system is presented for inclusion in an automatic optimization cycle using high fidelity CFD codes. Examples are included in this paper that illustrate the range of capabilities of the new parametric design and rapid meshing system (*PADRAM*) to modify blade's geometry and configuration and to generate a good quality viscous mesh very rapidly.

PADRAM can be used in both design and analysis of 2D, 3D, single-passage, multi passage, full annulus, multi-stage environment.

Acknowledgments

The authors would like to acknowledge the permission of Rolls-Royce plc to publish this paper. This work was performed under the UK DTI CARAD Programmed (Contract No. CKBB/C/016/00029). The bypass OGV noise simulation was performed under the European Union 5th framework project TurboNoiseCFD. The authors also wish to thank their colleague Mr John Coupland for useful discussions.

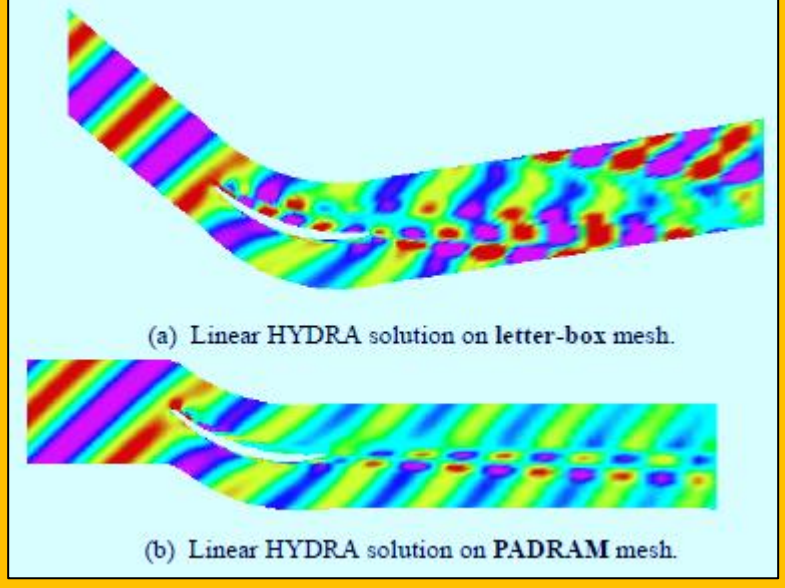


Figure 3.5.23 Bypass OGV Instantaneous Axial Velocity Perturbation (-8m/s purple to +8m/s red) at 2BPF

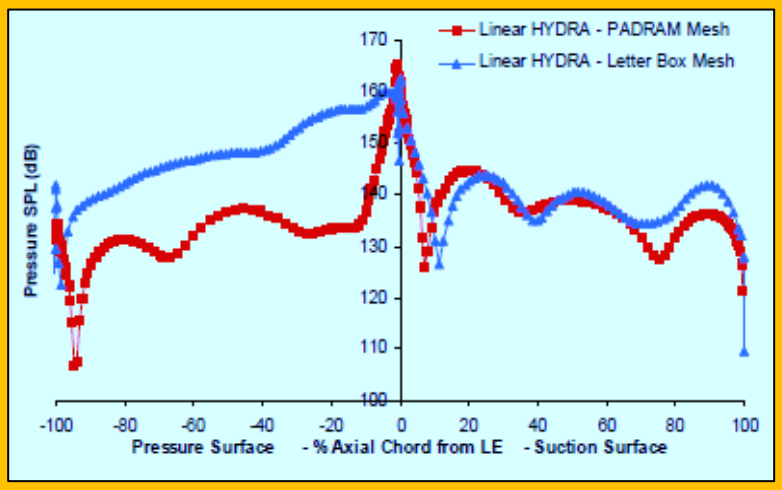


Figure 3.5.24 Bypass OGV Unsteady Surface Pressure Response in 2BPF

3.5.8 References

[1] Shahpar, S., 2001, "Three-dimensional Design and Optimization of Turbomachinery Blades using the Navier-Stokes Equations", ISABE 2001-1053, *proceedings of 15th ISABE conference*, India.

[2] Shahpar, S. Giacchi, D., and Lapworth, L., 2003, "Multi-objective Design and Optimization off Bypass Outlet-guide Vanes", ASME paper GT2003-38700, Atlanta.

[3] Shaw, J.A. and Weatherill, N.P., 1992, "Automatic topology generation for multiblock grids", *Applied Mathematics and Computation*, 53, pp 335-388.

[4] Dannenhoffer, J.F., 1993, "A New method for creating grid abstractions for complex configurations", AIAA paper 93-0428.

[5] Lohner, R., and Parikh, P., 1988, "Generation of three dimensional unstructured grids by the advancing front method", AIAA paper 88-0515.

[6] Baker, T.J., 1995, "Prospects and expectations for unstructured methods", Proceedings of the Surface *Modelling, Grid Generation and Related Issues in Computational Fluid Dynamics Workshop*, *NASA-CP-3291*.

[7] Steger, J., Dougherty, F.C., and Benek, J., 1983, "A chimera grid scheme", *Advances in Grid Generation*, Ghia, K.N. and Ghia, U., Eds., ASME FED, 5, pp 59-69.

[8] Benek, J.A., Buning, P.G., and Steger, J.L., 1985, "A 3-D Chimera Grid Embedding Technique", AIAA Paper 85-1523.
[9] Kao, K.H., Liou, M.S., and Chow, C.Y., 1994, "Grid Adaptation using chimera composite overlapping meshes", *AIAA Journal*, 32, pp 942-949.
[10] Kallinderis, Y., Khawaja, A., and McMorris, H., 1996, "Hybrid prismatic/tetrahedral grid generation for viscous flows around complex geometries", *AIAA Journal*, 34, pp 291-298.
[11] Irmisch, S., Walker, D., Bettelini, M., Haselbacher, A., and Benz, E., and Kallinderis, Y., 1999, "Efficient Use of Hybrid Grids in Modern Turbomachinery Applications", AIAA-99-0783.
[12] Aftomis, M.J., Melton, J.E., and Berger, M.J., 1995, "Adaptation and surface modeling for Cartesian mesh methods", AIAA-95-1725-CP.
[13] Thompson, J.F., Soni, B.k., Weatherill, N.P., 1999, "Handbook of Grid Generation", *CRC Press*.
[14] Jones, W.T., Samarah, J., 1995, "A Grid Generation System For Multi-Disciplinary Design Optimization", *AIAA-95-1689.*
[15] Dawes, W.N., 1986, "A Numerical Method for the Analysis of 3D Viscous Compressible Flow in a Turbine Cascade; Application to Secondary Flow in a Cascade with and without Dihedral", *ASME Paper 86-GT-145*.
[16] Moore, J.G., and Moore, J., 1990, "The Moore Elliptic Flow Program for Turbomachinery Flow Calculations – 1990 User Guide", *VPI & SU Turbomachinery Research Group Report No. JM/90-3*.
[17] Hall, E.J. and Delaney, R.A., 1995, "Investigation of Advanced Counter Rotation Blade Configuration Concepts for High Speed Turboprop Systems: Task VII – ADPAC User's Manual, *NASA Contract NAS3-25270, NASA CR-195472*.
[18] Sbardella, L., Sayma, A.I., and Imergun, M., 2000, "Semi-Unstructured Mesh Generator for Flow Calculations in Axial Turbomachinery Blading", *Int. Journal for Numerical Methods in Fluids,* 32, no. 5, pp 569-584.
[19] Wadia, A.R., Szucs, P.N., and Gundy-Burlet, K.L., 1999, "Design and Testing of Swept And Leaned Outlet Guide Vanes to Reduce Stator-Strut-Splitter Aerodynamic Flow Interactions", *Transactions of the ASME*, 121, pp416-425.
[20] Tschirner, T., Pfitzner, M., Mertz, R., "Aerodynamic Optimization of An Aeroengine Bypass Duct OGV-Pylon Configuration", ASME GT-2002-30493, Netherlands, 2002.
[21] Harvey, N.W., Rose, M.G., Taylor, M.D., Shahpar, S. and Gregory-Smith, D.G., 2000, "Non-axisymmetric Turbine End wall Design, Part I -3D Linear Design system*", Journal of Turbomachinery*, 122, number 2, pp286-294.
[22] Parry, A.B., 2002, *private communication*, Fan System Engineering, Rolls-Royce plc.
[23] Van Zante, D.E. *et al.,* 2000, "Recommendations for Achieving Accurate Numerical Simulation of Tip Clearance Flows in Transonic Compressor Rotors", *Journal of Turbomachinery,* 122, pp733-742.
[24] Moinier, P., and Giles, M.B., 1998, "Edge-Based Multigrid Schemes for Hybrid Grids", *6th ICFD Conference on Numerical Methods for Fluid Dynamics,* Oxford, UK.
[25] Muller, J-D and Giles, M.B., 1998, "Edge-Based Multigrid Schemes for Hybrid Grids", *6th ICFD Conference on Numerical Methods for Fluid Dynamics,* Oxford, UK.

3.6 Hyperbolic Schemes

This grid generation scheme is generally applicable to problems with open domains consistent with the type of PDE describing the physical problem. The advantage associated with Hyperbolic PDEs is that the governing equations need to be solved only once for generating grid. The initial point distribution along with the approximate boundary conditions forms the required input and the solution is the then marched outward. Steger and Sorenson[74] proposed a volume orthogonality method that uses Hyperbolic PDEs for mesh generation. For a 2D problem, considering computational space to be given by $\Delta\xi = \Delta\eta = 1$, the inverse of the Jacobian is given by,

$$x_\xi y_\eta - x_\eta y_\xi = I$$

Eq. 3.6.1

Where I represents the area in physical space for a given area in computational space. The second equation links the orthogonality of grid lines at the boundary in physical space which can be written as

$$d\xi = 0 = \xi_x dx + \xi_y dy$$

Eq. 3.6.2

For ξ and η surfaces to be perpendicular the equation becomes:

$$x_\xi y_\eta + y_\xi y_\eta = 0$$

Eq. 3.6.3

The problem associated with such system of equations is the specification of I. Poor selection of I may lead to shock and discontinuous propagation of this information throughout the mesh. While mesh being orthogonal is generated very rapidly which comes out as an advantage with this method. **Figure 3.6.1** displays a C-O type hyperbolic grid around an HSCT wing-fuselage configuration, with pressure contours mapped using an Euler solution and $M_\infty = 2.4$.

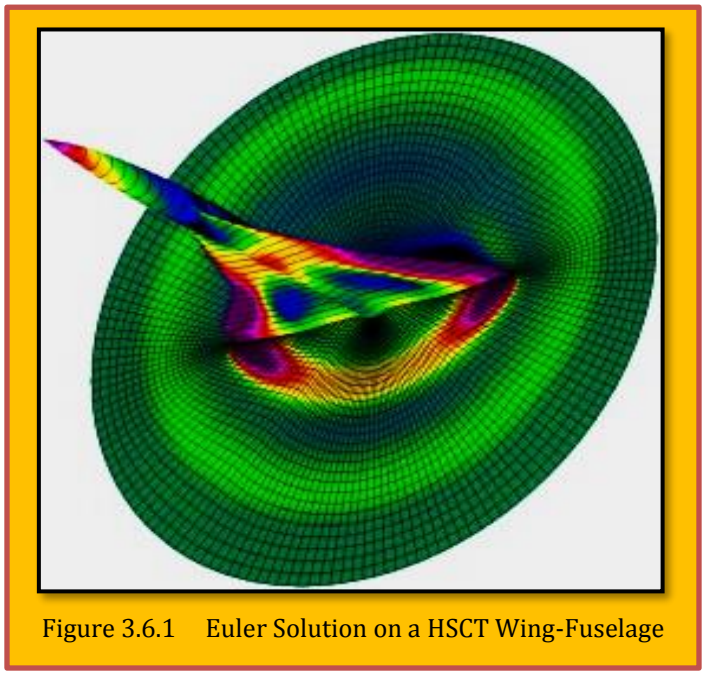


Figure 3.6.1 Euler Solution on a HSCT Wing-Fuselage

3.7 Parabolic Schemes

The solving technique is similar to that of hyperbolic PDEs by advancing the solution away from the initial data surface satisfying the boundary conditions at the end. Nakamura (1982) and Edwards (1985) developed the basic ideas for parabolic grid generation. The idea uses either of Laplace or the Poisson's equation and especially treating the parts which controls elliptic behavior. The initial values are given as the coordinates of the point along the surface $\eta = 0$ and the advancing the solutions to the outer surface of the object satisfying the boundary conditions along ξ edges. The control of the grid spacing has not been suggested till now. Nakamura and Edwards, grid control was accomplished using non-uniform spacing. The parabolic grid generation shows an advantage over the hyperbolic grid generation that, no shocks or discontinuities occur and the grid is relatively smooth. The specifications of initial values

[74] *Steger, J.L; Sorenson, R.L (1980). "Use of hyperbolic partial differential equation to generate body fitted coordinates, Numerical Grid Generation Techniques". NASA conference publication 2166: 463–478.*

and selection of step size to control the grid points is however time consuming, but these techniques can be effective when familiarity and experience is gained.

3.8 Variational Method

These methods have evolved from elliptic grid generation. To solve an elliptic PDE is often equivalent to minimizing a functional. Variational methods has been used for improving quality of a given grid[75]. In the vibrational methods, a grid functional is defined. Grid functional is an algebraic expression of the position vectors of the internal nodes of a mesh. Optimization of the grid functional may result in a grid with desired properties such as orthogonal grid lines, equal cell areas, linear or parallelogram cells and untangled mesh. There are many algebraic functional for grid generation and optimization. For example, algebraic grid generation methods such as ***Transfinite Interpolations*** though, one of simplest method of grid generation, but can produce folded grids for curved domains as seen in the **Figure 3.8.1**. One other disadvantage of algebraic grid generation is that boundary discontinuity can prorogate inside the domain. As indicated in **Figure 3.8.1**, ***Winslow functional*** smooth the grid, and removes the folded grid lines. There are far too many algebraic functional for grid generation optimization as reader should check with[76].

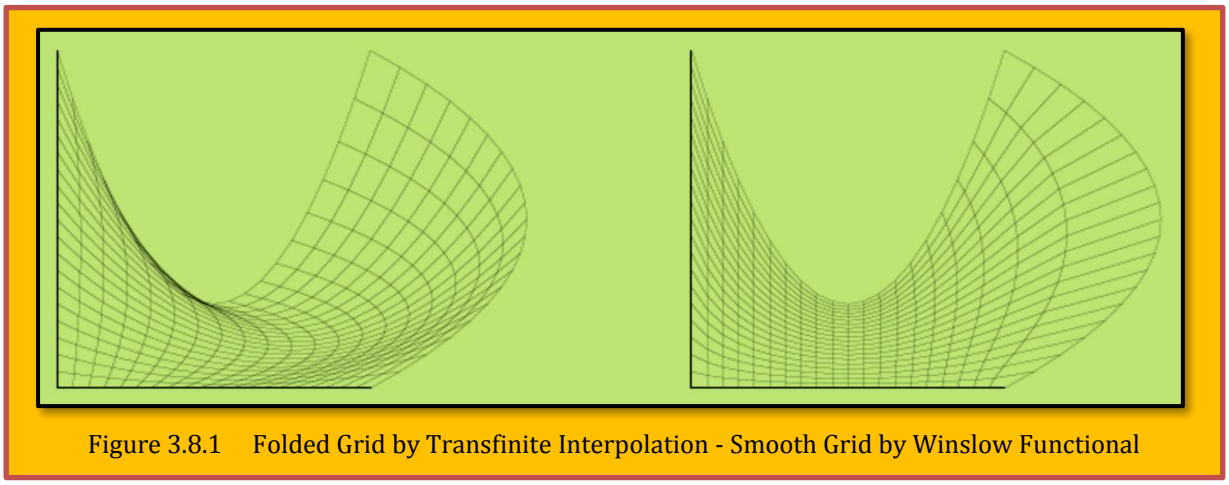


Figure 3.8.1 Folded Grid by Transfinite Interpolation - Smooth Grid by Winslow Functional

3.8.1 Variational High-Order Mesh Generation [77]

A literature review into ***high-order mesh generation***, such as ***vibrational methods***, has almost exclusively centered around a ***posteriori approaches***, whereby a coarse linear mesh is deformed to accommodate the curvature at the boundary. The challenge in this approach is to determine a method through which this curvature can be incorporated into the interior of the domain. Without this, the mesh is at best of a low quality, and at worst, will self-intersect (**Figure 3.8.1** – left), rendering it unsuitable for solver based calculations. Existing work in a posteriori generation has broadly centered around two lines of investigation.

The first of these focuses around the concept of solid body deformation, whereby the mesh is treated as a solid body which is deformed to incorporate curvature at the boundary. The work in this theme has focused around determining which model is *`best'*, either in terms of optimal quality or computational efficiency. Some models investigated include ***linear elasticity***, ***non-linear hyperelasticity*** and more recently, ***thermo-elasticity*** and the ***Winslow equations***.

The second theme follows a different route, whereby the mesh is equipped with an associated functional that denotes its energy. A non-linear optimization problem is then solved in order to minimize this functional and yield a valid mesh. Again, most studies in this area have focused around

---

[75] J.F. Thompson, B.K. Soni and N.P. Weatherill. Handbook of Grid Generation. CRC Press, 1998.

[76] Sanjay Kumar Khattri, "*Grid Generation and Adaptation by Functional*", Department of Mathematics, University of Bergen, Norway.

[77] Michael Turner, "*High-Order Mesh Generation For CFD Solvers*", Imperial College London, Faculty of Engineering Department of Aeronautics, A thesis submitted for the Degree of Doctor of Philosophy, 2017.

this choice of functional, which include ***scaled Jacobian distortion metrics spring analogies*** for surface deformation, ***unconstrained optimization of the Jacobian*** and a number of articles based on a ***shape distortion metric***. However, the connections between these two themes so far remain uninvestigated.
In the linear mesh generation community, it is known that through the calculus of variations, the elliptic partial differential equations defining these elasticity models can be recast into the minimization of an energy functional, which takes as its arguments the mesh displacement and its derivatives. However, the use of this approach in high-order mesh generation has remained mostly unnoted, asides from brief remarks.

3.9 Adaptive Structured Mesh

In an adaptive grid, the physics of the problem at hand must ultimately direct the grid points to distribute themselves so that a functional relationship on these points can represent the physical solution with sufficient accuracy[78]. ***The idea is to have the grid point's move as the physical solution develops, concentrating in regions of large variation in the solution as they emerge***. The mathematics controls the points by sensing the gradients in the evolving physical solution, evaluating the accuracy of the discrete representation of the solution, communicating the needs of the physics to the points, and finally by providing mutual communication among the points as they respond to the physics. The basic techniques involved then are as follows:

- A means of distributing points over the field in an orderly fashion, so that neighbors may be easily identified and data can be stored and handled efficiently.
- A means of communication between points so that a smooth distribution is maintained as points shift their position.
- A means of representing continuous functions by discrete values on a collection of points with sufficient accuracy, and a means for evaluation of the error in this representation.
- A means for communicating the need for a redistribution of points in the light of the error evaluation, and a means of controlling this redistribution.

Several considerations are involved here, some of which are conflicting. The points must concentrate, and yet no region can be allowed to become devoid of points. The distribution also must retain a sufficient degree of smoothness, and the grid must not become too skewed, else the truncation error will be increased as noted. This means that points must not move independently, but rather each point must somehow be coupled at least to its neighbors. Also, the grid points must not move too far or too fast, else oscillations may occur. Finally the solution error, or other driving measure, must be sensed, and there must be a mechanism for translating this into motion of the grid. The need for a mutual influence among the points calls to mind either some elliptic system, thinking continuously, of some sort of attraction (repulsion) between points, thinking discretely. Both approaches have been taken with some success, and both are discussed below. ***It should be noted that the use of an adaptive grid may not necessarily increase the computer time, even though more computations are necessary, since convergence properties of the solution may be improved, and certainly fewer points will be required***. With the time derivatives at fixed values of the physical coordinates transformed to time derivatives taken at fixed values of the curvilinear coordinates, no interpolation is required when the adaptive grid moves. Thus the first derivative transformation given the chain rule is given by

[78] Joe F. Thompson, J., F., Warsi, Z., U., A., Mastin, .W. "*Numerical Grid Generation; Foundations and Applications*", North-Holland Book, 1995.

$$\left(\frac{\partial A}{\partial t}\right)_{\xi,\eta,\zeta} = \left(\frac{\partial A}{\partial t}\right)_{x,y,z} + \nabla A.\left(\frac{\partial x}{\partial t}\right)_{\xi,\eta,\zeta} \quad \text{or}$$

$$\left(\frac{\partial A}{\partial t}\right)_{\xi,\eta,\zeta} = \left(\frac{\partial A}{\partial t}\right)_{x,y,z} + \sum_{i=1}^{3} \frac{\partial A}{\partial x_i} \underbrace{\left(\frac{\partial x_i}{\partial t}\right)}_{\text{mesh movement}}$$

Eq. 3.9.1

The computation thus can be done on a fixed grid in the transformed space, without need of interpolation, even though the grid points are in motion in physical space. The influence of the motion of the grid points registered through the grid speeds, $(x_i)_t$, appearing in the transformed time derivative. This is the appropriate approach when the grid evolves with the solution at each time step. Some methods, however, change the grid only at selected time steps, and here interpolation must be used to transfer the values from the old grid to the new since the grid movement is not continuous. A combination of the weight functions given by **Eq. 3.9.1** provides the desired tendency toward concentration both in regions of high gradient and near extrema. The effect of the inclusion of the curvature illustrated below:

$$w = (1 + \beta^2|K|)\left(1 + \alpha^2\left(\frac{\partial A}{\partial x}\right)^2\right)^{1/2} \quad \text{where} \quad K = \frac{\frac{\partial^2 A}{\partial x^2}}{\left(1 + \left(\frac{\partial A}{\partial x}\right)^2\right)^{3/2}}$$

Eq. 3.9.2

Where α and β are parameters to be specified. Clearly, concentration near high gradients is emphasized by large values of α, while concentration near extrema (or other regions of large curvature) is emphasized by large β. (**Figure 3.9.1**).

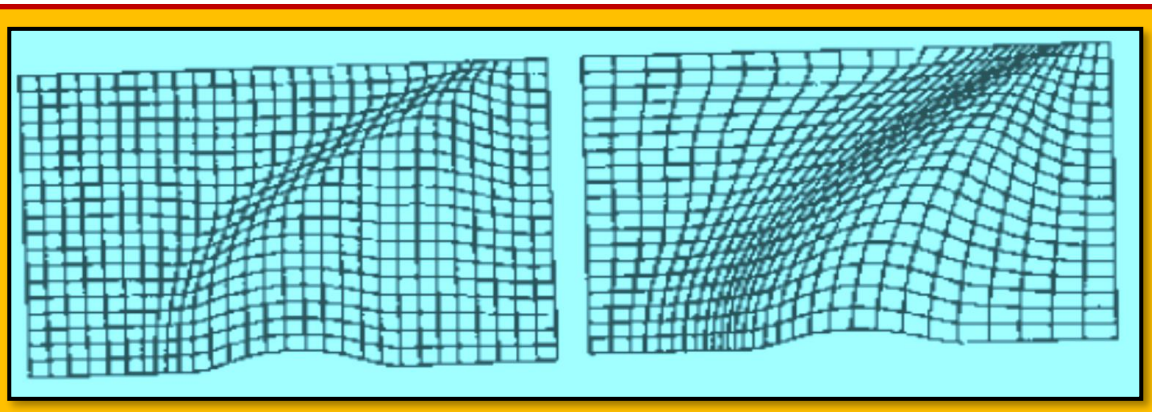


Figure 3.9.1 1D Weight Function for High Gradient and Curvature

3.9.1 Case Study – 2D Euler Flow Over an NACA Airfoil

In a grid adaptation method for structured grids without adding or removing grid points, adaptation is achieved through moving the grid points toward the desired locations. Changes in the mesh point locations can be controlled by two methods (Liu et. al.)[79]. In the first method, the arc elements forming the ides of a control volume are directly related to specified functions. For a three-dimensional problem, this implies that three arc elements need to be given. In the second method, the cell volume may be altered by specifying that the volume of each element change according to a specific rule. To control the cell size, only one relationship must be specified that relates the volume to the quantity responsible for changes in the mesh. The specification of only one control function is an advantage in simplicity but may be less flexible than independently controlling arc lengths. The cell volume control method is applied successfully to calculating transonic Euler flows with shock waves. The method is applied to computing the flow field over an airfoil. **Figure 3.9.2** shows the

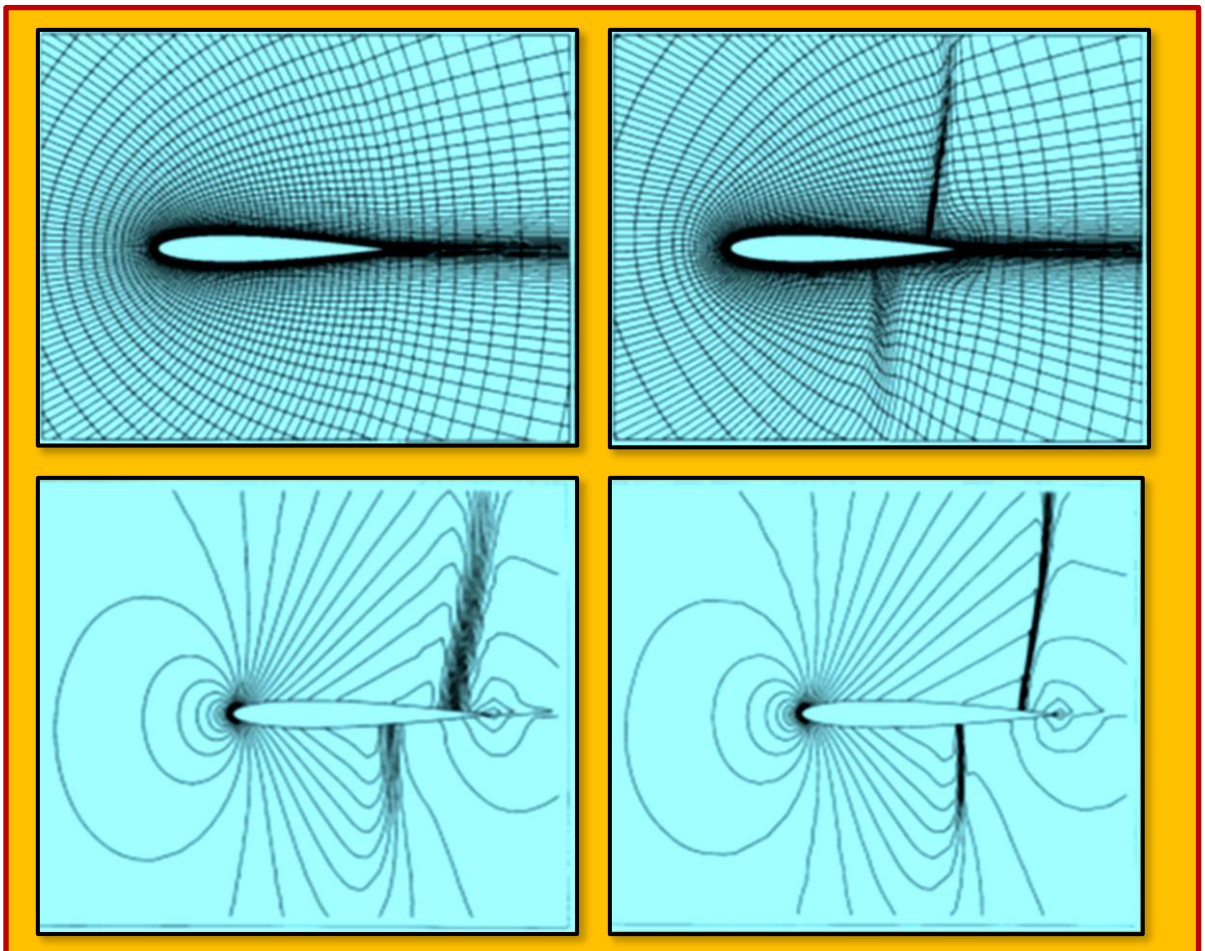


Figure 3.9.2 Mesh and Mach Contours for Transonic Flow

initial C-mesh of an NACA 0012 airfoil and the adaptive one on the right, with their respective Mach contours. Two flow cases were calculated over the initial grid.

[79] Feng Liu, Shanhong Ji, and Guojun Liao,*" An Adaptive Grid Method and Its Application to Steady Euler Flow Calculations",* Siam J. Sci. Comput. C° 1998 Society For Industrial And Applied Mathematics Vol. 20, No. 3.

3.9.1.1 Transonic Case

The first is a ***transonic case*** with free stream Mach number, $M_\infty = 0.85$ and an angle of attack, $\alpha = 1$ where Left is without mesh adaption and Right is with. There is one strong shock wave on the upper side of the airfoil and a weaker one on the lower side. It can be seen from **Figure 3.9.2** that the computed shock waves are rather thick. Shock waves zero thickness in the inviscid limit. To get better computational results, particularly to capture the shock waves more accurately, one would like to concentrate grid points around the shock waves. The deformation method is applied to get a new grid with prescribed distribution of cell sizes based on gradients of the flow field. The adaptive criterion here is to detect the shock waves. This suggests choosing the monitor function f of the form:

$$\frac{1}{f} = C_1(1 + C_2 \nabla P) \qquad \textbf{Eq. 3.9.3}$$

Where P is the pressure and $C_1$and C_2 are constants. It can be seen that grid points are clustered closely in the areas where the two shock waves occur, although grid lines are somewhat skewed in the clustered regions because the deformation method does not guarantee orthogonality. However, since our flow solver is based on a finite volume scheme which does not require the use of an orthogonal grid, we are content with the locally reduced cell sizes.

3.9.1.2 Supersonic Case

Another test case is the supersonic flow over the same airfoil with a free stream Mach number $M_\infty = 1.5$, and $\alpha = 0$. As can be seen, a strong bow shock wave appears in front of the airfoil leading edge. In addition, there are two weak shocks emanating from the trailing edge of the airfoil the Mach

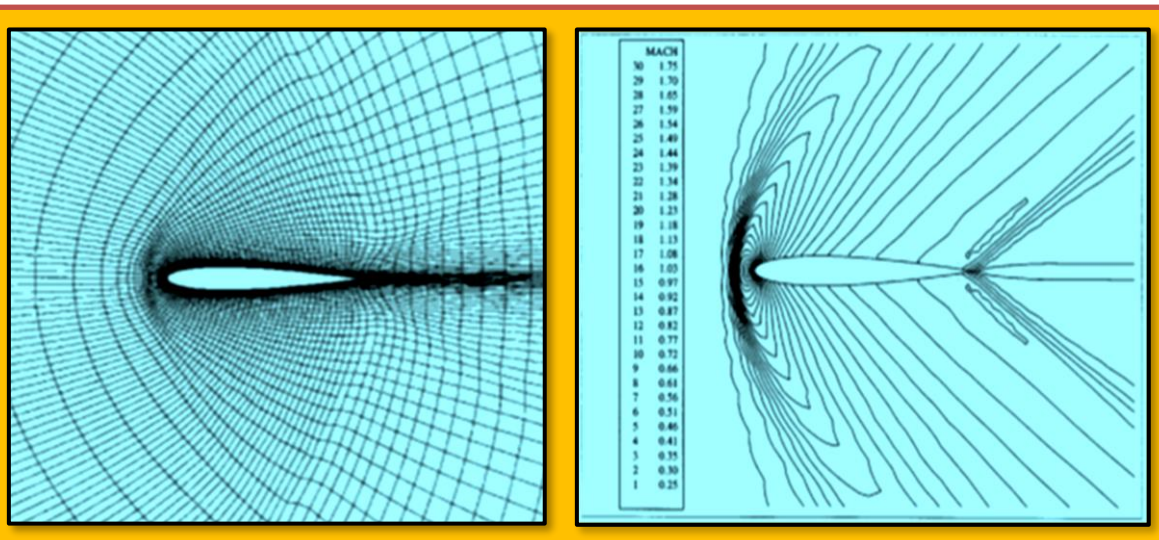
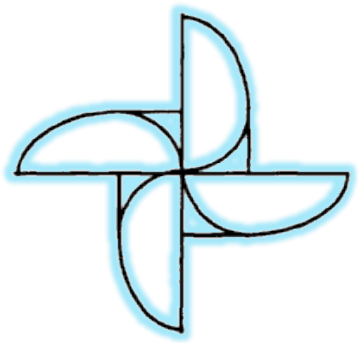


Figure 3.9.3 Grid Adaption and Mack Contours for Supersonic Airfoil

number distribution computed on the adapted grid. It can be seen that a sharper front of the bow shock is captured compared with that on the initial un-adapted grid. The resolution of the two trailing edge shocks is also slightly increased. The computational time needed for the grid adaptation and the flow solver for the supersonic case is the same as that for the transonic case. (**Figure 3.9.3**).



4 Appendix A

A.1 Computer Code for a Transfinite Interpolation

This subroutine is based on a transfinite interpolation with a Lagrangian blending function. The following section describes the subroutine arguments.

Nomenclature

F : Grid position (x, y, or z)

IL, JL : Number of grid points in i and j-directions, respectively.

II(i), JJ(j) : This array stores the locations of known grid lines in i- and j-directions, respectively (1 for known grid lines).

IS, IE, JS, JE : Starting and ending of region (computational) of interest.

IMAX, JMAX : Array dimension

Example: Consider surface IV in Figure 0.1. In this case, five grid lines are known: two lines at GH, two lines at GJLN, and one line at NO. The size of the grid is 95 in the I-direction and 50 in the J-direction. Point G is at (70, 15) and point O is at (95, 25).

```
IL=95
JL=50
II(i)=0 except i=69, 70, and 95,
JJ(j)=0 except j=14, and 15,
IS=70
IE=95
JS=15
JE=25
IMAX=95
```

```
JMAX=95
CALL TRANS2D(F,IL,JL,II,JJ,IS,IE,JS,JE,
                                IMAX,JMAX)

C
      SUBROUTINE TRANS2D(F,IL,JL,II,JJ,IS,IE,JS,JE,IMAX,JMAX)
      PARAMETER(NMAX=95)
      DIMENSION F(IMAX,JMAX),II(NMAX),JJ(NMAX)
      DIMENSION PH(NMAX,NMAX),SI(NMAX,NMAX),III(NMAX),JJJ(NMAX)
C
      IF(IL.GT.NMAX.OR.JL.GT.NMAX) THEN
        WRITE(*,*)  'THE PARAMETERS ARE SMALL, STOP IN SUB TRANS2D'
          STOP
      END IF
      NM=0
      DO 100 I=1,IL
         IF(II(I).NE.0) THEN
           NM=NM+1
           III(NM)=I
         END IF
 100  CONTINUE
C
      MM=0
      DO 110 J=1,JL
         IF(JJ(J).NE.0) THEN
           MM=MM+1
           JJJ(MM)=J
         END IF
```

```
 110  CONTINUE
C
C.....SETUP THE BLENDING FUNCTIONS (LAGRANGIAN INTERPOLATION)
C
      DO 200 N=1,NM
         DO 210 I=1,IL
            PH(N,I)=1.0
            DO 220 N1=1,NM
               IF(N1.EQ.N) GOTO 220
               PH(N,I)=PH(N,I)*FLOAT(I-III(N1))/FLOAT(III(N)-III(N1))
 220        CONTINUE
 210     CONTINUE
 200  CONTINUE
C
      DO 300 M=1,MM
         DO 310 J=1,JL
            SI(M,J)=1.0
            DO 320 M1=1,MM
               IF(M1.EQ.M) GOTO 320
               SI(M,J)=SI(M,J)*FLOAT(J-JJJ(M1))/FLOAT(JJJ(M)-JJJ(M1))
 320        CONTINUE
 310     CONTINUE
 300  CONTINUE
C
C.....COMPUTE THE TRANSFINITE INTERPOLATION
C
      DO 1000 J=JS,JE
```

```
         IF(JJ(J).NE.0) GOTO 1000
         DO 1100 I=IS,IE
            IF(II(I).NE.0) GOTO 1100
C
            F(I,J)=0.
C
            DO 1200 N=1,NM
               F(I,J)=F(I,J)+PH(N,I)*F(III(N),J)
 1200       CONTINUE
C
            DO 1300 M=1,MM
               F(I,J)=F(I,J)+SI(M,J)*F(I,JJJ(M))
 1300       CONTINUE
C
            DO 1400 N=1,NM
               DO 1500 M=1,MM
                  F(I,J)=F(I,J)-PH(N,I)*SI(M,J)*F(III(N),JJJ(M))
 1500          CONTINUE
 1400       CONTINUE
 1100    CONTINUE
 1000 CONTINUE
C
      RETURN
      END
```

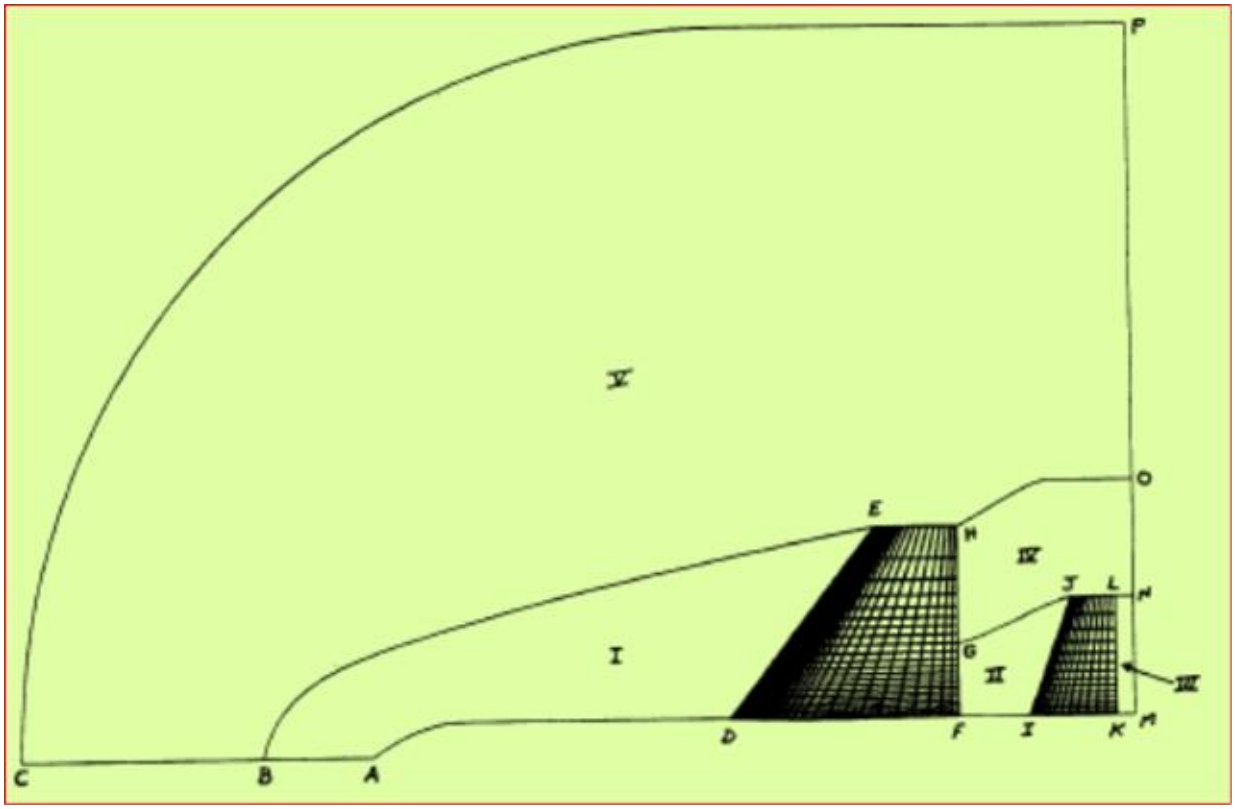


Figure 0.1 Symmetry plane (XY)